

密 级： 公开

加密论文编号：

论文题目： 面向云原生应用权限提升漏洞的智能化
防控技术研究

学 号： 论文作者学号

研 究 生： 本论文作者

专 业 名 称： 计算机科学与技术

2026 年 5 月 1 日

面向云原生应用权限提升漏洞的智能化防控技术研究

**Research on Dynamic Defense and Monitoring
Technologies for Privilege Escalation
Vulnerabilities in Cloud-Native Applications**

研究生：本论文作者

指导教师：本论文导师

北京科技大学 计算机与通信工程学院

北京 100083，中国

Doctor Degree Candidate：本论文作者

Supervisor: 本论文导师

School of Computer and Communication Engineering

University of Science and Technology Beijing

30 Xueyuan Road, Haidian District

Beijing 100083, P.R.CHINA

中图分类号:

学校代码:

10008

U D C:

密 级:

公开

北京科技大学硕士学位论文

论文题目: 面向云原生应用权限提升漏洞的智能化防控技术研究

研究生: _____ 本论文作者

指 导 教 师: 本论文导师 单位: 北京科技大学 职称: 教授

指 导 小 组 成 员: _____ 单位: _____ 职称: _____

_____ 单位: _____ 职称: _____

论文提交日期: 2026 年 5 月 1 日

学位授予单位: 北 京 科 技 大 学

摘 要

Kubernetes 生态中的第三方云原生应用存在可导致权限提升的高危漏洞。该类漏洞在披露至补丁落地的窗口期内，权限提升攻击风险可威胁 Kubernetes 集群所有节点。现有研究侧重对配置文件的静态风险分析，难以围绕具体漏洞快速生成可落地的临时防控策略；Kubernetes 现有安全工具则依赖专家经验进行策略制定，无法针对特定漏洞进行有效防控。

本文基于大语言模型的漏洞语义理解与代码生成能力，结合运行时监控、策略准入引擎、网络策略等多种安全工具实现对云原生应用权限提升相关漏洞的动态防护与监控，同时开发了支持工具。本文的主要工作如下。

(1) 提种一种基于安全策略生成的智能化漏洞防控方法 KPEDefense: 本文基于大语言模型将漏洞防控问题转化为包含安全工具领域特定语言的安全策略生成问题。首先，基于漏洞来源的开源代码仓库中多源信息构建安全知识库，同时提出了一种权限提升漏洞成因分类框架。然后，搭建由安全上下文构建、思维链推理、反馈优化等步骤组成的多智能体 workflow，针对特定权限提升漏洞防护与监控策略代码的自动生成与迭代优化。最后面向安全策略代码的实际应用需求，设计了覆盖有效性、非干扰性与可部署性的多源模型三维质量量化评估方法。

(2) 开发支持工具: 设计并实现 KPEDefense 方法支持工具，集成知识库管理、策略生成 workflow、专家评审排序与策略部署验证等功能模块，形成面向云原生权限提升漏洞的智能化自动防控流程。

(3) 进行实验验证与案例分析: 将近 5 年权限提升漏洞样本作为实验对象，基于 5 种大语言模型对 KPEDefense 方法进行实验研究。实验结果表明，KPEDefense 方法相较现有规则化风险扫描方法更有效性，且方法不同部分对安全策略有效性均有正向贡献。进一步地，本文对 7 个不同 CVE 进行案例分析，从三个维度进一步验证了 KPEDefense 的实际效果。

关键词: 云原生安全；权限提升漏洞；策略代码生成；补丁窗口期防控；多智能体协同

Research on Dynamic Defense and Monitoring Technologies for Privilege Escalation Vulnerabilities in Cloud-Native Applications

ABSTRACT

Third-party cloud-native applications in the Kubernetes ecosystem contain high-risk vulnerabilities that may lead to privilege escalation. During the window between vulnerability disclosure and patch deployment, the risk of privilege escalation attacks can threaten all nodes in a Kubernetes cluster. Existing research focuses on static risk analysis of configuration files and struggles to rapidly generate actionable interim mitigation strategies for specific vulnerabilities; meanwhile, existing Kubernetes security tools rely on expert experience for policy design and cannot effectively defend against specific vulnerabilities.

Leveraging the vulnerability semantic understanding and code generation capabilities of large language models, and combining runtime monitoring, policy admission engines, network policies, and other security mechanisms, this thesis achieves dynamic protection and monitoring for privilege-escalation-related vulnerabilities in cloud-native applications, along with a supporting prototype system. The main contributions are as follows.

(1) An intelligent vulnerability prevention and control method based on security strategy generation, KPEDefense: This thesis leverages large language models to transform the vulnerability prevention problem into a security strategy generation problem that produces domain-specific security tool code. First, a security knowledge base is constructed from multi-source information in vulnerability-originating open-source repositories, and a privilege escalation vulnerability root-cause classification framework is proposed. Then, a multi-agent workflow comprising security context construction, chain-of-thought reasoning, and feedback optimization is built for the automatic generation and iterative refinement of protection and monitoring strategy code targeting specific privilege escalation vulnerabilities. Finally, a multi-source-model three-dimensional quality quantitative evaluation method covering effectiveness, non-interference, and deployability is designed

to meet the practical requirements of security strategy code.

(2) Supporting prototype system: A KPEDefense support tool is designed and implemented, integrating knowledge-base management, strategy-generation workflow, expert review and ranking, and strategy-deployment verification modules, forming an intelligent and automated prevention and control process for cloud-native privilege escalation vulnerabilities.

(3) Experimental validation and case studies: Privilege escalation vulnerability samples from the past five years serve as experimental subjects, and experimental research on the KPEDefense method is conducted using five large language models. Experimental results show that KPEDefense is more effective than existing rule-based risk scanning methods, and that each component of the method positively contributes to strategy effectiveness. Furthermore, seven representative CVEs are selected for case analysis, further verifying the practical effectiveness of KPEDefense from three evaluation dimensions.

Key Words: Cloud-native security; Privilege escalation vulnerabilities; Strategy code generation; Patch-window mitigation; Multi-agent collaboration

目 录

摘要	I
ABSTRACT	III
符号清单与术语表	IX
1 引言	1
1.1 背景与意义	1
1.2 研究内容与成果	2
1.3 论文组织结构	3
2 背景介绍	5
2.1 相关概念与技术	5
2.1.1 容器	5
2.1.2 容器集群系统 Kubernetes	6
2.1.3 Kubernetes 权限模型	7
2.1.4 Kubernetes 权限提升漏洞	8
2.1.5 云原生安全技术	8
2.2 国内外研究现状	10
2.2.1 权限提升风险与防控	10
2.2.2 云原生应用权限安全	12
2.2.3 智能化漏洞防控相关研究	13
2.2.4 前期工作	15
2.3 小结	16
3 面向云原生权限提升漏洞的智能化防控技术	17
3.1 研究动机	17
3.2 方法框架	17
3.2.1 基本概念定义	18
3.2.2 安全知识库	19
3.2.3 基于多智能体的安全策略生成与优化	23
3.2.4 策略代码质量评估	26
3.3 方法示例	28
3.4 小结	30
4 面向云原生权限提升漏洞的智能化防控系统设计与实现	31
4.1 需求分析	31
4.2 总体设计	32

4.3 详细设计	33
4.3.1 实体类设计	33
4.3.2 详细流程设计	34
4.3.3 交互时序设计	34
4.4 系统展示与验证	35
4.4.1 知识库管理	36
4.4.2 智能体调度	37
4.4.3 质量评审	38
4.4.4 策略管理	39
4.5 小结	41
5 实验研究	43
5.1 研究问题	43
5.2 实验对象	43
5.3 实验设计	44
5.3.1 度量指标	44
5.3.2 基线方法	45
5.3.3 实验过程	45
5.4 结果分析与讨论	46
5.4.1 静态工具对比	46
5.4.2 方法部分消融实验贡献度	47
5.4.3 生成模型能力评估	48
5.4.4 评审方法可信度评估	50
5.4.5 资源消耗与效率评估	51
5.5 小结	52
6 案例分析	55
6.1 案例分析实验设计	55
6.1.1 CVE 案例选择	55
6.1.2 实验环境与复现流程	56
6.2 案例一：CVE-2022-24768 内部权限问题	57
6.2.1 漏洞复现	57
6.2.2 安全策略生成	58
6.2.3 策略验证	59
6.3 案例二：CVE-2025-2241 敏感信息泄露	59
6.3.1 漏洞复现	59
6.3.2 安全策略生成	61
6.3.3 策略验证	61

6.4 案例三：CVE-2023-30512 过度权限配置	61
6.4.1 漏洞复现	61
6.4.2 安全策略生成	63
6.4.3 策略验证	64
6.5 案例四：CVE-2020-4062 缺乏网络隔离策略	64
6.5.1 漏洞复现	64
6.5.2 安全策略生成	65
6.5.3 策略验证	66
6.6 案例五：CVE-2022-24877 代码注入（路径穿越）	66
6.6.1 漏洞复现	66
6.6.2 安全策略生成	67
6.6.3 策略验证	68
6.7 案例六：CVE-2022-29171 远程代码执行	68
6.7.1 漏洞复现	68
6.7.2 安全策略生成	69
6.7.3 策略验证	69
6.8 案例七：CVE-2023-22482 内部权限失控	70
6.8.1 漏洞复现	70
6.8.2 安全策略生成	71
6.8.3 策略验证	72
6.9 小结	72
7 结论与展望	75
7.1 总结	75
7.2 展望	76
参考文献	77
附录 A 云原生权限提升漏洞数据集统计	87
附录 B 云原生权限提升漏洞分类与 CWE 对应关系	89
附录 C CVE202224768 复现步骤与策略清单	91
C.1 漏洞复现详细步骤	91
C.2 策略代码清单	92
附录 D CVE-2025-2241 复现步骤与策略清单	97
D.1 漏洞复现详细步骤	97
D.2 策略代码清单	97
附录 E CVE-2023-30512 复现步骤与策略清单	99
E.1 漏洞复现详细步骤	99
E.2 策略代码清单	99

附录 F CVE-2020-4062 复现步骤与策略清单 101

 F.1 漏洞复现详细步骤 101

 F.2 策略代码清单 101

附录 G CVE-2022-24877 复现步骤与策略清单 103

 G.1 漏洞复现详细步骤 103

 G.2 策略清单 103

附录 H CVE-2022-29171 复现步骤与策略清单 107

 H.1 漏洞复现详细步骤 107

 H.2 策略清单 107

附录 I CVE-2023-22482 复现步骤与策略清单 109

 I.1 漏洞复现详细步骤 109

 I.2 策略清单 109

致谢 111

作者简历及在学研究成果 113

独创性说明 115

关于论文使用授权的说明 117

学位论文数据集 119

符号清单与术语表

本文中频繁使用的主要符号、核心术语与常用缩写汇总如下。为避免与正文重复，本页重点列出第三章方法定义与后续评价分析中反复出现的记号与表述。

符号清单

符号	含义说明
v	单个漏洞实例，三元组形式 $\langle id, d, m \rangle$ ，由编号、描述与元数据组成。
c	安全上下文，即带来源标识的有限证据集合。
t_i	第 i 条证据内容，如代码片段、公告文本或文档片段。
src_i	第 i 条证据来源，如文档路径、链接或代码位置。
n_c	安全上下文 c 中的证据条数，即 $ c $ 。
$\mathcal{K}_v$	针对漏洞 v 构建的安全知识库，存放多源证据、向量索引与元数据。
Π	候选策略全集，表示所有候选监控策略与防护策略的总集合。
π	单个安全策略，三元组形式 $\langle type, method, action \rangle$ ，由策略类型、控制机理与执行表达组成。
$type$	策略类型，取值为 $\{\text{mon}, \text{prot}\}$ ：mon 表示监控型（以观测告警为目标），prot 表示防护型（以阻断收敛为目标）。
$method$	控制机理序列，形如 $(m_1, \dots, m_k)$ ，每个 m_i 描述一项控制手段的作用路径与机理，如 Falco 规则告警、Kyverno 准入审计、节点权限收敛等。
$action$	执行表达序列，形如 $(a_1, \dots, a_k)$ ， a_i 是 m_i 对应的可部署表达，可为规则片段、配置声明或可执行命令等形式。
m_i	第 i 项控制机理，与 a_i 一一对应，共同构成一个防控措施单元。
a_i	第 i 项执行表达，与 m_i 一一对应，直接对应可部署的规则、配置或命令。
$\mathcal{A}$	生成方法配置集合，用于刻画提示词组织、目标侧重或 workflow 设定等差异。
$\mathcal{M}$	底层大语言模型集合，表示参与候选策略生成的模型类型。

(续表)

符号	含义说明
$\mathcal{G}_v$	漏洞 v 的生成配置集合, 满足 $\mathcal{G}_v \subseteq \mathcal{A} \times \mathcal{M}$ 。
$g = (a, h)$	生成配置, 表示方法配置 a 与底层大语言模型 h 的组合 (定义 5)。
$\Pi_v^{(g)}$	漏洞 v 在生成配置 g 下输出的候选安全策略子集。
Π_v	漏洞 v 的候选策略集合, 由不同生成配置输出的候选安全策略子集汇合而成。
τ	类型变量, 表示某一给定策略类型。
Π_v^τ	同类型候选子集, 表示漏洞 v 在类型 τ 下的候选安全策略集合。
$\Pi_v^{\text{mon}}, \Pi_v^{\text{prot}}$	漏洞 v 的监控型与防护型候选子集, 是 Π_v^τ 的两种具体情形。
Δ	评价维度集合, 取值为 $\{E, I, D\}$, 分别对应有效性、无干扰性与可部署性。
$q(\pi)$	三维质量向量, 表示策略 π 在三个维度上的质量。
$E(\pi)$	有效性, 用于衡量对主要攻击路径与关键风险面的覆盖程度。
$I(\pi)$	无干扰性, 衡量对正常业务的扰动程度及误报风险。
$D(\pi)$	可部署性, 衡量策略在现有环境中的落地难度。
$\mathcal{H}$	评价主体集合, 表示多个评审主体构成的集合。
w_h	评审主体 h 在聚合时的权重, 满足 $\sum_h w_h = 1$ 。
n_τ	同类型候选集合 Π_v^τ 的规模, 即 $ \Pi_v^\tau $ 。
$r_\delta^{(h)}$	秩函数, 表示评价主体 h 在维度 δ 上给出的排序位置。
$\hat{S}_\delta(\pi)$	共识得分, 表示多个评价主体在维度 δ 上对策略 π 的聚合质量分。
$\bar{R}_\delta$	维度 δ 上的共识排序, 按 $\hat{S}_\delta$ 由高到低排列。
$U_\delta(\pi)$	分歧度量, 用于刻画各评审主体归一化得分相对共识得分的离散程度。
$Q(\pi)$	总体质量函数, 表示综合三维共识得分后的最终质量。
$\alpha_E, \alpha_I, \alpha_D$	维度权重, 满足 $\alpha_E + \alpha_I + \alpha_D = 1$, 可按业务场景配置。
$\bar{R}^\tau$	类型 τ 下的最终总排名, 由 $Q(\pi)$ 降序排列得到。

术语表

为统一全文表述, 本文使用频率较高的核心术语与常用缩写说明如下。

术语	英文/缩写	说明
云原生	Cloud-native	本文研究对象所处的应用与基础设施环境。
Kubernetes	K8s	Google 开源容器编排平台
权限提升	Privilege Escalation	攻击者突破原有权限边界并获取更高权限。
补丁窗口期	Patch-gap window	漏洞披露至补丁可用或补丁完成部署的时间段。
临时防控	Interim mitigation	在补丁窗口期内用于降低风险的临时控制措施。
安全策略	Security strategy	监控策略与防护策略的上位概念。
监控策略	Monitoring strategy	面向异常行为感知、告警与取证策略。
防护策略	Protection strategy	面向攻击阻断、权限收敛与攻击面控制的策略。
安全上下文	Security context	与具体漏洞相关的证据集合及其来源标识。
安全知识库	Security knowledge base	存放多源证据、向量索引及元数据的知识支撑体系。
多智能体协同	Multi-agent collaboration	由多个功能角色协同完成分析、生成与评审的工作方式。
检索增强	RAG	Retrieval-Augmented Generation
思维链推理	CoT	Chain-of-Thought, 思维链。
反馈优化	Feedback	基于评审结果对候选策略进行优化。

常用缩写

缩写	英文全称	中文说明
CVE	Common Vulnerabilities and Exposures	通用漏洞与暴露。
CWE	Common Weakness Enumeration	通用弱点枚举。
CVSS	Common Vulnerability Scoring System	通用漏洞评分系统。
NVD	National Vulnerability Database	国家漏洞数据库。
CNCF	Cloud Native Computing Foundation	云原生计算基金会。
K8s	Kubernetes	Kubernetes 的常用缩写。
RBAC	Role-Based Access Control	基于角色的访问控制。
LLM	Large Language Model	大语言模型。
DSL	Domain Specific Language	领域专用语言

1 引言

本章介绍课题的研究背景与意义、研究内容与成果，以及论文的组织结构。

1.1 背景与意义

服务计算发展推动了服务软件大规模部署^[1]。服务计算场景下，软件体系结构经历单体应用向面向服务架构 (SOA) 的转变过程，并发展至微服务架构^[2]。微服务系统由多个可独立开发、部署和演化的微服务组成，可分布式部署到多个计算节点。云原生技术将微服务和云计算结合，实现微服务系统在大规模计算集群中的弹性扩缩、持续交付和高可用运行^[3,4]。

云原生技术离不开集群管理基础设施的发展。容器技术依托内核级隔离和镜像分发成为微服务高效封装与运行的单元^[5]，但不能满足多节点资源调度和自动编排等需求。谷歌公司开发了容器管理系统 Borg^[6]，并以此为参考在 2014 年开源容器编排平台 Kubernetes(K8S)，后捐赠给云原生基金会 CNCF 管理。Kubernetes 通过主从式集群架构与声明式 API 实现对容器资源与应用状态的分布式管理，成为重要的云原生基础设施^[7]。

云原生基础设施维护依赖众多开源云原生应用，但这些应用存在第三方安全风险。为实现日志采集、监控告警、身份认证等能力，系统管理方引入例如 Fluentd、Prometheus 和 Keycloak 等开源云原生应用。这些云原生应用能够增强平台能力并降低开发运维成本，但其组件深度参与集群运行与控制过程，给 Kubernetes 集群系统带来安全隐患。其中，**权限安全风险是容器集群基础设施的重要风险来源**。云原生应用通过 RBAC 机制^[8] 获取一定的集群访问权限。若相关组件存在过度授权^[9]、授权检查缺陷、敏感信息泄露^[10] 或网络隔离配置不当等缺陷，攻击者可借此实施权限提升攻击，严重时甚至可接管整个集群^[11]。相关漏洞可称为云原生应用的**权限提升漏洞**。

云原生应用的权限提升漏洞管理面临三方面现实挑战：（1）**补丁窗口期攻击面持续暴露**。不同开源社区安全响应能力参差不齐，漏洞从公开披露到补丁发布的周期因项目而异且难以预期。2020 至 2025 年间 55 个云原生权限提升漏洞的修复时间分布如图1-1 所示，其中 30.4% 的漏洞补丁修复存在滞后问题，攻击者可借助公开公告与修复提交发现漏洞利用

条件。此外，第三方组件升级受限于现有架构兼容性和业务连续性要求，导致补丁无法立即部署到生产环境。

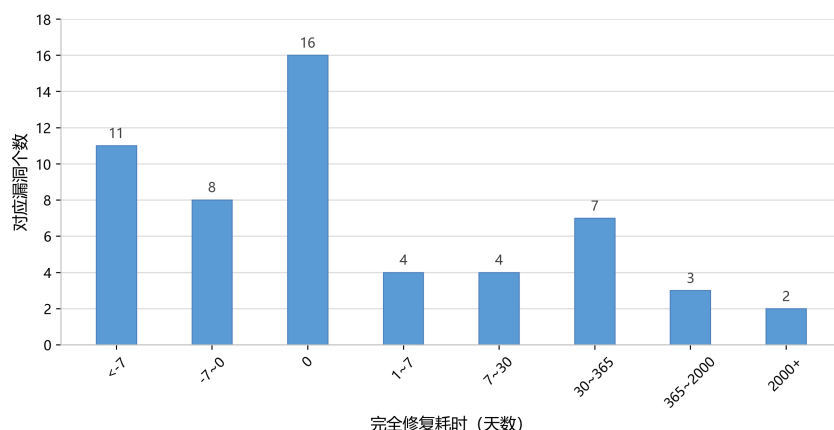


图 1-1 云原生环境下权限提升漏洞的修复时间分布

Fig. 1-1 Distribution of remediation time for privilege escalation vulnerabilities in cloud-native environments

- (2) **安全工具有效性不足且使用成本高。**现有技术或研究基于配置扫描^[12,13]、运行时检测^[14]、准入控制^[15,16]和网络隔离^[17,18]等方法应对安全问题，但上述工具的防控逻辑多为通用规则，并非针对具体漏洞定制，对特定高危 CVE 的实际拦截效果有限；部分工具还需通过 Rego、Falco 规则等领域特定语言手工编写防控规则，上手门槛较高。
- (3) **防控措施依赖专家知识。**云原生权限提升漏洞涉及 RBAC 权限配置^[8]、控制平面交互、应用特定行为等多个层面，涉及过度授权^[11]、风险组件劫持、敏感信息泄露^[10]等问题，实际处置需要熟悉 Kubernetes 的安全专家介入研判，难快速形成可落地的防控方案。

本文围绕上述三方面问题，研究基于大语言模型驱动的智能安全策略漏洞防控技术，并开发了支持工具。以收集到的云原生权限提升漏洞为实验对象，通过实验研究与案例分析验证了 KPEDefense 的实际防控效果。

1.2 研究内容与成果

围绕补丁窗口期内 Kubernetes 权限提升漏洞难以及时处置的问题，本文以面向具体漏洞生成可部署的临时防控策略为目标，研究基于多智能体安全策略管理的智能化漏洞防控方法，并完成支持工具实现以及实验验证。主要研究内容与成果如下。

- (1) **一种基于安全策略生成的智能化漏洞防控方法。**本文提出一种基于

安全策略生成的智能化漏洞防控方法 **KPEDefense**，该方法基于开源云原生应用信息构建安全知识库，通过多智能体协同 workflows 生成针对具体权限提升漏洞的防控策略代码，并构建基于多源模型的评估体系保障代码质量。

(2) 实现面向云原生权限提升漏洞的支持工具。本文设计并实现了 **KPEDefense** 支持工具，集成安全知识库管理、智能体调度、模型对接与安全策略部署验证等功能，实现方法落地。

(3) 实验验证与案例分析。本文通过样本实验、对比实验与典型案例验证，对 **KPEDefense** 在策略有效性、无干扰性和可部署性进行了系统评估。结果表明，**KPEDefense** 支持面向具体漏洞的临时防控策略自动化生成，并在实际场景中体现出较好的针对性与可行性。

1.3 论文组织结构

- 第一章， 引言，交代课题的研究背景与意义、研究内容与成果。
- 第二章， 背景介绍，梳理相关概念及技术、云原生安全技术与国内外研究现状。
- 第三章， 面向云原生权限提升漏洞的智能化防控技术，包括总体方法概述与基本概念定义、安全知识库、基于多智能体协同的策略生成与优化以及策略评估与质量控制。
- 第四章， 智能化防控系统的设计与实现，包括需求分析、总体设计、详细设计与实现。
- 第五章， 实验研究，围绕研究问题、实验对象与设计以及结果分析展开。
- 第六章， 案例分析，选取典型 CVE 案例进行漏洞复现与策略部署验证。
- 第七章， 研究工作的总结以及未来的展望。

2 背景介绍

本章介绍相关概念与技术，并梳理 Kubernetes 权限提升漏洞相关的国内外研究现状。

2.1 相关概念与技术

2.1.1 容器

计算服务的部署形态经历了从物理机、虚拟机到容器化的持续演进过程（见图 2-1）。虚拟机部署依赖 KVM 等虚拟化技术，能够提供完整操作系统级别的隔离，但也带来了较高的虚拟化开销、较长的启动时间和较大的镜像体积，从而影响应用交付效率与资源利用率。相比之下，容器通过操作系统内核级的资源隔离共享宿主机内核，在资源利用率和启动速度上更具优势。容器以**镜像**为交付单元，将应用及其运行依赖整体打包。其实现主要依赖以下三类内核机制：

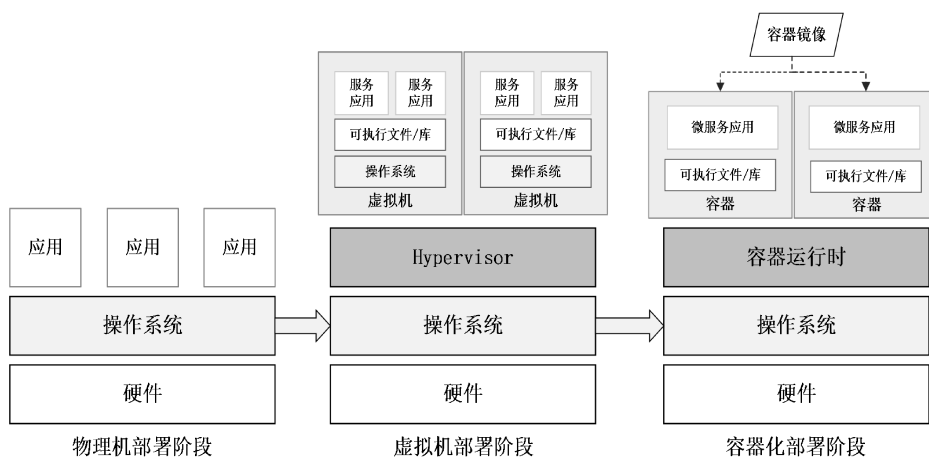


图 2-1 服务部署形式演进

Fig. 2-1 Evolution of service deployment paradigms

- **命名空间（Namespace）隔离**：通过隔离进程、网络、挂载点等，使容器内运行的程序有独立系统抽象资源；
- **控制组（Cgroup）限制**：负责对容器施加 CPU、内存等硬件资源使用的强制规划，提供物理资源隔离；
- **只读分层与隔离挂载**：利用 OverlayFS 等联合文件系统^[19]，通过复用底层只读镜像并叠加独立的顶层读写区域，实现容器文件系统隔离。

2.1.2 容器集群系统 Kubernetes

Kubernetes 是面向容器集群负载的分布式编排系统，简称 K8S。它源自谷歌内部的集群编排系统 Borg^[6]，并于 2014 年正式开源。其核心思想是以**声明式 API** 描述系统期望状态，再由控制平面通过循环控制持续驱动对象状态转变为期望状态^[20]。图2-2展示了 Kubernetes 的基本体系结构。一个 Kubernetes 集群由**控制平面**和多个**工作节点**构成。控制平面负责集群控制、调度与状态协调，其中：**API Server** 是资源访问与状态变更的统一入口，并结合键值数据库 ETCD 完成认证、授权、准入和资源对象持久化；**调度器**根据资源请求、亲和性与反亲和性等约束为工作负载选择节点；**控制器**负责持续调节各类资源状态。工作节点侧，**kubelet** 负责与 API Server 通信并维护本节点工作负载运行，**容器运行时**负责镜像拉取、容器创建和生命周期管理，**kube-proxy** 负责服务转发与网络规则实现。

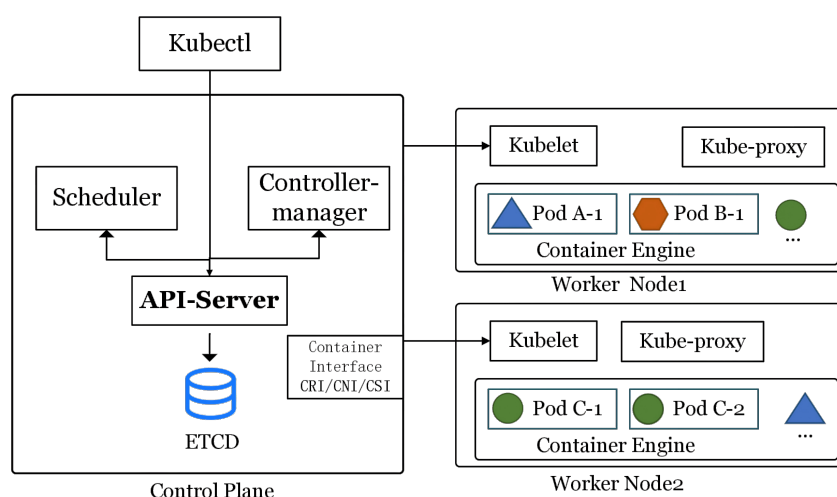


图 2-2 Kubernetes 体系结构示意图

Fig. 2-2 Overview of Kubernetes architecture

Kubernetes 通过一系列持久化**资源对象**描述集群中不同资源的编排与管理。这类对象通过结构化数据存储在 etcd 数据库中。例如，**Pod** 是最小调度单元，包含一个或多个共享网络与存储的容器；**Deployment**、**StatefulSet** 与 **DaemonSet** 分别面向无状态、有状态和节点守护型工作负载提供副本控制与更新策略；**Service** 与 **Endpoint/EndpointSlice** 提供稳定的服务发现与负载均衡抽象；**Ingress** 面向南北向流量提供入口控制；**ConfigMap** 与 **Secret** 用于配置和敏感信息的声明式注入；**Volume**、**PV** 与 **PVC** 则提供存储生命周期与绑定关系的抽象。以一个典型的 **Deployment** 对象为例，图 2-3 展示了其 YAML 声明

格式及各字段含义。其中 `apiVersion` 与 `kind` 共同确定资源所属的 API 组、版本及对象类型；`metadata` 包含对象的基本标识，其中 `name` 与 `namespace` 唯一确定对象在集群中的位置，`labels` 为对象附加键值对标签以供选择器关联，`annotations` 则用于存放不影响调度逻辑的补充说明信息；`spec` 字段承载对象的期望状态声明，其具体字段因 `kind` 不同而差异显著，控制器将持续对比此声明与集群实际状态并自动进行调节。

```
apiVersion: apps/v1      # API 组与版本（如 v1、apps/v1）
kind: Deployment         # 资源类型（如 Pod、Service、ConfigMap 等）
metadata:
  name: web-app           # 对象名称，在同命名空间内唯一
  namespace: default      # 所属命名空间
  labels:                 # 键值对标签，供选择器关联匹配
    app: web-app
  annotations:            # 补充说明信息，不影响调度逻辑
    owner: team-a
spec:                    # 期望状态声明（字段因 kind 而异）
  ...
```

图 2-3 Kubernetes Deployment 资源对象 YAML 配置示例

Fig. 2-3 Example YAML configuration of a Kubernetes Deployment resource object

2.1.3 Kubernetes 权限模型

基于角色的访问控制（RBAC）是 Kubernetes 授权机制的核心^[20]。图 2-4 左侧展示了权限元模型的形式化表达。该模型可以概括为主语、动作与对象之间的授权关系，即界定何种实体被允许对何种资源执行何种操作，其中：

- **服务账户（ServiceAccount）** 对应模型中的主语节点，表示发起请求的身份载体。在 Kubernetes 中，工作负载通常通过 ServiceAccount 与控制平面交互。
- **权限角色（Role/ClusterRole）** 对应模型中的动作与对象约束，用于声明在特定作用域内针对各类资源（resources）允许执行的操作动词（verbs）集合。
- **角色绑定（RoleBinding/ClusterRoleBinding）** 负责建立服务账户与权限角色之间的关联关系，从而完成授权。

图 2-4 右侧给出了一个典型的 RBAC 授权流程示例：首先创建名为 `kada` 的 ServiceAccount；随后定义名为 `basic-operation` 的 Role，并在其 `rules` 字段中授予对 `pods` 与 `secrets` 的 `get`、`list`、`create` 权限；最后通过 RoleBinding 将该 Role 绑定至 `kada` 账户。这类配置在接入第三方应用时较为常见，但若授予的权限范围超出实际需要，就可能引发权限提升风险。

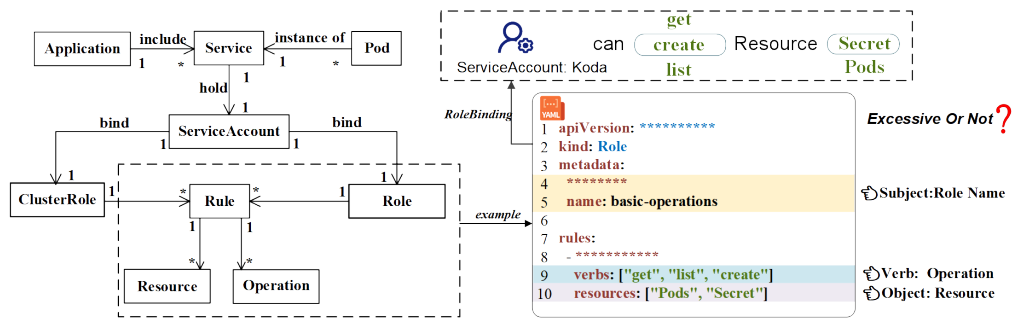


图 2-4 Kubernetes RBAC 权限配置 YAML 文件示意^[10]

Fig. 2-4 Example of Kubernetes RBAC permission configuration in YAML^[10]

2.1.4 Kubernetes 权限提升漏洞

权限提升是指攻击者在仅具备受限权限或初始执行能力的情况下，借助漏洞或错误配置将自身权限提升到更高等级的过程^[21]。

在 Kubernetes 场景下，权限提升表现为授权范围扩大或横向获取，或节点侧系统权限提升，从而影响更大范围的资源与工作负载。一方面，单个节点往往承载多个 Pod，一旦节点侧权限提升成功，攻击者便可能对同节点上的多个工作负载实施敏感信息窃取、运行环境篡改或恶意组件注入。另一方面，Kubernetes 的资源对象与运维动作高度集中且自动化，控制平面授权不当也会显著放大攻击后果。结合图 2-4 中的示例，若主体被授予针对 Secret 的 get/list 权限，攻击者即可读取服务账户令牌、数据库口令或 API 密钥等敏感信息；若被授予针对 Pod 的 create 权限，则可能进一步创建携带恶意镜像、特权配置或宿主机挂载的工作负载，作为横向移动、凭据窃取乃至后续节点侧利用的跳板。

图2-5 以 CVE-2024-33522 为例展示了典型的节点侧权限提升漏洞。该漏洞源于 Calico CNI 安装程序的 SUID 位配置不当。若攻击者借助弱口令或服务暴露等前置手段获取节点本地访问权限，便可注入恶意二进制文件，并诱导安装程序以高权限执行。越权成功后，攻击者可以直接接管容器运行时数据、窃取高权限凭据并干扰网络组件，其影响将突破单一工作负载的隔离边界，迅速蔓延至整个集群。

2.1.5 云原生安全技术

云原生安全技术围绕镜像构建、资源准入、网络通信和运行时行为等容器应用的生命周期环节，常见工具大多以规则化策略或声明式配置为载体，用于在不同阶段识别或约束风险。从作用方式看，这类技术大致可归纳为静

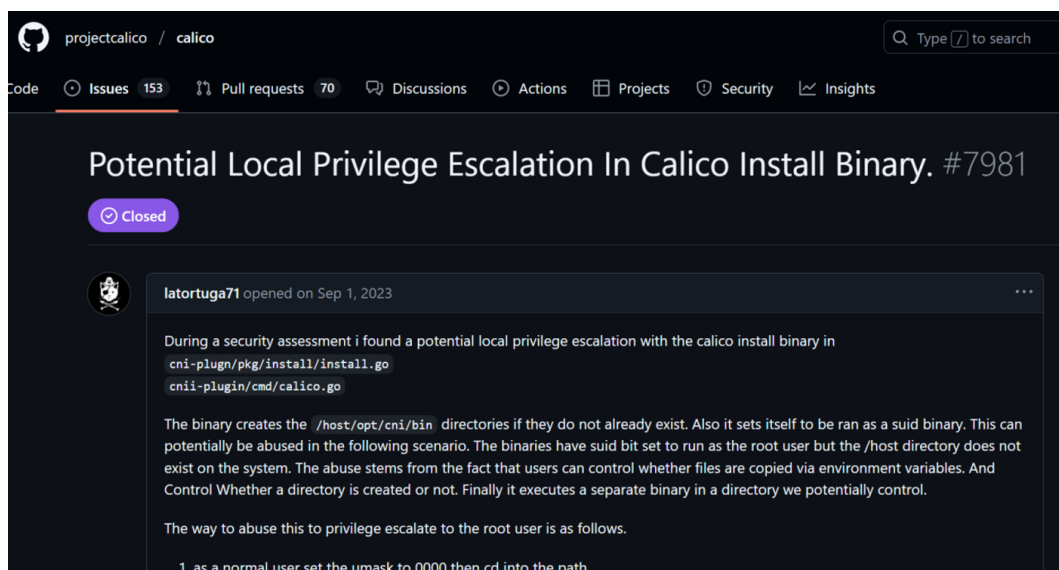


图 2-5 CVE-2024-33522 (Calico) 节点侧权限提升漏洞 Issue 报告

Fig. 2-5 Issue report of node-side privilege escalation vulnerability CVE-2024-33522 in Calico

态与动态两方面，它们共同构成 Kubernetes 集群全生命周期安全治理的基础能力。

静态方面，静态扫描与配置审计主要用于在镜像构建与部署前发现已知漏洞、弱配置和权限异常。代表性工具包括 Trivy^[22]、Grype/Anchore^[23] 等镜像与依赖漏洞扫描工具，Syft^[24] 等 SBOM 生成工具，以及 Kubescape^[25]、kubelinter^[26]、kube-score^[27]、KubeSec^[28]、Polaris^[29]、Checkov^[30]、KubiScan^[31]、Krane^[32] 和 RBAC Police^[33] 等面向清单、IaC 与 RBAC 的规则化审计工具。这类方法便于集成到 CI/CD 流程中，能够提前暴露高风险对象，但其判断依据主要来自既有漏洞库、配置基线和审计规则，对特定漏洞的真实利用条件仍缺乏上下文感知能力。

动态方面，准入控制、网络隔离与运行时监控更侧重集群运行阶段的风险约束。OPA/Gatekeeper^[15] 与 Kyverno^[16] 在 API Server 持久化资源前执行策略校验，可用于拦截特权容器、危险挂载和敏感权限变更；NetworkPolicy 及其在 Calico、Cilium 和 Weave Net 等插件中的实现，用于收敛服务间连通范围并降低横向移动风险；Falco^[14] 则依托 eBPF 技术^[34] 采集系统调用和 Kubernetes 事件，并通过规则引擎对异常行为进行实时检测与告警。这些技术分别从事前阻断、横向隔离和事中观测三个层面增强了集群安全性，但整体上仍以通用规则为主，难以直接围绕某一具体漏洞在补丁窗口期快速生成兼顾针对性与可部署性的临时防控策略。

图 2-6 给出了 Falco 与 OPA/Gatekeeper 的典型规则示例，分别代表运行

时监控与准入控制两类防护手段的声明方式。Falco 规则通过 `condition` 字段声明检测条件（如特权容器启动事件），并在 `output` 字段中定义告警内容格式；OPA/Gatekeeper 策略则以 Rego 语言编写 `deny` 规则，在 API Server 持久化资源前对不合规请求直接拒绝。两者均以声明式配置表达安全意图，前者侧重事中观测与告警，后者侧重事前拦截与阻断。

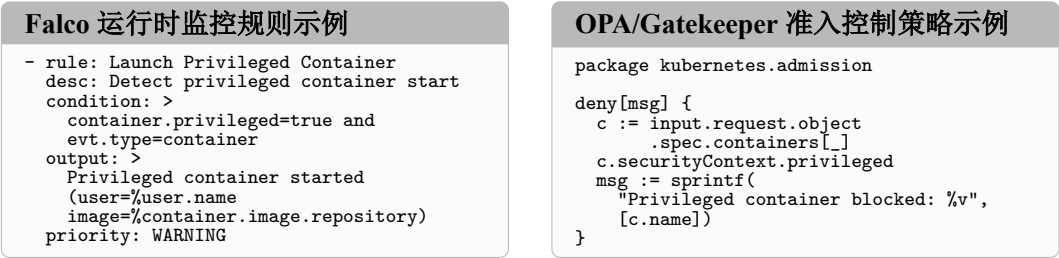


图 2-6 Falco 运行时监控规则与 OPA/Gatekeeper 准入控制策略示例
Fig. 2-6 Examples of Falco runtime monitoring rule and OPA/Gatekeeper admission control policy

2.2 国内外研究现状

根据本研究主题，本节从权限提升风险与防控、云原生权限安全、大语言模型与智能化漏洞防控三个方向梳理国内外相关工作，归纳各方向的主要进展与局限。之后介绍本课题前期工作研究并说明本文的研究定位。

2.2.1 权限提升风险与防控

权限提升防控研究在操作系统、移动平台与企业系统领域已形成较为系统的积累。本节从权限管理形式化框架、权限提升检测与防控两个层面介绍相关工作。

(1) 权限管理形式化框架。通过权限范围形式化描述与管理可规模化控制最小化权限原则，以控制权限提升风险。最小化权限原则是权限提升风险防控的前提，Saltzer 与 Schroeder 将最小化权限原则描述为**每个程序与每个用户均应仅在完成任务所必需的最小权限范围内运行**^[35]。Ferraiolo 等^[8]提出基于角色的访问控制模型（RBAC），通过角色组织权限并建立主体与权限之间的间接映射关系以应对大规模系统中直接授权管理复杂性。Sandhu 等^[36]进一步提出 ARBAC97 模型，将角色管理与授权管理纳入统一框架。Ferraiolo 等^[37]给出了企业内网环境下 RBAC 的参考实现，说明相关研究已较早进入通用信息系统与组织级应用场景。除基于角色访问控制模型外，Hu 等^[38]系统总结了基于属性的访问控制（ABAC），强调依据主体、客体、环境等多维

属性动态决定访问授权，从而提升权限控制在复杂开放环境中的细粒度表达能力。总体而言，RBAC 与 ABAC 分别从角色抽象和属性约束两个角度，为最小权限原则在实际系统中的落地提供了基本的权限管理框架。

相关研究进一步从系统结构和权限收缩角度探索权限提升风险的抑制机制。Provos 等^[39] 提出特权分离方法，将操作系统服务划分为不同权限级别的组件并将高权限代码压缩到可信基范围内，从而为非特权应用最小化特权使用范围；Watson 等^[40] 则通过对指令集架构、编译器及操作系统进行扩展，以支持细粒度的、基于能力的内存保护机制。这表明，权限提升防控并非仅依赖访问控制规则本身，也可以通过功能分解与权限隔离减少高权限执行路径。与此同时，角色建模与结构隔离均不足以彻底消除权限提升风险，冗余权限与过度授权在实际系统中仍普遍存在^[41]。针对这一问题，后续研究依据日志等应用真实行为，结合规则和机器学习技术逆向推导应用所需的最小权限集^[42,43]。权限提升风险防控由权限分配问题扩展到授权管理、结构隔离和权限收缩。

(2) 权限提升检测与防控。围绕权限提升风险的检测与防控分为静态架构分析、动态行为监控和数据流追踪三类。静态分析主要关注权限配置与应用组件的最小权限集是否匹配，例如 PScout^[44] 通过源码分析构建代码调用到权限检查点之间的可达路径从而检测过度权限，DelDroid^[45] 通过静态分析权限范围并推导最小权限集，SecComp^[46] 则在组件调用层面对非授权操作施加约束，以防御组件劫持引发的权限提升。动态行为监控侧重于运行时提权行为的自动识别，Bugiel 等^[47] 指出，Android 多个应用进行协同共谋权限提升攻击；并提出一种面向系统的策略驱动运行时监控框架，从中间件与内核多个层面联合约束应用间通信与资源访问；Sharmeen 等^[48] 结合深度学习与半监督学习方法对权限提升行为进行检测，表明相关方法正由静态授权关系检查转向对真实运行过程的动态观测。数据流追踪通过污点分析技术关注敏感资源如何传播而导致的可能权限提升。Tripp 等^[49] 提出面向 Web 应用的混合污点分析框架 TAJ，TaintDroid^[50] 与 TaintMan^[51] 则分别在 Android 虚拟化环境和非 Root 真机环境中实现了对显式及部分隐式信息流的动态跟踪。总体而言，权限提升风险防控需考虑对静态代码、越权路径、运行行为和敏感数据传播。

围绕权限提升风险已有研究已形成从权限约束到行为监控、再到信息流追踪的较完整分析路径，指出权限提升漏洞具有复杂攻击模式。但相关工作

主要针对移动应用与 Web 系统，尚难直接回答云原生权限提升漏洞的识别与防控问题。

2.2.2 云原生应用权限安全

云原生权限安全研究覆盖配置合规扫描、供应链与模板安全、权限风险挖掘与攻击链分析、运行时防护四个方向。

(1) **配置合规与静态扫描。**Kubernetes 的配置文件中存在权限风险，相关研究基于实证研究进行归纳并总结出规则化静态检测方法。Islam Shamim 等^[52,53]系统总结了 Kubernetes 安全实践，并进一步指出部署清单偏离最佳实践可能直接演化为可利用的提权路径，Curtis 与 Eisty^[54]基于 Stack Overflow 数据的实证分析也表明配置安全始终是开发侧的核心关切。在此基础上，Rahman 等^[13]对开源清单中的典型安全误配置进行了大规模归纳与实证研究。不过，静态扫描虽然能够发现显式违规项，却难以判断这些配置在真实场景中是否能够被有效利用。围绕这一问题，Shamim 等^[12]评估了 Internet 安全中心组织提供的 Kubernetes 安全配置标准¹的合规现状，指出从业人员对安全基线的理解存在明显差异，现有静态应用程序安全测试工具的检测覆盖也存在不足；其后续工作^[55]进一步论证了将动态分析和安全测试纳入配置核验流程的必要性。

Helm Chart 是用于管理 Kubernetes 应用的部署模板配置文件，其中也存在安全风险。Zerouali 等^[56]和 Blaise 与 Rebecchi^[57]分别从 Helm Charts 的生态规模和依赖关系的角度，分析了其带来的供应链风险，指出安全问题并非仅由单一的配置错误引发，更多是通过模板依赖和复用机制在整个系统中传播。Zerouali 等发现，大多数 Helm Charts 存在过时镜像和安全漏洞，而 Blaise 与 Rebecchi 则通过图生成和攻击重建方法，揭示了 Helm Charts 中复杂攻击路径的潜在威胁。Haque 等^[58]提出的 KGSecConfig 框架，通过统一建模跨平台的安全配置概念，有效解决了多平台安全语义碎片化的问题。这些研究表明，云原生安全分析已逐步从单一配置项的检查，扩展到模板、依赖链和安全语义的系统化建模。

(2) **权限风险挖掘与攻击链分析。**权限风险挖掘相关研究关注权限风险如何沿系统结构演化为完整攻击链。Dell’Immagine 等^[59]面向微服务部署定义了一组安全反模式并实现自动检测工具 KubeHound，表明权限风险可能嵌

¹<https://www.cisecurity.org/benchmark/kubernetes>

入服务交互结构之中。Pecka 等^[60]指出一种 Kubernetes 持续集成场景下的权限提升攻击场景，并通过工具合规应保证系统安全。Yang 等^[11]系统分析了 Kubernetes 生态中第三方插件的权限配置，设计了多种攻击路径利用策略，说明节点逃逸、控制面访问与第三方组件权限可以共同演化为权限提升攻击，甚至控制整个 K8s 集群。Gu 等^[9]提出一种过度权限分析工具 EPScan，通过面向容器的程序静态分析自动比对实际资源访问行为与声明权限，识别可被利用的过度特权。与前述配置合规研究相比，这类工作更强调风险如何沿授权链路形成并被放大。不过，其输出大多仍停留在风险发现与报告层面，尚未进一步扩展到围绕具体漏洞生成临时防控策略的研究。

(3) 运行时防护。安全性防护可通过运行时监控可以检测并防护安全攻击。Minna 等^[17]指出，容器编排引入的网络抽象与传统安全模型存在显著差异，并指出传统安全防御框架在 Kubernetes 系统中存在可迁移性问题。Budigiri 等^[18]基于 eBPF 评估了网络安全策略的性能代价，Kim 等^[61]对比了多种容器网络接口插件的安全能力，二者共同揭示了网络策略在过滤深度与转发性能之间的权衡。在更具体的缓解机制方面，Zhu 与 Gehrmann^[62]以及 Le 等^[63]则分别从 AppArmor 配置自动生成和系统调用感知调度角度强化节点侧防护。这类研究提高了云原生权限安全能力，但不能围绕具体漏洞构造针对性处置措施。

综上，云原生权限安全研究主要关注配置风险的静态扫描和权限风险挖掘，缺乏对漏洞防控的关注；而运行时防护相关研究缺乏其依赖的检测规则研究，缺乏对特定漏洞的监控或防护能力。

2.2.3 智能化漏洞防控相关研究

本节将介绍智能化漏洞防控相关研究在大语言模型安全能力评测、漏洞检测与修复、多智能体安全任务等方面的研究。

(1) 大语言模型安全能力评测。大语言模型安全任务评测研究探索了大模型针对软件安全保障的能力边界。Chang 等^[64]系统综述了 LLM 通用评测方法与基准设计原则，为安全领域的专项评测提供了方法参照。在此基础上，安全领域的评测工作逐渐从通用能力测试转向面向真实安全任务的专门基准。Yin 等^[65]围绕漏洞检测、定位与修复等多个软件漏洞子任务对主流 LLM 进行测评，结果表明当前模型在漏洞定位与跨语言理解方面仍存在明显短板。Fu 等^[66]将评测对象进一步扩展到 LLM 智能体，提出面向真实安全运维环境

的评测框架，以考察智能体在信息不完整与噪声干扰条件下的稳定性与鲁棒性。Zhang 等^[67] 则构建了覆盖告警研判、威胁检测与安全响应等多类防御任务的语言智能体评测基准 **DefenderBench**，为不同模型和智能体框架之间的横向比较提供了统一平台。上述工作表明，若缺乏面向真实安全任务的系统化评测，模型能力便难以被可靠比较，更难支撑其在漏洞防控场景中的实际应用。

(2) 智能化漏洞检测与修复。软件缺陷管理可借助大语言模型智能。Ghosh 等^[68] 提出以安全领域本体辅助 LLM 完成 CVE 漏洞评估，通过引入漏洞类型、攻击向量和影响范围等结构化知识约束模型推理，提升了漏洞语义理解与风险评估的准确性。在云原生误配置修复方向，Malul 等^[69] 提出 **GenKubeSec**，将 LLM 引入 Kubernetes 误配置的检测、定位、推理与修复建议生成，构建了面向云原生配置安全的全流程自动化处理框架；Minna 等^[70] 进一步评估了 LLM 修复 Helm Chart 配置异味的效果，指出仅以局部 YAML 片段为输入往往难以生成完整修复方案，且模型存在误删无关配置的风险，说明上下文完整性与输出可靠性是当前方法的关键约束。除修复建议之外，LLM 也开始被用于检测规则生成。Konstantaras 等^[71] 探索了由模型从威胁描述中自动归纳生成静态检测规则的方法，为将自然语言形式的攻击知识转化为可部署规则提供了初步路径。总体而言，这类研究展示了从漏洞描述到防控措施生成的自动化潜力，但已有工作多聚焦于通用误配置或孤立漏洞场景，输出也多停留在修复建议或单条规则层面，尚未形成面向云原生权限提升漏洞的多策略协同生成与部署验证闭环。

(3) 多智能体驱动的安全任务自动化。当安全任务具有多阶段依赖、高上下文需求和反馈迭代特征时，单一模型往往难以稳定完成整条任务链条，因此多智能体驱动的安全任务自动化成为新的研究方向。Alsabbagh 等^[72] 提出 **AutoSecAgent**，通过智能体编排、递归记忆与检索增强生成，使系统能够在多步骤安全任务中跨轮次积累上下文并稳定执行，从而缓解单模型在长链路任务中的信息遗忘问题。Yang 等^[73] 则针对云原生生态中已知第三方组件漏洞的利用验证问题，设计了多智能体协同与迭代反馈机制，通过多轮自主探索实现从漏洞信息到可验证利用策略的端到端自动化。

综上，多智能体框架已在渗透测试和漏洞利用生成等攻击侧任务中显示出较强潜力，可以探索大预言模型自动生成安全防护工具特定代码的安全能力边界，以解决复杂权限提升漏洞的防控问题。

2.2.4 前期工作

本课题组围绕 Kubernetes 权限风险检测已形成较为系统的前期积累，其中代表性成果为系统性权限风险检测框架 KubeGuard^[10]。该框架面向过度权限、权限提升和敏感信息泄露三类风险，分别设计了最小权限集构建、规则化审计以及敏感词监测机制。如图2-7所示,KubeGuard 的核心贡献在于验证了从权限视角开展风险建模与检测的可行性。一方面，该工作利用 NLP 技术识别应用所需的最小权限集，以判断权限配置是否存在过度授权；另一方面，通过动态规则审计识别多类权限提升风险。具体而言，该框架将权限提升场景归纳为凭证窃取、账户冒用、RBAC 操纵和间接执行四类，并分别提出相应的检测思路。此外，系统还通过监测 Pod 间消息传输并结合关键词匹配，对涉及敏感资源的内容进行告警，以覆盖敏感信息泄露风险。在系统实现与效果验证方面，课题组进一步实现了支持工具 KubeGuarder，并在开源 Kubernetes 应用与模拟场景中开展实验评估。实验结果表明，该系统在过度权限检测方面较静态基线方法具有更高的有效性与准确性，同时能够识别多类型权限提升与敏感信息泄露风险。

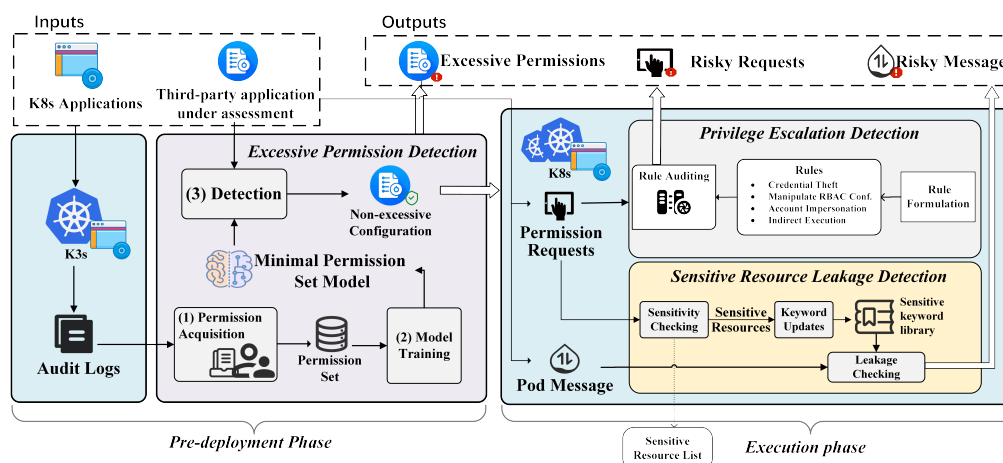


图 2-7 KubeGuard 方法框架图^[10]

Fig. 2-7 Framework of the KubeGuard method^[10]

针对权限提升问题，现有工作在真实生产环境中存在三方面局限：1）规则化。基于规则化的权限提升动态检测方法难以覆盖新型复杂的提权漏洞；2）误报率。针对本身具有高权限的敏感应用易产生误报信息；3）动态性。无法动态监控工作节点内部的运行时特征，只能在攻击后检测，无法在事前或事中进行阻断防护。

2.3 小结

本章围绕本文所关注的云原生权限提升漏洞智能化防控问题，介绍了容器、Kubernetes 及其权限模型等基础概念与云原生安全技术，并从权限提升风险与防控、云原生权限安全以及智能化漏洞防控三个方向梳理了国内外研究现状，此外、还详细介绍了前期研究成果并阐述本文的不同之处。

3 面向云原生权限提升漏洞的智能化防控技术

本章阐述面向云原生权限提升漏洞的智能化防控方法。首先介绍研究动机与核心问题，再依次展开基本概念定义、安全知识库构建、多智能体策略生成与优化、以及基于排序的质量评价四项核心技术。

3.1 研究动机

云原生权限提升漏洞防控面临补丁窗口期暴露、专家依赖和工具有效性不足三方面挑战。大语言模型由于生成速度和成本问题不能直接替代安全准入控制器或运行时系统,但能通过漏洞语义理解和复杂推理生成 **Kuberneetes 安全监控与防护工具的领域特定语言**,从而实现对漏洞的临时性防控。该思路落地须要进一步回答以下三个问题:

- (1) **如何构建面向具体漏洞的安全上下文**。权限提升漏洞相关的漏洞成因与影响边界等关键信息分散在代码仓库、问题讨论区与修复提交等多类来源中,需建立可检索、可追溯的领域安全知识库,将分散证据精炼为可供大模型使用的安全上下文。
- (2) **如何基于漏洞语义生成安全策略代码**。大语言模型的单步问答不能实现从漏洞语义理解到防控策略的有效生成。漏洞成因分析、攻击路径分析、安全工具选择以及策略代码生成等子问题均能影响安全策略生成质量,应设计完整 workflow 分析上述问题。
- (3) **如何评估安全策略中代码质量**。由于大语言模型幻觉问题,生成的安全工具 DSL 代码可能存在防控覆盖能力不足或干扰正常业务等问题,需对安全策略进行质量评价。然而策略内代码由于异构性和人工评估成本而存在测试预言问题^[74],需保证评估效率的同时提升评估结果的可靠性与一致性。

本文提出 **KPEDefense**,一种面向云原生权限提升漏洞的智能化防控方法,以安全知识库解决安全上下文构建问题,以多智能体协同 workflow 组织策略代码生成 workflow,并构建基于排序的多源模型质量量化评估方法。

3.2 方法框架

本文云原生权限提升漏洞防控问题建模为面向具体漏洞临时防控策略生成问题,以权限提升类 CVE 为输入,形成从漏洞理解到策略生成与质量评估

的整体方法。本方法三个部分构成如图 3-1 所示：云原生安全知识库部分从第三方开源应用的开源代码仓库中的多源信息中采集上下文特征，形成包含 CVE/CWE 统计、权限提升风险分类框架与向量化领域知识的安全知识库，为后续分析提供可检索的证据支撑；基于多智能体的策略生成与优化部分以权限提升 CVE 为输入，通过安全知识检索上下文构建、思维链推到、反馈优化等多智能体协同工作流生成安全策略候选集合；策略代码质量评估部分采用大模型排名式多维量化指标进行静态评估。此外动态评估通过案例分析对安全策略的有效性进行实证验证（见章节 6）。下文从基本概念定义（3.2.1 节）、安全知识库构建（3.2.2 节）、多智能体策略生成与优化（3.2.3 节）以及策略代码质量评估（3.2.4 节）四个方面展开。

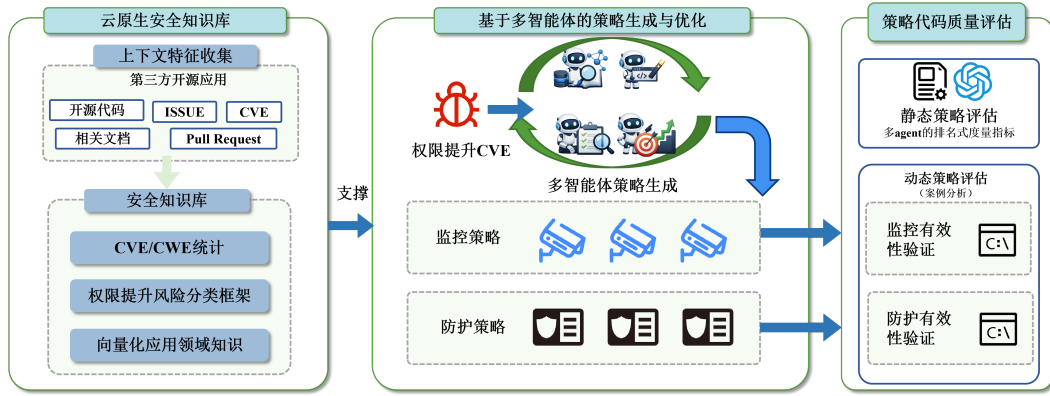


图 3-1 基于多智能体的策略化漏洞防控研究总体框架

Fig. 3-1 Overall framework of multi-agent strategy-based vulnerability prevention and control

3.2.1 基本概念定义

本文将核心对象形式化为漏洞实例、安全上下文与安全策略三类实体；主要记号与术语见第 IX 页。

定义 1（漏洞实例） 设 $\mathcal{V}$ 为漏洞实例空间，任一漏洞实例表示为

$$v = \langle id, d, m \rangle \in \mathcal{V} \quad (3-1)$$

其中 id 为漏洞 CVE 编号， d 为漏洞描述文本， m 为元数据，记录严重性、受影响组件及版本范围等属性。

定义 2（安全上下文） 给定漏洞实例 v ，其安全上下文 c 为带来源标识的有限证据集合：

$$c = \{ \langle t_i, src_i \rangle \mid i \in [n_c] \} \quad (3-2)$$

其中 t_i 为证据内容, src_i 为来源标识 (如文档路径、公告链接或代码位置), 用于支撑策略生成中的证据追溯与复核, $[n_c]$ 为检索到的证据数量。

定义 3 (防控条目) 安全策略的基本组成单元为三元组

$$e = \langle \tau, m, \mathbf{a} \rangle \quad (3-3)$$

其中 $\tau \in \{\text{mon}, \text{prot}\}$ 为控制类型: **mon** (监控型) 持续观测运行时状态并触发异常告警; **prot** (防护型) 主动阻断权限滥用并收缩攻击面。 m 为控制机理, 描述该措施的作用路径, 如 Falco 规则告警、Kyverno 准入审计或 RBAC 权限收缩。 $\mathbf{a} = (a_1, \dots, a_k)$ 为 m 对应的执行表达序列 (规则片段、配置声明或可执行命令), 每个 a_i 对应 m 的一个可部署表达。

定义 4 (安全策略) 针对漏洞实例 v 的安全策略为防控条目的有限集合

$$\pi = \{e_1, e_2, \dots, e_n\} \quad (3-4)$$

其中每个 $e_i = \langle \tau_i, m_i, \mathbf{a}_i \rangle$ 为定义 3 所述防控条目。多条目策略将防护型与监控型措施协同组织, 弥补单一类型在阻断能力或持续观测方面的局限。

定义 5 (生成配置) 设 $\mathcal{A}$ 为方法配置集合, $\mathcal{M}$ 为大语言模型集合, $g = (a, h) \in \mathcal{A} \times \mathcal{M}$ 为方法配置 a 与底层模型 h 的组合。

定义 6 (候选策略集合) 围绕漏洞实例 v 的候选策略集合记为

$$\Pi_v = \bigcup_{g \in \mathcal{G}_v} \Pi_v^{(g)} \quad (3-5)$$

其中 $\mathcal{G}_v \subseteq \mathcal{A} \times \mathcal{M}$ 为 v 采用的生成配置集合 (定义 5), $\Pi_v^{(g)} \subseteq \Pi$ 为配置 g 输出的候选子集, $\Pi = 2^{\mathcal{E}}$ 为安全策略全集。 $\mathcal{A}$ 与 $\mathcal{M}$ 的具体取值在第五章实验部分给出。

3.2.2 安全知识库

如图 3-2 所示, 本节围绕漏洞实例 v 构建可追溯云原生权限提升安全知识库 $\mathcal{K}_v$, 并通过检索形成安全上下文 c , 为后续策略生成提供带来源标识的证据支撑。

1) 数据采集

本文以 Kubernetes 及其生态开源应用为研究对象, 首先从 CNCF Landscape 获取云原生应用清单, 将清单条目名称映射到其官方代码仓库, 形成

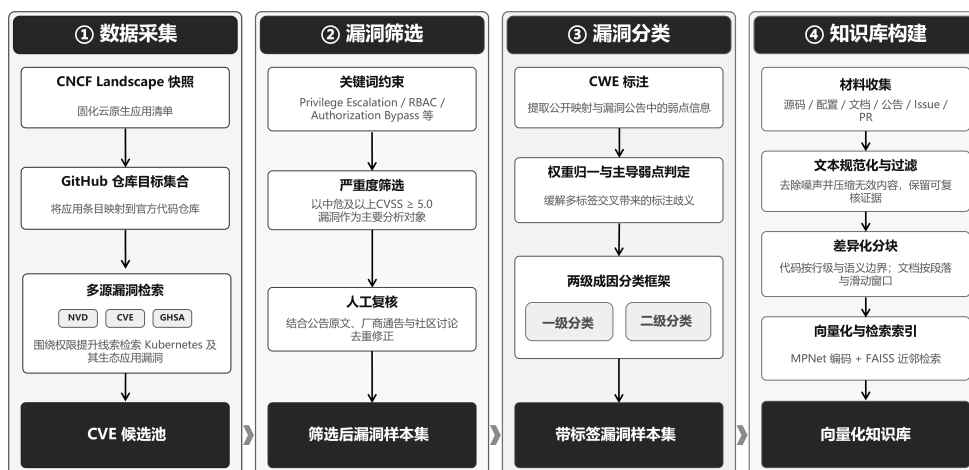


图 3-2 云原生权限提升漏洞知识库构建流程

Fig. 3-2 Construction of the cloud-native privilege escalation vulnerability knowledge base

后续漏洞检索与证据收集的目标集合。使用 NIST NVD¹、MITRE CVE 官方库²与安全公告平台 GitHub Security Advisory³三个来源，围绕权限提升相关关键词对相关开源项目开展多源漏洞检索，获得初步 CVE 候选池。

2) 漏洞筛选

为从 CVE 候选池中筛选出高质量样本，本文采用关键词约束、严重性阈值与人工复核相结合的重重过滤策略：

- **关键词约束**：在 CVE 描述与参考链接中匹配 Privilege Escalation、Authorization Bypass、Improper Access Control、RBAC、ServiceAccount、ClusterRole、Container Escape/Sandbox Escape/Breakout 等关键词；
- **严重性阈值**：采用 CVSS v3.x 评分，并以 ≥ 5.0 的中危及以上漏洞为主要对象；
- **人工复核**：对候选条目结合公告原文、厂商通告与社区讨论进行复核，剔除重复或与权限提升无关的记录，并补全攻击前提、影响面与可用缓解信息。

3) 漏洞分类

漏洞样本的分类方式影响后续策略生成方向。为此，本文构建了面向策略生成场景的两级成因分类框架，旨在为模型提供稳定且可解释的风险语义

¹NIST NVD (National Vulnerability Database): <https://nvd.nist.gov/>

²MITRE CVE: <https://cve.mitre.org/>

³GitHub Security Advisory: <https://github.com/advisories>

线索，同时可以提示安全工具类型的使用。

为建立该分类框架，首先参考现有权限提升漏洞 CVE 采用的弱点分类体系 CWE（Common Weakness Enumeration）。CWE 是 MITRE 维护的软件弱点枚举体系，可用于将具体 CVE 归并到更抽象的弱点类型。CWE 体系并不是严格互斥的分类框架。同一漏洞往往会同时对应多个弱点条目，不同条目在抽象层次上也可能彼此交叉。在本文收集的 55 个权限提升漏洞中，有 20 个样本同时关联两个及以上 CWE 标签，占比 36%。为更客观地反映 CWE 分布，统计时令每个 CVE 的总权重为 1，多标签样本按标签数平均分配。结果表明，权限提升漏洞特征丰富，而 CWE 的类目划分并不能指出权限提升漏洞攻击的关键缺陷或攻击路径。

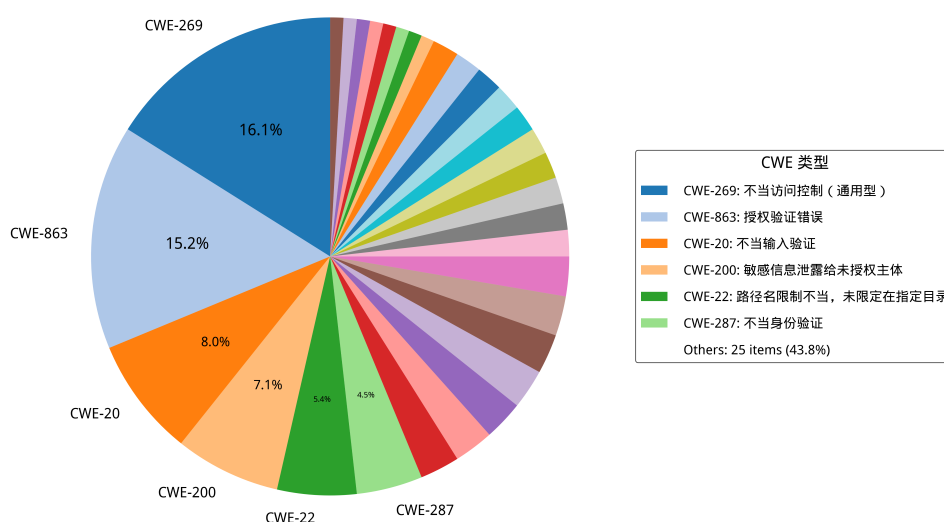


图 3-3 Kubernetes 权限提升 CVE 对应的 CWE 类型占比
Fig. 3-3 Distribution of CWE categories corresponding to Kubernetes privilege escalation CVEs

基于这一问题，本文进一步提出面向云原生权限提升漏洞的两级成因分类框架。该框架基于漏洞的关键成因，其中一级分类用于概括高层风险来源，如安全逻辑缺陷、配置缺陷、代码注入和敏感信息泄露；二级分类则进一步细化到更贴近防控动作选择的类型，如授权验证不完善、默认不安全配置等。在策略生成阶段，分类结果作为提示词中的结构化约束，引导模型先识别漏洞的主要成因，再定位关键控制点，最后组织可执行的防控动作，从而无关信息以提高策略的有效性。完整的分类见表 3-1，其与 CWE 的对应关系见附录表 B-1。

以下对各一级分类分别展开说明。

表 3-1 云原生权限提升漏洞分类框架（CWE 对应详见附录表 B-1）
Tab. 3-1 Classification framework for cloud-native privilege escalation vulnerabilities
(see Appendix Table B-1 for the CWE mapping)

一级分类	二级分类	示例 CVE
配置缺陷	冗余权限	CVE-2023-30512
	网络隔离不足	CVE-2020-4062
安全逻辑缺陷	访问控制缺陷	CVE-2022-24768
	身份验证绕过	CVE-2023-22482
	输入验证不足	CVE-2023-3955
	授权验证不完善	CVE-2022-21701
敏感信息泄露	API 端口暴露	CVE-2022-1902
	日志泄露	CVE-2019-10165
	配置文件泄露	CVE-2019-3869
代码注入	命令注入	CVE-2021-41254
	路径遍历	CVE-2022-24877

1) **配置缺陷**指应用或基础设施的配置不当导致的安全漏洞，主要包括两种情形：

- **过度权限配置**：Kubernetes RBAC 配置不当、Role/ClusterRole 规则范围过宽或绑定关系不合理等，使低权限主体获得超出其职责的资源访问与高危操作能力。该类问题在第三方应用接入场景中尤为常见，并已被证明可被用于接管集群关键资源^[11,41]。
- **网络隔离不足**：容器、Pod 或服务间缺乏网络隔离，导致攻击者可以横向移动获取其他权限。

2) **安全逻辑缺陷**指应用代码中的安全机制设计或实现不当，涵盖以下情形：

- **访问控制缺陷**：应用的权限检查不完全或存在逻辑漏洞，允许绕过权限验证；
- **身份验证绕过**：认证机制设计缺陷，攻击者可以冒充他人身份或跳过认证步骤；
- **输入验证不足**：对请求参数、资源标识或边界条件的校验不充分，导致攻击者借由异常输入扩大权限边界或触发越权路径；
- **授权验证不完善**：权限检查在某些操作路径或边界条件下缺失。

3) **敏感信息泄露**指重要的身份、密钥或权限信息意外暴露，常见形式包括：

- API 端口暴露：管理接口、内部 API 或调试端口未受保护暴露在网络上；
- 日志泄露：错误泄露的日志文件中包含密钥、命令历史等敏感信息；
- 配置文件泄露：配置文件包含硬编码的凭证或密钥。

4) 代码注入指应用对用户输入的处理不当，使攻击者得以注入恶意代码，典型子类包括：

- 命令注入：应用直接执行用户提供或可控的命令，攻击者可以执行任意系统命令；
- 路径遍历：应用文件访问检查不完整，攻击者可以访问系统其他部分的文件。

4) 知识库构建

知识库构建阶段的目标是将与漏洞 v 相关的多源材料组织为带来源标注的证据集合，并建立支持按需取证的检索能力。围绕 v 构建知识库 $\mathcal{K}_v$ 后，策略生成阶段即可通过语义检索提取相关证据片段，并按需组装为安全上下文 c ，为后续分析提供可追溯的事实依据。

安全知识库基于开源仓库内多源信息构建分层向量数据库。首先自动化抓取每个漏洞关联的开源应用仓库，收集代码与文档，以及与漏洞相关的安全公告、议题（Issue）/合并请求（Pull Request）讨论和修复提交信息。在完成材料收集后，本文对不同材料采用差异化分块策略以兼顾证据粒度与可复核性：对代码和配置按行切分，同时对文档则按段落与窗口切分，并保留适当重叠以减少跨块语义断裂。为实现语义层面的证据召回，本文为每个文本块构造向量表示，并建立近邻检索索引。具体采用 `sentence-transformers` 框架中的 MPNet 句子编码器 `all-mpnet-base-v2` 完成文本表示^[75,76]，并基于 FAISS^[77] 构建相似度索引。为保证可追溯性与可复现性，索引与元数据同步持久化，内容包括向量索引文件、块级元数据以及构建配置。

上述流程得到的向量知识库为后续策略生成提供了可追溯的证据支撑：模型分析漏洞和对应防控手段时可检索相关事实与上下文片段，并通过来源标识完成定位与复核。

3.2.3 基于多智能体的安全策略生成与优化

策略生成阶段以漏洞实例 v 与安全上下文 c 为输入，在不同生成配置 g 下开展多次独立生成，并将各配置输出的安全策略子集按定义 6 汇合为候选

策略集合 Π_v 。如图 3-4所示，整个 workflow 分为安全上下文构建、基于漏洞分析思维链的策略生成、基于评审反馈的策略优化三个步骤，分别简写为 RAG, COT 和 Feedback。

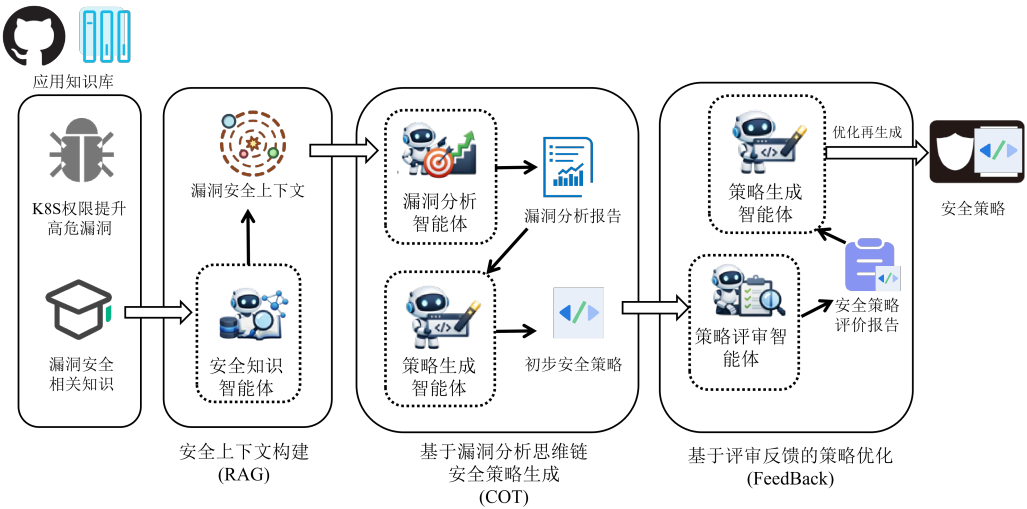


图 3-4 多智能体协同的策略化漏洞防控工作流

Fig. 3-4 Multi-agent collaborative workflow for strategy-based vulnerability prevention and control

1) 安全上下文构建

安全上下文构建中需要基于可追溯证据构建面向漏洞分析的精准上下文，以提高检测的准确性。为此构建安全知识智能体，其职责是将离散证据整理为带来源标识的结构化摘要，不直接输出策略建议。具体提示词范例见图 3-5。

安全知识智能体提示词简化示例

```
你是云原生安全知识整理助手。
## 输入
漏洞实例 v 与通过 RAG 工具检索到的候选证据片段

## 任务
提炼受影响组件、触发条件、权限边界、攻击前提与缓解线索，
按定义组织安全上下文 c。

## 输出
<< 安全上下文 c JSON 格式>>，含结构化摘要；
不直接生成安全策略。
```

图 3-5 安全知识智能体提示词简化示例

Fig. 3-5 Simplified prompt template for the security knowledge agent

2) 基于漏洞分析思维链的策略生成

漏洞分析智能体基于漏洞摘要与安全上下文 c 形成结构化分析结果，覆盖漏洞类型、攻击路径、影响范围与成因分类。成因分类直接引用第 3.2.2 节的两级框架，其作用在于提示后续防控工具的选择：一级类确定风险来源，二级类进一步锁定对应的控制手段从而将泛化建议约束为与漏洞语义一致的具体措施，例如配置缺陷类漏洞优先考虑 RBAC 最小权限与准入控制，代码注入类漏洞优先考虑运行时策略与输入过滤。策略生成智能体在上述分析约束下确定实施路径，并将其映射为策略对象；证据不足时显式保留待补充信息与必要假设。具体智能体的提示词见图 3-6 和图 3-7。

漏洞分析智能体提示词简化示例

```

你是 Kubernetes 漏洞分析助手。
## 输入
漏洞实例  $v$  与安全上下文  $c$ 

## 任务
给出漏洞类型、攻击路径、影响范围与成因分类，
并指出导致权限提升的关键环节。

## 输出
<< 漏洞分析 JSON 格式>>: 包含结构化分析结果
不直接输出安全策略。

```

图 3-6 漏洞分析智能体提示词简化示例

Fig. 3-6 Simplified prompt template for the vulnerability analysis agent

策略对象为防控条目的集合，每个条目采用 τ （控制类型）、 m （控制机理）与 a （执行表达序列）三字段模式。策略可包含多条不同类型的控制措施，覆盖监控与防护等多重目标。候选策略至少需覆盖明确的防控目标、包含可验证的实施步骤，并对潜在副作用与回滚边界作出说明。候选策略进入评估前须经规范化整理：统一字段格式、补齐必要步骤、清理冗余信息，并在证据不足处显式标记假设条件与前置依赖。

3) 基于评审反馈的策略优化

在候选策略生成之后，流程进入评审与迭代优化环节。策略评审智能体从有效性、无干扰性与可部署性三个维度对 Π_o 中的候选给出结构化评审，包括有效点、风险、验证要点与缺失信息；策略优化智能体据此合并可行措施、剔除不可行建议，并补齐前置条件与回滚方案。优化阶段的关键在于前序评审信息的吸收边界与修订依据是否得到显式保留，而非提示词的表述本身。具体提示词范例见图 3-8 和图 3-9。

策略生成智能体提示词简化示例

你是云原生安全策略生成助手，精通 Kubernetes 权限管理操作和常用云原生安全工具，如 Falco、OPA Gatekeeper、NetworkPolicy、Kyverno。

输入
漏洞实例 v 、漏洞分析结果、安全上下文 c 与生成目标

任务
围绕监控或防护目标生成候选安全策略，满足 $=\{e_1, e_2, \dots\}$ ，其中 $e_i = \langle m, a \rangle$ 为防控条目，优先给出可部署、可验证、可回滚的措施。

工具
生成代码后须调用相应工具进行校验：
- YAML/JSON 格式校验：yamllint、jq --validate
- DSL 编译校验：Falco 规则使用 falco -L 验证语法；Kyverno 策略使用 kubectl kyverno validate；OPA Rego 使用 opa check；Kubernetes 资源使用 kubectl --dry-run=server 验证。校验通过方可输出。

输出
<< 候选策略 JSON 格式>>：输出一个或多个候选，合并后构成当前生成配置下的候选子集；若信息不足，显式说明 assumptions。

图 3-7 策略生成智能体提示词简化示例

Fig. 3-7 Simplified prompt template for the strategy generation agent

策略评审智能体提示词简化示例

你是云原生安全策略评审助手。

输入
漏洞实例 v 、候选策略集合 Π_v 及补充约束

任务
从有效性、无干扰性与可部署性三个方面比较候选，指出优势、风险、验证要点与缺失信息。

输出
<< 评审结论 JSON 格式>>：包含总体结论、逐策略点评与排序建议；显式标注待补充信息。

图 3-8 策略评审智能体提示词简化示例

Fig. 3-8 Simplified prompt template for the strategy review agent

3.2.4 策略代码质量评估

策略代码质量评估有两方面意义：一方面可在候选策略集合内识别质量差异，从中选出最优策略；另一方面，构建了一种评估基准，用于比较不同大模型与智能体工作流的生成质量，以期为生产环境提供有效参考。大语言模型在绝对评分任务中难以形成跨样本一致的内部尺度函数，易受文本长度、表达风格与候选顺序等非语义因素影响，产生系统性偏置^[78-80]；排序任务将评价约化为局部比较，有利于降低尺度漂移与偏置累积，结果与人类评审的一致性通常更高^[81-83]。

安全策略代码的质量需从多维度进行综合评估。具体而言，（1）有效性：

策略优化智能体提示词简化示例

```

你是云原生安全策略优化助手。
## 输入
漏洞实例  $v$ 、候选策略集合与评审结论

## 任务
保留有效控制，删除高噪声或不可行措施，
补齐前置条件、验证步骤与回滚边界，形成优化后候选。

## 输出
<< 优化后候选策略 JSON 格式>>: 说明保留、删除与修订原因；
策略为防控条目集合  $\{e_1, e_2, \dots\}$ ，每个  $e_i = \langle t, m, a \rangle$ 。
    
```

图 3-9 策略优化智能体提示词简化示例

Fig. 3-9 Simplified prompt template for the strategy optimization agent

安全策略应能准确检测并有效防御与漏洞相关的权限提升攻击；（2）无干扰性：策略在运行时可能过度限制功能权限，故应最大限度降低对 Kubernetes 集群其他业务正常运行的干扰；（3）可部署性：策略代码应能够快速部署至 Kubernetes 集群，其中的策略条目 e 应在保持精简的前提下确保有效性。基于上述考量，本文构建了一种三维质量评价体系，从有效性、无干扰性及可部署性三个维度对策略质量进行形式化刻画，定义了相应的数值型量化指标，并给出了针对安全策略基于多源模型排序的静态方法。

定义 7（三维质量向量） 一个安全策略的质量向量量化为一个三维 0 1 的数值向量，数值越高表明质量越好：

$$q(\pi) = (E(\pi), I(\pi), D(\pi)) \quad (3-6)$$

定义 8（质量向量函数） 设 $\mathcal{H}$ 为评审主体集合， $h \in \mathcal{H}$ 为参与独立评审的大语言模型配置。在维度 $\delta \in \{E, I, D\}$ 上，评审主体 h 对候选集 Π_v 给出严格全序，记 $r_\delta^{(h)}(\pi)$ 为 π 在该排序中的秩（最优获秩 1，最劣获秩 $n_v = |\Pi_v|$ ），则该安全策略在评价维度 δ 上的质量向量值为：

$$s_\delta^{(h)}(\pi) = \frac{n_v - r_\delta^{(h)}(\pi)}{n_v - 1} \in [0, 1] \quad (3-7)$$

对所有评审主体取均值，得维度 δ 的共识得分

$$\hat{S}_\delta(\pi) = \frac{1}{|\mathcal{H}|} \sum_{h \in \mathcal{H}} s_\delta^{(h)}(\pi) \quad (3-8)$$

三维质量向量各分量由对应维度的共识得分给出，即 $E(\pi) = \hat{S}_E(\pi)$ ， $I(\pi) = \hat{S}_I(\pi)$ ， $D(\pi) = \hat{S}_D(\pi)$ 。按 $\hat{S}_\delta$ 降序排列即得维度 δ 的共识排序 $\bar{R}_\delta$ 。

定义 9（总体质量函数） 将三维共识得分加权聚合，得总体质量函数 $Q: \Pi_v \rightarrow [0, 1]$ ：

$$Q(\pi) = \alpha_E \hat{S}_E(\pi) + \alpha_I \hat{S}_I(\pi) + \alpha_D \hat{S}_D(\pi), \quad \alpha_E + \alpha_I + \alpha_D = 1 \quad (3-9)$$

其中 $\alpha_E, \alpha_I, \alpha_D$ 为维度权重, 可按业务场景配置: 应急响应场景可提高 α_E , 生产稳定性优先场景可提高 α_I , 默认取 $\alpha_E = 0.5, \alpha_I = 0.3, \alpha_D = 0.2$ 。按 $Q(\pi)$ 降序排列得总排名 $\bar{R}^\tau$ 。

3.3 方法示例

Calico 是一套开源的网络和网络安全方案, 用于容器、虚拟机、宿主机之前的网络连接。以 Calico CNI 应用中的 CVE-2024-33522 漏洞为例, 进行 KPEDefense 方法示例, 分为安全知识库构建、安全策略生成、安全策略评审三个步骤。

步骤 1 (对应 3.2.2 节): 安全知识库构建

在安全知识库构建阶段, 首先以 calico 开源仓库 (<https://github.com/projectcalico/calico>) 中通过爬虫技术抓取代码文件、文档以及 Issue, Pull Request 等多源信息, 入库时保留路径、提交哈希与时间戳等元数据。共计 42 篇 md 格式文档和 3476 条 Issue 与 8826 条 Pull Request 讨论记录。

步骤 2 (对应 3.2.3 节): 安全策略生成与优化

我们依据 KPEDefense 和 DeepSeek-V3.2 模型进行单次候选安全策略的生成与优化。首先安全知识智能体基于步骤 1 构建的安全知识库检索到的信息构建出一个安全上下文 c 如图 3-10 所示。

```
# 漏洞实例 v
id: CVE-2024-33522
d: Calico CNI 安装路径存在 SUID 位, 允许本地权限提升.....
m:
  component: calico/cni <= v3.27.2 fixed_in: v3.27.3
  cvss: 7.8 (High)
  refs: GitHub Security Advisory, fix commit

# 安全上下文 c (代表性片段)
t1: "install-cni 脚本安装时保留 /opt/cni/bin/calico 的 SUID 位..."
    src: github.com/projectcalico/calico/issues/8299
t2: "CVE-2024-33522 patch: remove chmod +s from install script..."
    src: github.com/projectcalico/calico/commit/a3f1c2d
t3: "节点加固: find /opt/cni/bin -perm -4000 定期检查 SUID 文件..."
    src: Calico_Security_Guide.md
```

图 3-10 漏洞实例 v 与安全上下文 c 示例

Fig. 3-10 Example of vulnerability instance v and corresponding security context c

漏洞分析智能体结合 c 识别出三条主要威胁成因或攻击路径, 其中直接根因是 SUID 位权限滥用, 针对 CNI 目录可疑写入是潜在的横向攻击, 而宿主机路径暴露则是可能转为持久化高风险。策略代码生成智能体针对各路径生成一个安全策略 π_1 , 其中防护策略 e_1 通过 DaemonSet 批量移除 SUID 位

的配置, 监控策略 e_2 覆盖 `/opt/cni/bin` 异常写入与可疑执行, 防护策略 e_3 限制普通工作负载挂载高风险宿主机目录。

策略评审智能体从三个维度考察 Π_v , 并输出一个简单的评审报告, 报告指出 π_1 的有效性较高但缺乏持续观测, π_2 的无干扰性最佳但覆盖面不足, π_3 可补齐路径约束但需配合其他措施。策略优化智能体据此对其进行了进一步优化, 最终的安全策略中的代码展示如图 3-11。

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: calico-cni-suid-remediate
  namespace: kube-system
spec:
  template:
    spec:
      hostPID: true
      tolerations:
        - operator: Exists
      containers:
        - name: fixer
          image: busybox:1.36
          securityContext:
            privileged: true
            command: ["/bin/sh", "-c"]
            args:
              - |
                find /host/opt/cni/bin -maxdepth 1 -perm -4000 -exec chmod u-s {} \;
                find /host/opt/cni/bin -maxdepth 1 \(\ -perm -4000 -o -perm -2000 \) -ls
          volumeMounts:
            - name: host-cni
              mountPath: /host/opt/cni/bin
          volumes:
            - name: host-cni
              hostPath:
                path: /opt/cni/bin

- rule: Write under CNI bin directory
  desc: Detect unexpected writes or permission changes under /opt/cni/bin
  condition: open_write and fd.name startswith /opt/cni/bin
  output: "Unexpected write to CNI binaries (user=%user.name file=%fd.name
  ↳ proc=%proc.cmdline)"
  priority: WARNING

apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: restrict-cni-bin-hostpath
spec:
  validationFailureAction: Audit
  rules:
    - name: forbid-cni-bin-mount
      match:
        any:
          - resources:
              kinds: ["Pod"]
      exclude:
        any:
          - resources:
              namespaces: ["kube-system", "calico-system"]
```

图 3-11 针对 CVE-2024-33522 漏洞生成的安全策略代码

Fig. 3-11 Security policy code generated for CVE-2024-33522

步骤 3（对应 3.2.4 节）：安全策略评审

多个评审模型在有效性 E 、无干扰性 I 、可部署性 D 三维上对 Π_v 独立排序, 按式 (3-8) 聚合得各策略的三维质量向量 $q(\pi)$, 再按式 (3-9) 以默认权重 ($\alpha_E = 0.5, \alpha_I = 0.3, \alpha_D = 0.2$) 计算 $Q(\pi)$ 。综合排名较优的 π_1 , 被选为最终交付策略 π^* 。

3.4 小结

本章提出了一种面向云原生权限提升漏洞的智能化防控方法。通过构建多源证据驱动的安全知识库，然后基于多智能体协同机制，将分散信息组织为可检索的安全上下文，并实现从漏洞语义到防控策略的自动化生成与优化；通过基于排序的质量评价方法，对候选策略进行有效筛选和评估。

4 面向云原生权限提升漏洞的智能化防控系统设计与实现

本章讨论 KPEDefense 系统的设计与实现，从需求分析、总体设计和详细设计三个层面展开说明。

4.1 需求分析

KPEDefense 系统围绕安全工程师与专家评审员两类角色组织核心用例，见图 4-1。安全工程师负责选择漏洞实例、发起分析会话、查看证据与候选策略，并完成部署与验证；专家评审员负责对同类型候选策略进行三维质量评审、排序确认与反馈修订。结合用例图中角色的职责以及第三章的方法链条，系统需要具备以下四项核心能力：

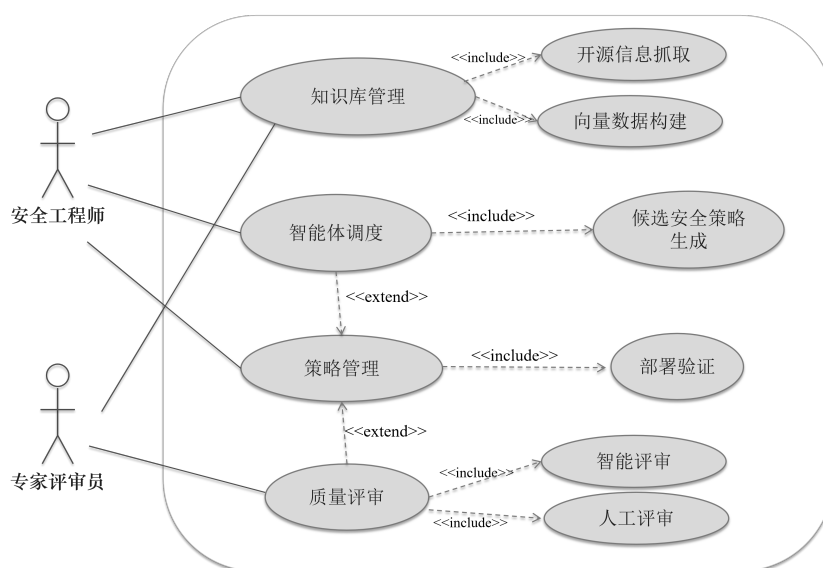


图 4-1 系统用户角色与核心用例

Fig. 4-1 System user roles and core use cases

- (1) **知识库管理**：支持自动化抓取相关开源仓库信息，可导入、检索与维护权限提升相关 CVE 条目及其关联证据。并进一步构建向量化数据库，为后续构建安全上下文 c 提供可追溯数据。
- (2) **智能体调度**：围绕漏洞实例，通过构建不同智能体进行相关安全知识上下文构建、漏洞分析等进行候选安全策略集合生成。此外可以交由策略管理模块，对生成的策略进行聚合评估与相关策略部署。
- (3) **质量评审**：对专家人工评审意见进行统计，围绕有效性、无干扰性和可部署性对同类型候选进行排序比较，并形成可审计的确认结果；同时支持

多源大模型的排名式静态评估方案，以支持策略管理。

- (4) **策略管理**：将通过评估检查的安全策略下发至目标环境进行部署验证，并记录执行状态、验证结果与审计信息。

4.2 总体设计

KPEDefense 系统采用分层架构，前端服务为界面层，后端服务包括服务层、智能引擎层和数据层，见图 4-2。本系统通过 Kubernetes 的 Deployment 资源实现前端服务与后端服务的容器化部署，并由服务层通过 Kubernetes API 与集群进行交互，Kubernetes 集群中集成了 Falco、OPA/Gatekeeper、Kynervo、NetworkPolicy 等安全工具。

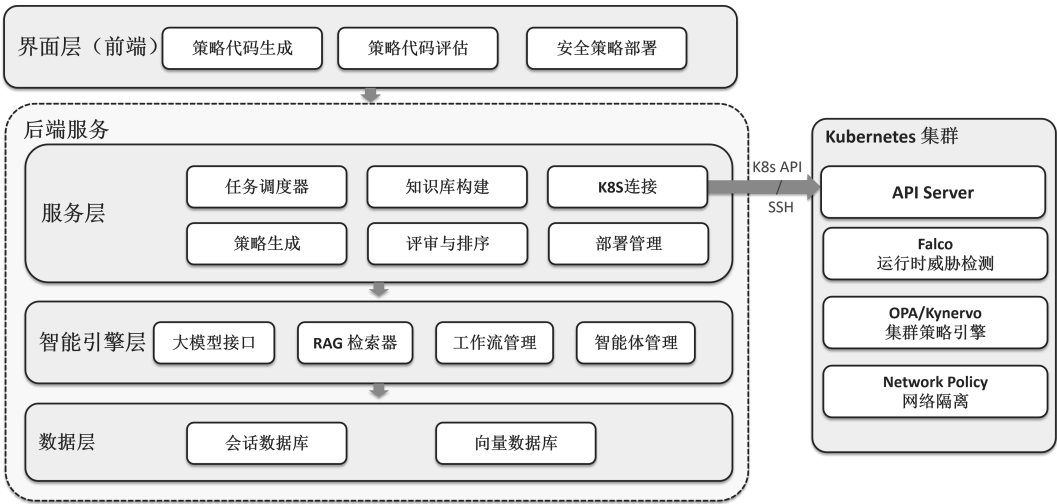


图 4-2 KPEDefense 支持工具总体架构图
Fig. 4-2 Overall system architecture of KPEDefense

- (1) **界面层**：基于 Vue3 构建单页应用，采用 Pinia 管理应用级状态，并集成 ECharts 可视化组件，负责 CVE 详情展示、策略交互与评审结果呈现。
- (2) **服务层**：采用 Flask 实现 REST API 网关，负责请求路由、访问控制与接口编排；同时通过任务调度器管理耗时较长的会话任务，并协调知识检索、候选策略生成、评审提交与部署验证等流程。
- (3) **智能引擎层**：封装系统核心分析能力，统一调度安全知识整理、漏洞分析、候选策略生成、评审反馈吸收与策略优化等智能体，完成从漏洞实例到候选策略集合 Π_v 的迭代生成。
- (4) **数据层**：负责数据持久化与检索支撑，包括存放 CVE 元数据、证据片段、中间分析结果与候选策略对象的文件存储，以及支持安全上下文构建的向量索引库。

4.3 详细设计

本小节介绍 KPEDefense 系统详细设计，包括实体类设计、详细流程设计、交互时序设计。

4.3.1 实体类设计

实体类设计围绕四类核心实体展开，如图 4-3 所示，分别对应第三章中的漏洞实例 v 、分析会话、候选策略集合 Π_v 与评审结果。**漏洞实体**（CVE）以 `status` 枚举字段驱动处置状态机，与会话呈一对多关系；**会话实体**（Session）记录模型配置与提示词模板，承载从证据检索到策略生成的完整上下文；**策略实体**（Strategy）保存生成内容、推理链快照与 RAG 召回片段，支持来源追溯与版本对比；**排序实体**（Ranking）存储评审员标识、时间戳与多维评分，同时关联会话与策略，记录专家决策的即时快照。

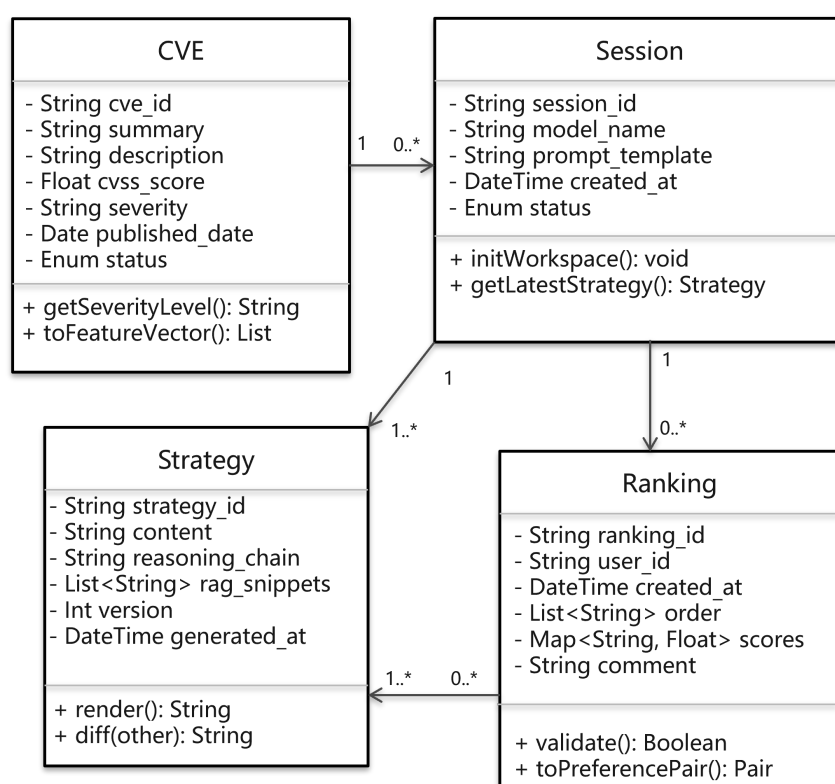


图 4-3 系统核心实体类图

Fig. 4-3 Core entity class diagram

4.3.2 详细流程设计

图 4-4 以程序流程图描述 CVE 处置总体流程。漏洞条目导入后安全工程师确认范围并启动漏洞分析与安全策略生成会话。系统会搜索已创建的安全知识库并构建对应该漏洞的安全上下文。基于安全上下文和漏洞信息，系统内通过调用不同智能体完成初步的候选安全策略集，包括多个监控策略和防护策略。针对若干不同配置生成的候选安全策略集，采用多源大模型进行排序化评审，也可增加专家评审环节。之后由相关责任人决策是否需要重新优化。若审核没问题，则交由后续的策略管理进行策略部署验证。

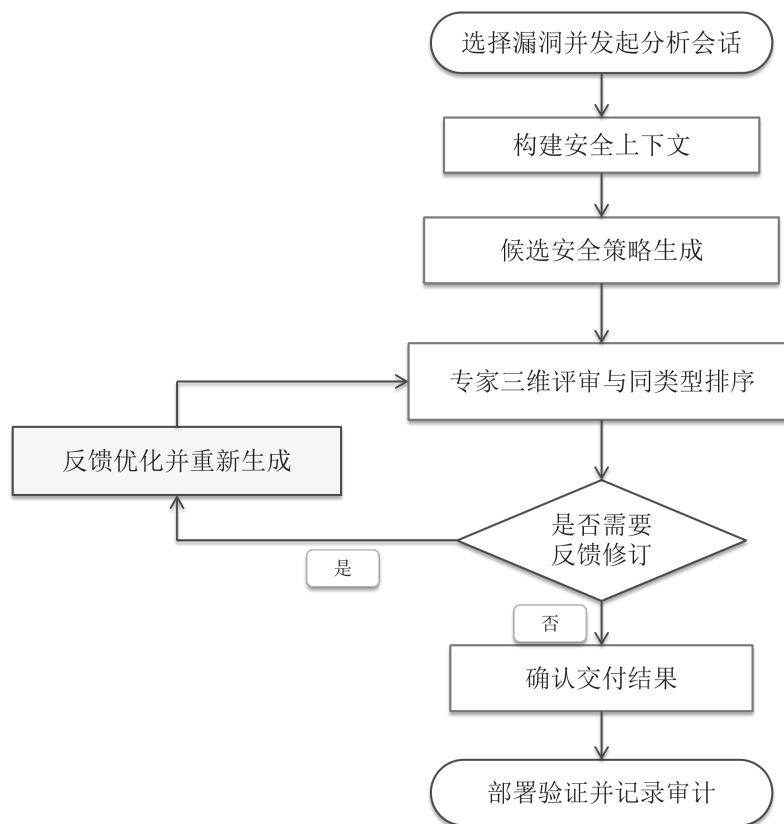


图 4-4 CVE 漏洞处置总体流程图

Fig. 4-4 Overall workflow of CVE vulnerability handling

4.3.3 交互时序设计

安全策略生成涉及证据检索、结构化分析、候选组织与反馈修订等多轮处理，单次任务耗时较长，因此系统采用异步交互时序。完整链路分为四个阶段，前两个阶段见图 4-5，后两个阶段见图 4-6。

阶段一：会话创建。安全工程师选定目标 CVE 后提交创建请求，服务层在数据持久层初始化会话目录结构并返回 `session_id`，后续所有操作均绑定

至该会话上下文。

阶段二：安全上下文构建与候选策略生成。工程师发起策略生成请求后，服务层以异步任务形式启动工作流，依次完成证据检索、安全上下文 c 构建、漏洞分析、候选策略生成与评审优化，将中间分析结果与候选策略集合 Π_v 同步写入持久层，任务完成后经轮询接口将结果返回前端。

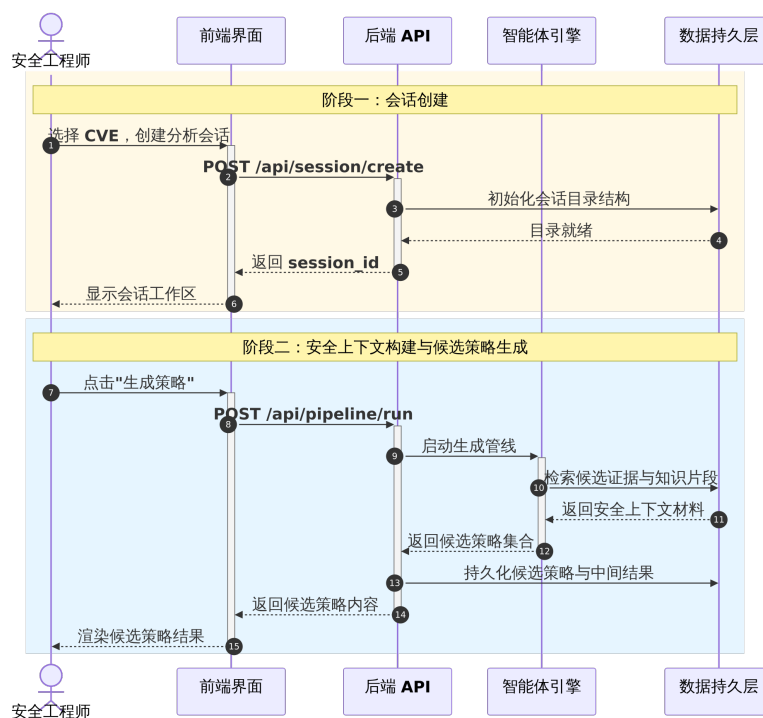


图 4-5 会话创建与候选策略生成时序图

Fig. 4-5 Session creation and candidate strategy generation sequence

阶段三：评审反馈驱动优化。专家评审员提交修改意见后，服务层携带反馈内容重新调用智能体引擎，策略优化智能体据此修订 Π_v 并将更新版本写入持久层。该阶段可按需触发多次，每次修订均以独立记录追加存储，原始版本不被覆盖。

阶段四：质量排序与结果确认。专家完成拖拽排序并填写三维评分后提交确认请求，服务层首先执行同类型候选的全序完整性校验，通过后将排序结果、维度评分与评审员标识一并写入持久层。

4.4 系统展示与验证

本节按功能模块说明系统的交互流程与实现方式，并验证系统功能。下文依次介绍知识库管理、策略生成与优化、专家评审与排序以及策略部署管理四个模块。

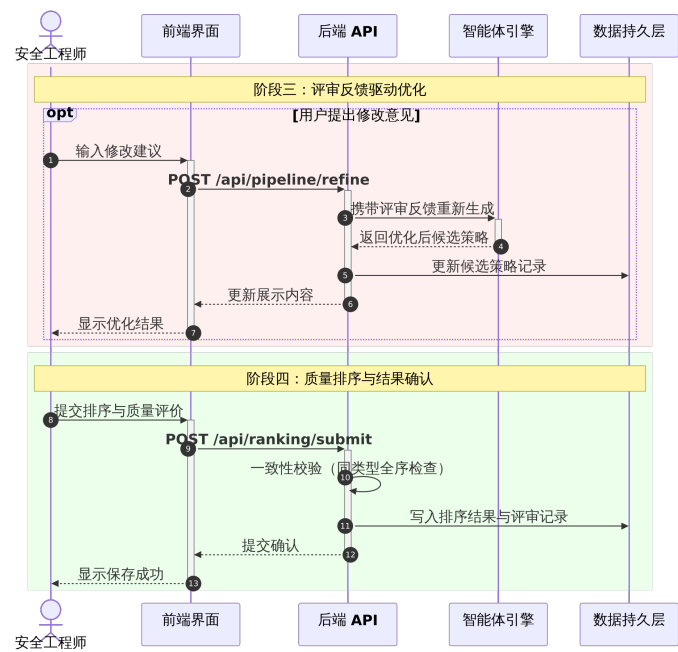


图 4-6 评审反馈优化与质量排序确认时序

Fig. 4-6 Review feedback optimization and quality ranking confirmation sequence

4.4.1 知识库管理

知识库管理模块承担权限提升相关漏洞条目的存储，是安全上下文构建的入口环节。该模块对应服务层的 CVE 知识库组件，直接影响 CVE 实体的 status 字段及后续会话流程。如图 4-7 所示，系统主界面以列表视图展示已录入的 CVE 条目，核心字段包括 CVE 编号、漏洞摘要、CVSS 风险评分和当前处置状态。支持安全工程师依据漏洞编号、漏洞类型或关键描述快速定位目标记录。

KPEAgent UI

Data Table

Raw Results

Multi Results

Strategy Ranking

维度仪表盘

多Agent生成工作流

策略部署

部署目标设置

权限提升CVE漏洞数据库

Search (coming soon)

	CVE Date	CVEID	CVSS v3.x Score	Published Date	Severity	Summary	Third-party A...	fix date	risk_level	成因
	2017-07-17T11...	CVE-2017-100...	9.8	2017/3/21	CRITICAL	Kubernetes ve...	Kubernetes	2024.3.7	低危	K8S漏洞
	2018-05-17T0...	CVE-2018-0268	10	2018 May 16	CRITICAL	A vulnerability...	Cisco DNA	2018 年 4 月 5 日	低危	网络隔离
	2018-05-18T1...	CVE-2018-100...	8.8	五月 18, 2018; ...	HIGH	Kubernetes C...	Kubernetes	6-May-18	低危	K8S漏洞
	2018-05-18T1...	CVE-2018-5256	7.5	2018/5/18	HIGH	CoreOS Tecto...	CoreOS Tectonic	(2024-08-05)	低危	内部权限失控
	2019-07-30T2...	CVE-2019-10165	2.3	Jun 7, 2019	LOW	OpenShift Co...	OpenShift	7-Jun-19	低危	敏感信息泄露
	2019-08-29T0...	CVE-2019-11247	8.1	5-Aug-19	HIGH	The Kubernet...	Kubernetes	1-Aug-19	低危	过度权限配置
	2019-03-28T1...	CVE-2019-3869	7.2	2019/03/13	HIGH	When running ...	Tower	27-Mar-19	低危	内部权限失控
	2020-12-01T0...	CVE-2020-15257	5.2	1-Dec-20	MEDIUM	containerd is ...	containerd	1-Dec-20	低危	容器运行时漏洞
	2021-06-07T2...	CVE-2020-1742	7	六月 07, 2021; ...	HIGH	An insecure m...	nmstate/kube...	2020 年 5 月 4 日	低危	代码注入
	2020-06-22T1...	CVE-2020-4062	9	19-Jun-20	CRITICAL	In Conjur OSS ...	Conjur OSS	19-Jun-20	低危	网络隔离
2020-07-22T1...	CVE-2020-8559	6.8	9-Jul-20	MEDIUM	The Kubernet...	Kubernetes	13-Jul-20	低危	敏感信息泄露	

图 4-7 CVE 知识库查看与管理界面

Fig. 4-7 Interface for CVE knowledge base browsing and management

4.4.2 智能体调度

智能体调度模块负责将漏洞证据转化为可交付的候选策略，并通过评审反馈持续优化安全策略。系统以会话为基本单元，围绕证据检索、思维链分析、候选策略生成、评审与优化五个阶段组织处理流程。如图 4-8所示，工作界面左侧支持选择目标 CVE 并创建会话，右侧以进度条展示各阶段的运行状态。

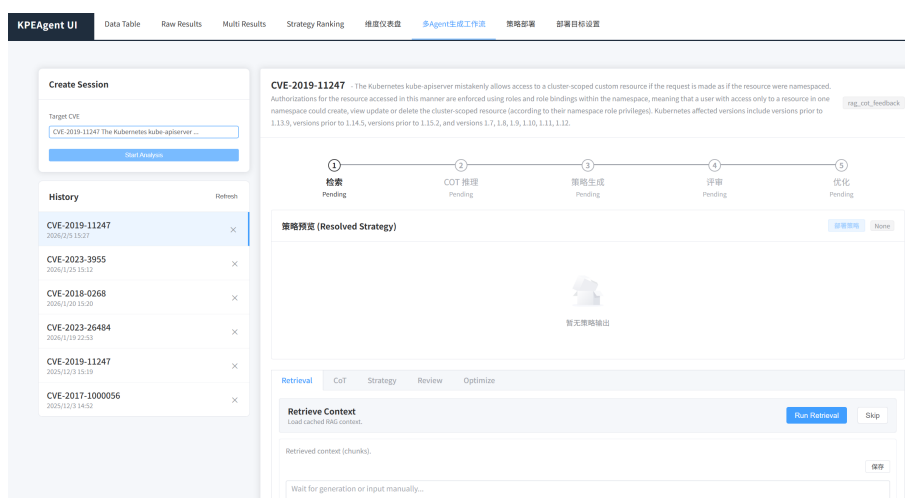


图 4-8 会话创建与 workflow 开始界面
Fig. 4-8 Session creation and workflow initiation interface

会话创建完成后，系统依据漏洞编号、漏洞描述与知识库范围生成检索请求，并在后台异步执行证据召回。证据检索阶段（图 4-9）从漏洞实例抽取语义特征，在知识库中召回相关片段，界面同时展示来源文档与相关度得分，供人工核验。

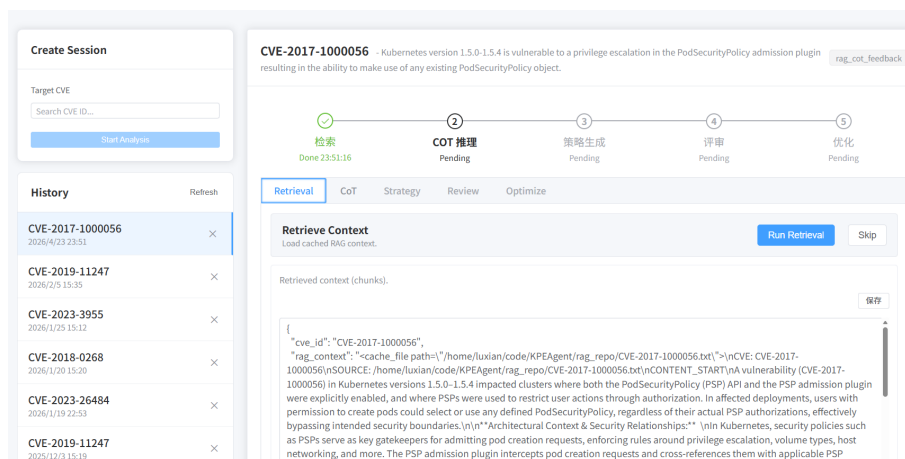


图 4-9 安全上下文检索结果展示
Fig. 4-9 Display of security-context retrieval results

完成安全上下文构建后，开始 COT 推理阶段，如图 4-10 所示。该阶段支持底层模型切换，点击“Run COT”按钮后，系统调用指定模型对检索到的安全上下文开展引导式推理，分析漏洞的成因，攻击模式，可能的防控手段等，形成后续策略生成所需的结构化结论。上述分析结果以结构化字段形式写入持久层，供后续评审、策略优化与部署验证阶段使用。

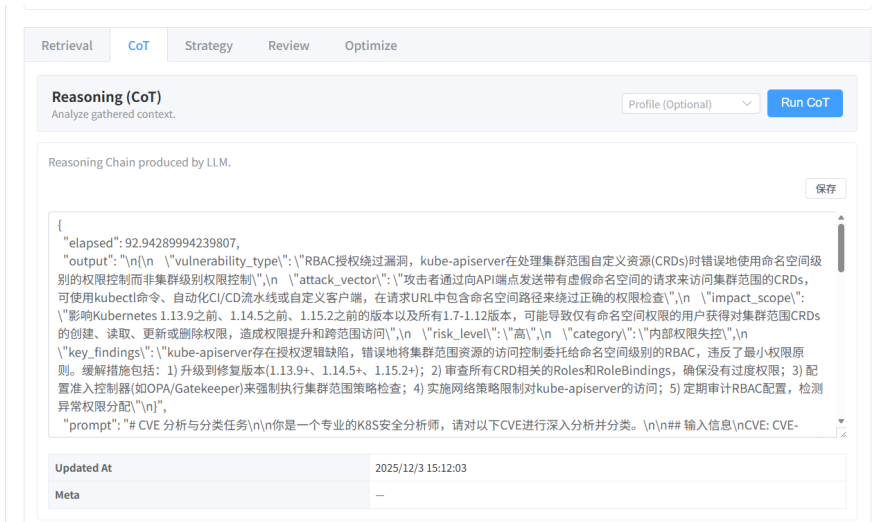


图 4-10 COT 推理模型选择与启动界面

Fig. 4-10 CoT reasoning model selection and launch interface

完成思维链推理后系统进入候选策略生成阶段，如图 4-11 所示。该阶段以前序检索结果与结构化分析结论为输入，生成可供后续评审的候选安全策略集合。每个安全策略条目以描述摘要和具体工具代码的形式展示。

策略生成完成后，可调用评审智能体对候选策略展开结构化评审（图 4-12），输出内容包含有效点、风险项、信息缺口（gaps）与干扰源（noise_sources），为后续优化提供明确依据。与仅返回单一结论的生成方式相比，这种结构化反馈更有利于定位候选策略中的薄弱环节，并支持按问题类型进行针对性修订。同时这部分同时支持人工输入评审提示词，增强评审意见的有效性。

完成评审与迭代优化后，系统输出候选策略集合的最终展示结果（图 4-13），工程师可在界面查看各候选的完整控制条目并发起部署。最终展示阶段不仅呈现候选内容本身，还保留了候选来源、评审意见与版本演化信息，从而为后续人工确认与策略落地提供完整依据。

4.4.3 质量评审

质量评审模块为安全专家提供安全策略的排序式评价界面。评审阶段收集围绕有效性、干扰风险与前置条件的逐项反馈，驱动策略优化。用户可展

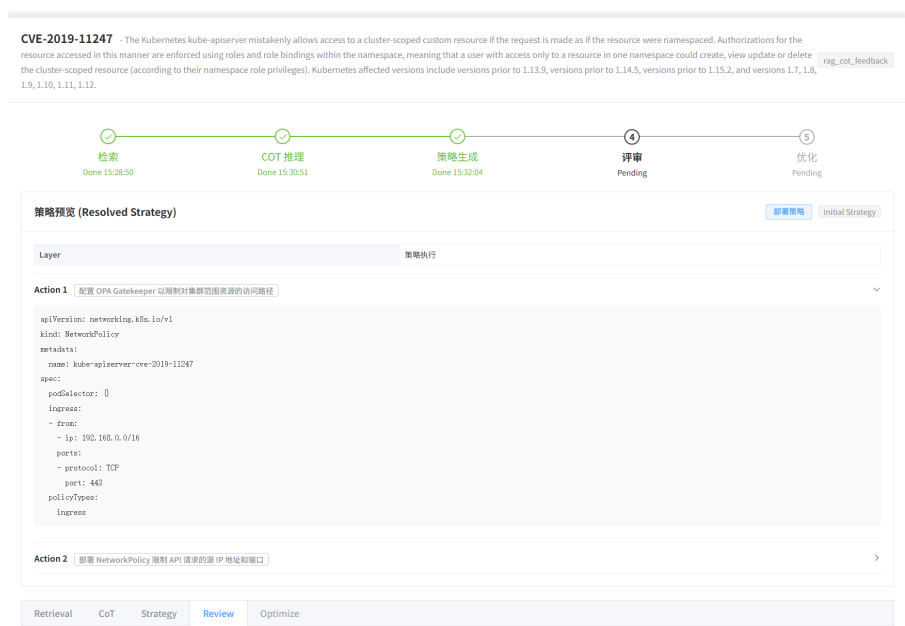


图 4-11 候选策略生成完成后进入评审阶段

Fig. 4-11 Transition to the review stage after candidate strategy generation



图 4-12 评审智能体结构化评审输出

Fig. 4-12 Structured output of the review agent

开候选策略详情面板查看完整执行步骤与策略 JSON 内容，以支持基于实施细节的评价。排序阶段如图 4-14 所示，专家对同类型候选进行拖拽排序以进行质量评估；系统提交前执行候选策略完整性校验，确认后将排序结果、评分评价与评审员标识写入持久层，形成可审计的数据集。

排序提交页面（图 4-15）支持填写排序理由并查看历史排序记录，历史记录按三个评价维度分别展示每次提交的候选策略排序结果，便于对比多轮评审意见的演变趋势。

4.4.4 策略管理

策略部署管理模块将通过评审的候选策略下发至目标环境，支持 Kubernetes（Python SDK 连接 API Server）与 SSH 两种部署方式。部署前需在目标

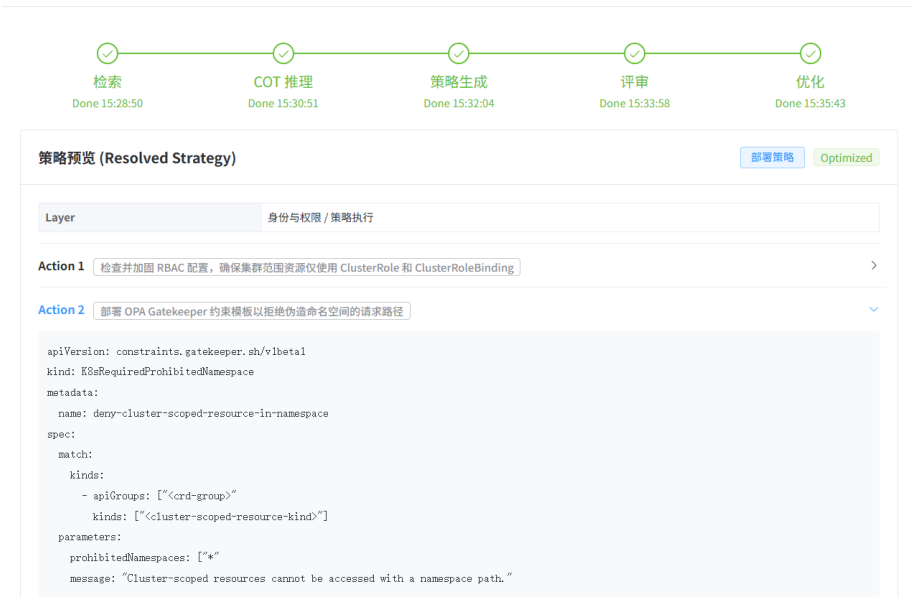


图 4-13 候选策略集合最终生成结果展示

Fig. 4-13 Final display of the candidate strategy set

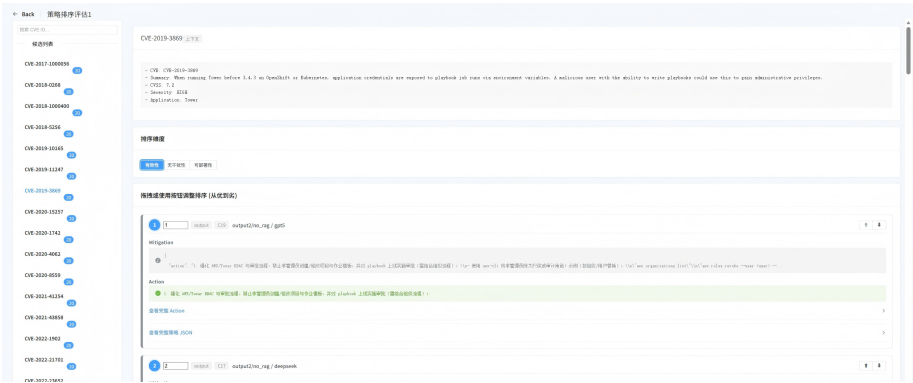


图 4-14 策略拖拽排序主界面

Fig. 4-14 Main interface for drag-and-drop strategy ranking

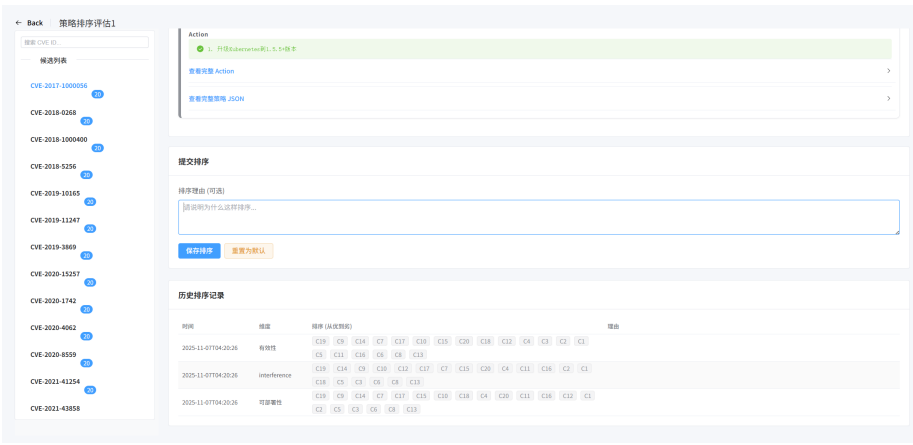


图 4-15 排序提交页面 (包含排序理由输入与历史记录)

Fig. 4-15 Ranking submission page with reason input and historical ranking records

设置页面 (图 4-16) 配置连接参数: kubectl 类动作通过 Python SDK 直连 API Server, 仅需 kubeconfig 文件路径; SSH 方式则用于执行非 kubectl 的 shel-

l/helm 命令，需配置主机地址、用户名与密钥路径。如图 4-17 所示，部署管理面板统一展示跨集群、跨主机的策略实例，记录策略名称、目标对象、部署时间与状态。

图 4-16 部署目标连接配置页面

Fig. 4-16 Deployment target connection configuration page

图 4-17 策略部署管理面板

Fig. 4-17 Strategy deployment management panel

4.5 小结

本章完成了 KPEDefense 系统的设计与实现。需求分析部分明确了两类用户角色和四项功能需求；总体设计部分给出了分层架构、数据模型、详细流程和交互时序；详细设计部分说明了知识库管理、智能体调度、策略评审以及策略部署管理四个模块的实现与交互方式。

5 实验研究

本章将近五年的云原生权限提升漏洞作为实验对象，评估 KPEDefense 的合理性和有效性，及其与现有静态分析工具的优势。

5.1 研究问题

本章围绕以下问题展开实验：

- (1) 相较于 KPEDefense, 现有静态分析方法的局限性如何？通过与常见 Kubernetes 安全风险静态扫描工具的对比，验证 KPEDefense 在权限提升漏洞防控的优势。
- (2) 方法设计中三个核心部分在质量指标上的贡献如何？需要通过消融实验验证检索增强、思维链推理与反馈优化分别对策略有效性、无干扰性与可部署性的影响，以及这种影响在不同模型下是否稳定。
- (3) 不同大语言模型在策略生成任务中的能力差异如何？需要评估不同模型在三维质量指标和输出稳定性上的表现差异。
- (4) 基于大语言模型的静态评估方法是否具备足够的可信度？需要从评审模型间的一致性以及与人工评审的对照结果两个角度进行验证。
- (5) KPEDefense 的时间与经济成本如何？需要量化不同配置下的词元消耗、时间开销和经济成本，为实际部署中的选型提供依据。

5.2 实验对象

本章实验以第三章收集的 55 个 Kubernetes 生态权限提升漏洞实例为对象。在生成模型方面，本章选取截至 2025 年 10 月份五个具有代表性的大语言模型参与策略生成与评审。表 5-1 列出各模型的基本信息。五个模型在架构设计、训练数据和参数规模上存在差异，可用于考察方法在不同模型条件下的适用性。

表 5-1 实验采用的大语言模型
Tab. 5-1 Large language models used in experiments

模型代号	开发机构	发布日期	参数规模
Qwen3-Max	阿里云	2025 年 9 月	1000B+
DeepSeekV3.2	深度求索	2025 年 9 月	670B
GPT-5	OpenAI	2025 年 6 月	未公开
Grok4	xAI	2025 年 7 月	未公开
GLM-4.5	智谱 AI	2025 年 7 月	355B

5.3 实验设计

5.3.1 度量指标

策略质量评估指标共识排序 $\hat{S}_\delta(\pi)$ (式(3-8)) 计算的三维质量向量 $q(\pi) = (E(\pi), I(\pi), D(\pi))$ (定义 7)。此外, 为衡量评审模型之间的排序一致性, 引入 Kendall's W 协调系数^[84] (定义 10) 与 Spearman 等级相关系数^[85] (定义 11) 作为辅助度量; 资源消耗则记录每次调用的输入与输出词元数及运行时间, 按模型和方法配置分别聚合统计。

定义 10 (Kendall's W 协调系数) 设 $|\mathcal{H}|$ 个评审主体对 $n_v = |\Pi_v|$ 个候选策略在同一维度 δ 上独立给出严格全序排名, 令 $r_\delta^{(h)}(\pi)$ 为评审主体 h 对策略 π 在维度 δ 上的秩, $R_i = \sum_{h \in \mathcal{H}} r_\delta^{(h)}(\pi_i)$ 为候选 π_i 的秩和, $\bar{R} = \frac{1}{n_v} \sum_{i=1}^{n_v} R_i$ 为秩和均值, 则 Kendall's W 定义为

$$W = \frac{12S}{|\mathcal{H}|^2(n_v^3 - n_v)}, \quad S = \sum_{i=1}^{n_v} (R_i - \bar{R})^2 \quad (5-1)$$

$W \in [0, 1]$, $W = 1$ 表示所有评审主体排序完全一致, $W = 0$ 表示排序彼此随机无关。

定义 11 (Spearman 等级相关系数) 设评审主体 h_1 与 h_2 对同一候选集 Π_v 在维度 δ 上给出秩向量, 对任意候选 π_i 令 $d_i = r_\delta^{(h_1)}(\pi_i) - r_\delta^{(h_2)}(\pi_i)$ 为两者秩之差, 则 Spearman 等级相关系数定义为

$$\rho_{h_1, h_2}^\delta = 1 - \frac{6 \sum_{i=1}^{n_v} d_i^2}{n_v(n_v^2 - 1)} \quad (5-2)$$

$\rho \in [-1, 1]$, $\rho = 1$ 表示两位评审主体排序完全吻合, $\rho = -1$ 表示完全相反。实验中对所有评审主体对 $(h_1, h_2) \in \binom{\mathcal{H}}{2}$ 计算该系数, 并汇报其分布的均值、中位数与极值。

5.3.2 基线方法

为与传统静态安全工具进行比较，本文选取四种代表性工具或文献作为基线。在得到风险合规报告后，再人工评估其对策略生成的实际帮助，包括能否定位与漏洞路径相关的高风险配置，以及是否给出可直接落地的修正建议。

表 5-2 静态扫描工具
Tab. 5-2 Static scanning tools

工具	来源
kube-linter	https://github.com/stackrox/kube-linter
Kubescape	https://github.com/kubescape/kubescape
KubeSec	https://kubesecc.io/
SLI-Kube	Rahman 等 ^[13]

5.3.3 实验过程

实验围绕候选策略生成、自动评审和人工对照三个环节展开。首先，在统一的提示词模板、输入组织方式和输出结构约束下，分别使用五个大语言模型对 55 个漏洞实例生成候选策略集合 Π_v （定义 6）。为评估各关键模块的作用，设置三项消融配置：移除安全上下文（KPEDefense w/o RAG）、移除思维链推理（KPEDefense w/o COT）和移除反馈优化（KPEDefense w/o Feedback），并与完整工作流程（KPEDefense）进行对比。方法配置 a 与底层模型 h 的叉积共构成 20 种生成配置 $g = (a, h)$ ，如表 5-3 所示。除被移除模块外，其他模块通过修改后的简化提示词链接。

表 5-3 生成配置的完整枚举编号
Tab. 5-3 Complete enumeration of generation configurations

模型 h	KPEDefense	KPEDefense w/o RAG	KPEDefense w/o COT	KPEDefense w/o Feedback
Qwen3-Max	g_1	g_2	g_3	g_4
DeepSeekV3.2	g_5	g_6	g_7	g_8
GPT-5	g_9	g_{10}	g_{11}	g_{12}
Grok 4	g_{13}	g_{14}	g_{15}	g_{16}
GLM-4.5	g_{17}	g_{18}	g_{19}	g_{20}

在评审阶段，将上述五个模型同时作为评审模型，对每个漏洞实例对应的候选策略在三个评价维度上分别进行独立排序。为降低评审偏差，评审前对候选策略进行随机化与匿名化处理，打乱展示顺序并隐去来源信息，以减少顺序效应和模型偏好对结果的影响。随后，对各评审模型给出的排序结果进行等权聚合，得到该漏洞实例上的共识排序 $\hat{S}_\delta(\pi)$ （式(3-8)），作为自动评审的最终结果。

针对所有 CVE 生成的候选安全策略集合进行人工独立评审，以检验自动评审结果与人工判断的一致性。人工评审与自动评审采用相同的评价维度、排序规则和候选展示方式，从而保证两者具有可比性。通过自动评审与人工对照相结合的方式，可以进一步验证评审结果的可靠性，并为分析自动评审方法的有效性提供依据。

此外，对所有 CVE 相关的应用版本代码使用四种静态工具进行风险扫描，生成扫描结果并统计风险报告。

5.4 结果分析与讨论

本节依次从静态工具对比、消融实验、生成模型能力、评审可信度、和资源消耗五个方面展开分析。

5.4.1 静态工具对比

现有静态工具对权限提升 CVE 防控的参考价值有限。本文选取 Kubescape、KubeSec、kube-linter 和 SLI-Kube 作为对照工具，在相关 Kubernetes 部署清单上进行扫描，并将报告条目按统一风险类型归并统计，结果见表 5-4。四类工具的报告主要集中在配置基线与通用加固项，如资源配置缺失、身份与密钥暴露、网络隔离和非 Root 运行。这类结果对日常治理和合规检查仍有价值，但与本文收集的 55 个权限提升漏洞逐一对应时，能够直接指向漏洞触发前提的情况并不多。只有 CVE-2020-4062 与 CVE-2024-31989 在报告中出现了较明确的关联线索，而且都落在网络隔离项上，分别由 Kubescape 与 SLI-Kube 命中。其余样本的关键攻击路径通常依赖具体组件行为、权限边界或运行时条件，超出了静态规则直接覆盖的范围。就补丁窗口期的漏洞防控而言，静态扫描更适合作为保障工具，而 KPEDefense 更适合提供面向具体漏洞场景的处置建议。**KPEDefense** 的有效性将通过第6章案例分析进一步说明。

表 5-4 静态扫描工具检测结果
Tab. 5-4 Detection results of static scanning tools

风险类型	分类	Kubescape	KubeSec	kube-linter	SLI-Kube
资源配置缺失	配置缺陷	110	47	73	0
非 Root 运行	配置缺陷	54	53	51	0
根文件系统非只读	配置缺陷	39	34	59	0
网络隔离	配置缺陷	132	0	0	15
特权与权限提升	配置缺陷	39	0	12	0
主机级暴露	配置缺陷	30	0	9	3
身份与密钥	敏感泄露	192	58	0	2
系统调用隔离	配置缺陷	0	58	0	1
探针配置	配置缺陷	92	0	15	0

5.4.2 方法部分消融实验贡献度

消融实验表明,三个模块均能提升策略质量,其中反馈优化的影响最为显著。本节比较完整工作流 KPEDefense 与三种消融配置 KPEDefense w/o RAG、KPEDefense w/o COT、KPEDefense w/o Feedback,评价维度为有效性 E 、无干扰性 I 与可部署性 D 。55 个漏洞样本先由五个生成模型产出候选策略,再交由五个评审模型独立排序,最后经等权聚合得到各维度共识得分。图 5-1 与表 5-5 给出排名对比和具体数值。其中反馈优化 (Feedback) 移除后有效性降幅达 79%,无干扰性与可部署性也同步出现最大下降。多轮反馈迭代对各维度质量的提升作用最为全面,然而代价是更高的词元消耗与时间开销。思维链推理 (COT) 的影响则呈现明显的维度分化:移除后有效性降至 0.234,但可部署性几乎不变 ($D = 0.403$ 降至 $D = 0.398$)。

表 5-5 消融实验方法配置得分
Tab. 5-5 Average scores in ablation study

方法配置	有效性 (E)	无干扰性 (I)	可部署性 (D)
KPEDefense	0.581	0.431	0.403
KPEDefense w/o RAG	0.476	0.370	0.321
KPEDefense w/o COT	0.234	0.340	0.398
KPEDefense w/o Feedback	0.121	0.269	0.288

安全上下文 (RAG) 的贡献度相对较小。移除后有效性降至 0.476,降幅

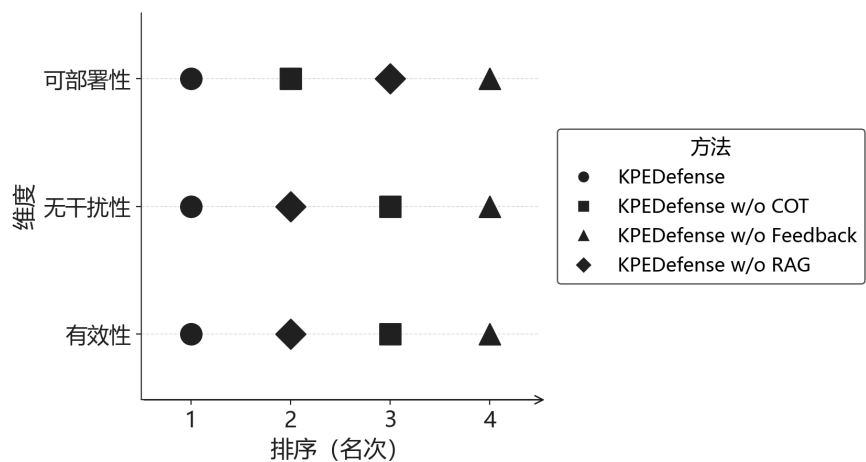


图 5-1 方法配置在三个维度上的总体排名

Fig. 5-1 Comparison of method configurations across three dimensions

约 18%，无干扰性和可部署性也分别下降约 14% 与 20%。这与当前大语言模型的训练语料已大量涵盖开源仓库及相关 CVE 记录有关，使得去除检索模块后模型仍能在一定程度上获取漏洞相关知识。

5.4.3 生成模型能力评估

本节将讨论对象由 workflow 模块转向生成模型本身，分别从三维质量指标和输出稳定性两个方面展开。

五个生成模型在三个维度上并无统一的优劣排序，不同模型的优势维度存在明显分化。图 5-2 展示了各模型在有效性 E 、无干扰性 I 与可部署性 D 三个维度上的排名分布。GPT-5 在有效性维度上明显领先，得分达到 $E = 0.814$ ，但无干扰性和可部署性分别只有 0.175 和 0.215，说明它更容易给出约束较强、处置力度较大的方案。DeepSeek 则呈现出相反倾向，其无干扰性和可部署性分别达到 $I = 0.497$ 和 $D = 0.605$ ，均为五个模型中最高，但在有效性上并不占优，因此更倾向于生成改动范围较小、部署代价较低的策略。其余模型位于这两种倾向之间。

选型时需根据场景优先级权衡维度侧重。GPT-5 在有效性上优势明显且输出零失误，适合以封堵充分性为首要目标的场景；DeepSeek 在无干扰性和可部署性上领先，更适合业务连续性要求较高的环境。Qwen3-Max、GLM-4.5 和 Grok4 各维度表现较为均衡，可在不同成本约束下提供可用的折中方案。

在输出稳定性方面，统计了各模型在生成阶段触发重试概率如表 5-7 所示。重试主要对应三类失败情形：结构化输出中的 JSON 结构错误，生成内容的语法错误，以及内容违规。各模型的输出稳定性差异显著。GPT-5 在三

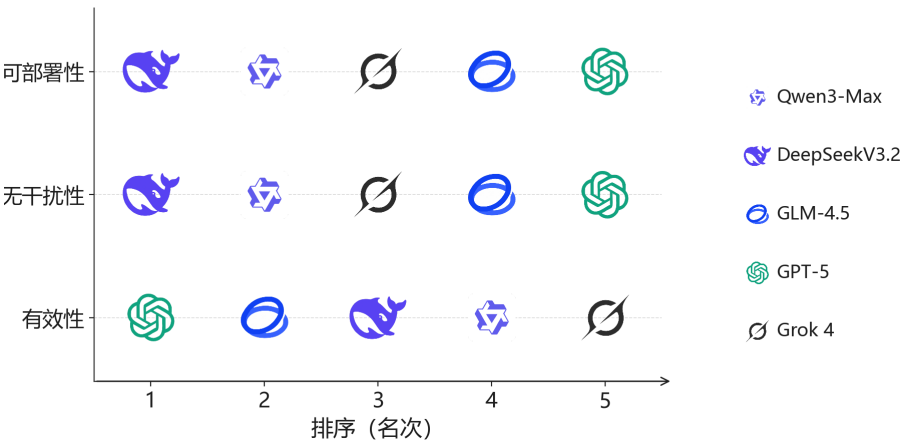


图 5-2 生成模型在三个维度上的排名对比
Fig. 5-2 Comparison of generation model rankings across three dimensions

表 5-6 生成模型三维质量得分
Tab. 5-6 Quality scores of generation models

生成模型	有效性 (<i>E</i>)	无干扰性 (<i>I</i>)	可部署性 (<i>D</i>)
GPT-5	0.814	0.175	0.215
GLM-4.5	0.327	0.324	0.336
DeepSeekV3.2	0.284	0.497	0.605
Qwen3-Max	0.193	0.425	0.477
Grok4	0.147	0.342	0.346

类错误上均未触发重试，输出最为稳定；Grok 4 和 DeepSeek 的总失误差也较低（0.99% 和 1.59%）。GLM-4.5 的总失误差最高（8.53%），主要集中在 JSON 格式错误，且是唯一出现内容违规的模型。

表 5-7 生成模型输出稳定性统计
Tab. 5-7 Output stability statistics of generation models

错误类型	DeepSeek V3.2	GLM -4.5	GPT -5	Grok 4	Qwen3 -Max
JSON 格式错误	0.99%	4.56%	0.00%	0.99%	2.58%
语法错误	0.60%	1.98%	0.00%	0.00%	0.99%
内容违规	0.00%	1.98%	0.00%	0.00%	0.00%
合计	1.59%	8.53%	0.00%	0.99%	3.57%

5.4.4 评审方法可信度评估

前两节分别讨论了 workflow 差异和模型差异，而这些比较都建立在评审结果足够可靠的前提之上。本节从内部一致性和与人工评审一致性两个角度验证这一前提。

本小节使用度量指标中定义的 Kendall’s W 协调系数与 Spearman 等级相关系数，考察五个评审模型在三个维度上的排序一致性。表 5-8 给出量化结果。

表 5-8 不同维度下大语言模型评审一致性统计
Tab. 5-8 Consistency statistics of large-language-model judges across dimensions

维度	Kendall’s W	Spearman 相关系数			
		均值	中位数	最小值	最大值
有效性	0.955	0.944	0.944	0.904	0.991
无干扰性	0.846	0.808	0.839	0.585	0.932
可部署性	0.687	0.609	0.689	0.238	0.947

基于多源模型的评审方法在有效性和无干扰性方面性能较好。有效性维度上的一致性最高。表 5-8 显示，有效性维度的 Kendall’s W 达 0.955，Spearman 均值为 0.944，说明不同评审模型对漏洞封堵充分性的判断标准较为接近。无干扰性居中（ $W = 0.846$ ）。可部署性上的分歧最大（ $W = 0.687$ ），GPT-5 与其他模型的差异尤为突出，这可能与各模型对部署步骤复杂度的理解方式不同有关。图 5-3 左图展示了生成模型在人工评审与多源模型评审下的雷达图对比，两者在有效性维度上的判断较为一致，差距主要体现在可部署性方面；考虑到有效性是防控策略质量的首要考量，这一差异在可接受范围内。

与人工评审结果对照显示，在方法层面多源模型评审方法可以在三个维度表征不同方法的优劣。图 5-3 右图展示了人工评测与多源模型评测的雷达图对比，两者在方法配置的总体排名上基本保持一致，差别主要出现在可部署性上，人工评审对部署难度的区分更敏感。这为未来其他智能体安全策略生成方法提供了有效的测试基准方法。

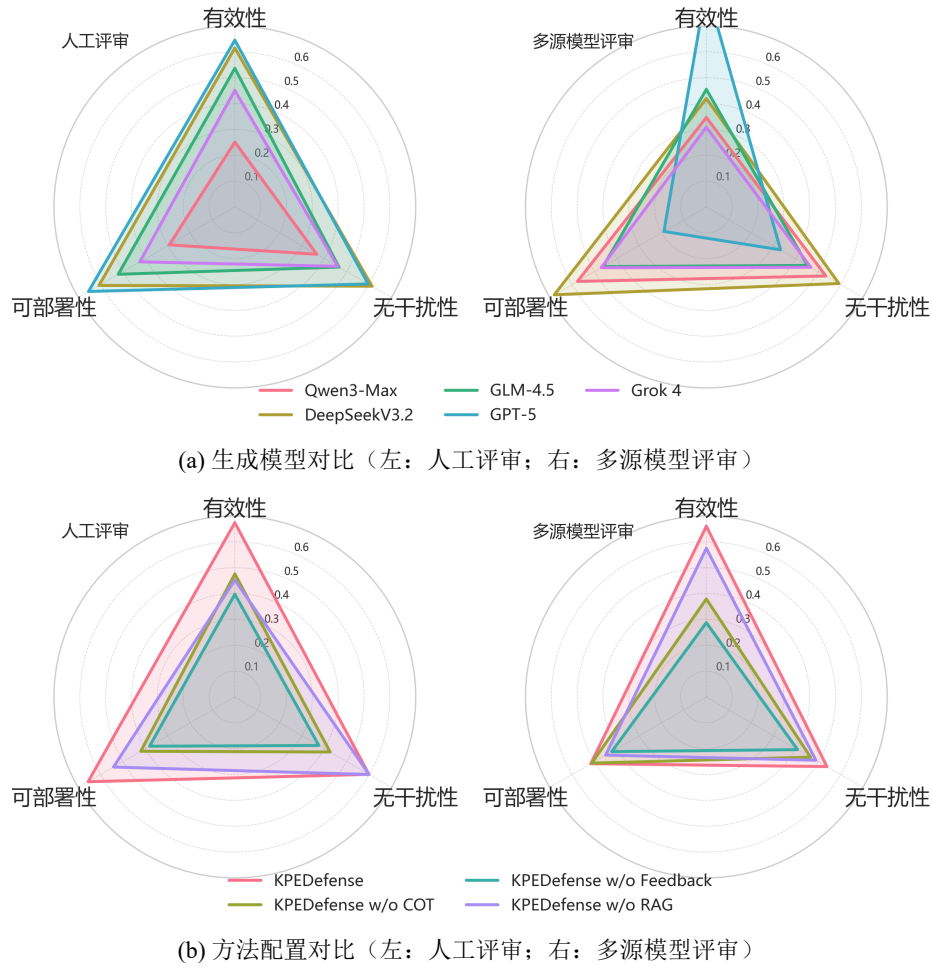


图 5-3 人工评审与多源模型评审的三维度雷达图对比

Fig. 5-3 Comparison of three-dimensional radar charts between human evaluation and multi-model evaluation

5.4.5 资源消耗与效率评估

本节从词元成本与时间开销两个维度考察 KPEDefense 的资源代价。表 5-9 给出不同方法配置与生成模型下的平均词元消耗及对应成本：

反馈优化是词元消耗的主要来源，模型间成本差异显著。完整 workflow 均值 12.99 元，去掉反馈后降至 9.88 元（降幅 24%）；去掉 RAG 后为 11.10 元，说明检索上下文也带来可观开销。模型间差异更突出：GPT-5 单次 50.40 元，输出端占 47.11 元，倾向生成篇幅较长的策略文本；GLM-4.5 和 Qwen3-Max 则分别仅 0.49 元和 1.50 元，成本显著更低。

时间开销与词元成本的格局一致，反馈迭代同样是最主要的时间瓶颈。完整 workflow 方法均值 146.39 秒，去掉反馈后降至 86.98 秒（降幅 41%）；去掉 COT 的变化较小（135.64 秒），说明思维链推理的时间额外开销有限。模型间分布悬殊：GPT-5 完整配置下单次耗时 354.48 秒，去反馈后仍达 228.50

表 5-9 词元消耗与经济成本统计
Tab. 5-9 Token usage and economic cost statistics

方法	模型	输入 Token	输出 Token	输入成本 (CNY)	输出成本 (CNY)	总成本 (CNY)	平均成本 (CNY)
KPEDefense	Qwen3-Max	228842	92360	0.57	0.92	1.50	12.99
	DeepSeekV3.2	213174	360923	0.51	2.60	3.11	
	GLM-4.5	217216	158493	0.17	0.32	0.49	
	GPT-5	376812	672931	3.30	47.11	50.40	
	Grok4	246549	108188	2.96	6.49	9.45	
KPEDefense w/o COT	Qwen3-Max	147128	44759	0.37	0.45	0.82	11.29
	DeepSeekV3.2	143191	310209	0.34	2.23	2.58	
	GLM-4.5	138905	141923	0.11	0.28	0.39	
	GPT-5	201623	646417	1.76	45.25	47.01	
	Grok4	165321	61481	1.98	3.69	5.67	
KPEDefense w/o Feedback	Qwen3-Max	129241	44841	0.32	0.45	0.77	9.88
	DeepSeekV3.2	120138	280609	0.29	2.02	2.31	
	GLM-4.5	122551	155095	0.10	0.31	0.41	
	GPT-5	178449	554871	1.56	38.84	40.40	
	Grok4	145538	62736	1.75	3.76	5.51	
KPEDefense w/o RAG	Qwen3-Max	184820	46291	0.46	0.46	0.92	11.10
	DeepSeekV3.2	170295	318002	0.41	2.29	2.70	
	GLM-4.5	170290	116108	0.14	0.23	0.37	
	GPT-5	338456	603583	2.96	42.25	45.21	
	Grok4	203726	64326	2.44	3.86	6.30	

秒；Qwen3-Max 和 Grok4 在所有配置下均维持在 15~39 秒，响应更及时。DeepSeek 耗时较高（225.20 秒）但主要来自 API 延迟，而非词元量本身。

不同场景应按响应窗口与预算约束进行选型。若优先控制成本与响应时间，Qwen3-Max 和 Grok4 较为合适，两者单次成本分别为 1.50 元和 9.45 元，均可在 40 秒内完成一轮处置。若以有效性为首要目标，GPT-5 代价最高但封堵能力最强，适合高危漏洞场景。DeepSeek 时间成本较高，但在无干扰性和可部署性上具有优势，更适用于对业务连续性要求较高的生产环境。

5.5 小结

本章围绕 KPEDefense 开展实验验证，从消融实验、生成模型能力、评审方法可信度、静态工具对比和资源消耗五个方面进行了分析。实验结果验证了 KPEDefense 在策略生成与评审中的有效性，并展示了不同模型与配置在质量和成本上的差异。

表 5-10 时间成本统计
Tab. 5-10 Time cost statistics

方法配置	模型	平均耗时 (秒)	方法均值 (秒)
KPEDefense w/o RAG	Qwen3-Max	27.57	132.48
	DeepSeekV3.2	212.84	
	GLM-4.5	55.34	
	GPT-5	340.42	
	Grok4	26.23	
KPEDefense w/o COT	Qwen3-Max	26.22	135.64
	DeepSeekV3.2	219.34	
	GLM-4.5	74.22	
	GPT-5	328.47	
	Grok4	29.93	
KPEDefense w/o Feedback	Qwen3-Max	15.94	86.98
	DeepSeekV3.2	140.94	
	GLM-4.5	35.51	
	GPT-5	228.50	
	Grok4	14.00	
KPEDefense	Qwen3-Max	38.93	146.39
	DeepSeekV3.2	225.20	
	GLM-4.5	75.78	
	GPT-5	354.48	
	Grok4	37.55	

6 案例分析

本章选取 7 个典型 CVE 案例，考察 KPEDefense 针对真实云原生权限提升漏洞进行安全防控的有效性。每个案例包括漏洞复现、策略生成与部署验证三个阶段。

6.1 案例分析实验设计

6.1.1 CVE 案例选择

为保证案例具有代表性且可复现，本文按以下准则选择云原生权限提升 CVE 作为案例：

- **风险等级与影响范围：**CVSS 评分 ≥ 5.0 ，在 Kubernetes 生态中影响较广，如核心组件或高使用率第三方应用漏洞。
- **成因类型覆盖：**所选案例覆盖两级成因分类框架的主要类别，包括配置缺陷、安全逻辑缺陷、敏感信息泄露与代码注入等，验证方法的适用范围。
- **可复现性：**具备公开 PoC 或完整技术细节，可在隔离环境中安全复现；受影响版本与修复版本明确，便于对比验证。

表 6-1 本章所选云原生权限提升 CVE 案例
Tab. 6-1 Overview of selected CVE cases in this chapter

序号	CVE 编号	影响组件	一级分类	二级分类
1	CVE-2022-24768	Argo CD	安全逻辑缺陷	访问控制缺陷
2	CVE-2025-2241	Hive	敏感信息泄露	API 端口暴露
3	CVE-2023-30512	CubeFS CSI	配置缺陷	冗余权限
4	CVE-2020-4062	ConjurOSS	配置缺陷	网络隔离不足
5	CVE-2022-24877	Flux	代码注入	路径遍历
6	CVE-2022-29171	Sourcegraph	代码注入	命令注入
7	CVE-2023-22482	Argo CD	安全逻辑缺陷	身份验证绕过

6.1.2 实验环境与复现流程

环境搭建。所有漏洞复现与策略验证均在隔离的 Kubernetes 测试集群中进行。为便于复核与复现，实验环境的软硬件配置与安全组件版本统一汇总于表 6-2。

表 6-2 实验环境软硬件配置与工具版本汇总
Tab. 6-2 Summary of hardware and software environment configuration and tool versions

项目	配置
计算资源	本地虚拟机，96 核 CPU，96GB 内存，200GB 存储
集群工具与版本	Minikube v1.18.1；Kubernetes v1.18.3
节点规模与规格	1 个控制节点与 2 个工作节点；每节点 4 核 CPU/8GB 内存
CNI/网络策略	Calico v3.28.2, Kubernetes-Network-Policy v1.5.5
准入控制	OPA/Gatekeeper v3.14.0；Kyverno v1.11.0
运行时安全监控	Falco v0.38.1
包管理工具	Helm v3

漏洞案例分析流程。对每个权限提升 CVE，结合本文方法依据如下步骤进行：

1. **漏洞复现：**为有效进行安全策略验证，需根据 CVE 描述进行漏洞复现。部署受影响版本的目标组件，创建模拟业务负载与低权限身份；根据公开 PoC 或技术分析模拟权限提升攻击，记录攻击步骤与异常行为；验证是否成功实现权限提升。
2. **安全策略生成：**基于本文方法和 KPEDefense 系统，使用至少三个大模型生成候选安全策略，并通过静态评估从有效性 E （阻断攻击路径）、无干扰性 I （对业务的影响）、可部署性 D （配置完整性与可执行性）三个维度综合评估，筛选出一个最佳的安全策略集合。
3. **策略验证：**部署生成的安全策略，并根据漏洞复现执行攻击测试，验证策略能否对漏洞进行有效监控和防护，并根据三维质量评价体系分析结果。

6.2 案例一：CVE-2022-24768 内部权限问题

6.2.1 漏洞复现

漏洞背景

Argo CD 是 Kubernetes 生态中广泛使用的声明式 GitOps 持续交付工具。其典型部署形态是由 Argo CD 控制平面以相对高权限身份持续对齐 Git 仓库中的期望状态，即对集群资源拥有创建、更新与删除能力。该架构提升交付一致性的同时，也引入了新的安全风险：一旦普通用户能够影响 Argo CD 的控制决策，便可能借助控制平面权限放大自身权限边界。

CVE 描述：CVE-2022-24768 为 Argo CD 的不当访问控制（Improper Access Control）漏洞。所有未修复版本（自 1.0.0 起）均存在风险，0.5.x 与 0.8.x 系列中也包含该问题的受限版本。攻击者需要满足以下前置条件之一：（i）对某个 Application 的源 Git/Helm 仓库具备 push 权限；或（ii）在 Argo CD 中对该 Application 具备 sync 与 override 权限。满足条件后，攻击者可通过构造/修改仓库内容或应用配置，诱导 Argo CD 以其控制平面权限执行超出攻击者原始 RBAC 边界的操作，进而可获得管理员权限。官方已在 Argo CD 2.1.14、2.2.8、2.3.2 中发布补丁。

主要成因分类：安全逻辑缺陷 / 访问控制缺陷（已授权用户的权限放大）。
攻击链条描述：攻击链可概括为：

1. **合法身份阶段：**攻击者以普通 Argo CD 用户身份登录；
2. **配置影响阶段：**凭借对 Application 的 sync+override 或对源码仓库的 push 权限，注入恶意资源清单；
3. **控制平面滥用阶段：**Argo CD 按 GitOps 同步逻辑将恶意资源应用到集群；
4. **权限提升阶段：**借助被创建的高权限 RBAC 绑定获得集群级管理员能力。

漏洞复现

实验部署 Argo CD v2.3.1（受影响版本），配置 Kubernetes 测试集群（kind 或 minikube），以低权用户 bob 身份接入；bob 仅在特定项目范围内持有 sync 与 override 权限，对集群级资源无直接写入能力。测试仓库（https://github.com/Ivanbeethoven/bad_repo）中含正常业务资源 app.yaml 与恶意 ClusterRoleBinding hack-admin.yaml，二者均处于 Argo CD 同步路径

下。关键命令与清单见附录C。

bob 账户触发 Application 同步后,Argo CD 控制平面将恶意 ClusterRoleBinding 一并应用至集群,以 --as bob 执行 `kubectl auth can-i '*' '*'` 返回 yes,低权身份提升至集群管理员级别,权限边界放大经实验证实。

```
r2cn-ubuntu-01 → ~ kubectl describe clusterrolebinding bob-admin-binding
Name:          bob-admin-binding
Labels:        app.kubernetes.io/instance=bob-app
Annotations:   <none>
Role:
  Kind: ClusterRole
  Name: cluster-admin
Subjects:
  Kind  Name  Namespace
  ----  ---  -
  User  bob
```

图 6-1 漏洞复现实验结果示意（验证权限变化）

Fig. 6-1 Vulnerability reproduction result (verification of privilege changes)

```
# 检查 bob 对 namespaces 的权限
kubectl auth can-i create namespaces --as bob
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"rbac.authorization.k8s.io/v1","kind":"ClusterRoleBinding","metadata":{"annotations":{},"labels":{"app.kubernetes.io/instance":"bob-app"},"name":"bob-admin-binding"},"roleRef":{"apiGroup":"rbac.authorization.k8s.io","kind":"ClusterRole","name":"cluster-admin"},"subjects":[{"apiGroup":"rbac.authorization.k8s.io","kind":"User","name":"bob"}]}
  creationTimestamp: "2025-12-17T12:02:56Z"
  labels:
    app.kubernetes.io/instance: bob-app
  name: bob-admin-binding
  resourceVersion: "529144"
  uid: 45b0bb4b-605e-4b00-a4d7-9fe650c3171e
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: User
  name: bob
yes
yes
yes
yes
Warning: resource 'namespaces' is not namespace scoped
```

图 6-2 漏洞复现实验结果示意（验证是否有命名空间级别权限）

Fig. 6-2 Vulnerability reproduction result (verification of namespace-level privileges)

6.2.2 安全策略生成

将漏洞场景描述输入 KPEDefense 系统,调用 GPT5、GLM-4.5 与 DeepSeek-V3.2 三个大模型生成候选安全策略,依据有效性(E)、干扰性(I)、可部署性(D)三个维度静态评估后筛选最优策略集合。最终选定 GPT5 生成的策略,以 RBAC 权限收敛与准入控制为核心,覆盖 Kubernetes 集群侧与 Argo CD 应用侧两个层面,详细配置见附录C。

6.2.3 策略验证

将最优策略集合部署至实验集群后，以 bob 账户重执行漏洞复现步骤，对有效性、无干扰性与可部署性三个维度逐项验证。

有效性：策略部署后，触发对恶意 RBAC 绑定的同步请求被准入控制拒绝；执行 `kubectl auth can-i '*' '*' --as bob` 返回 no，权限提升路径关闭。Kubernetes RBAC 收敛后 bob 对 AppProject 的写入同样被拒绝，韵从限制了通过修改项目策略绕过准入规则的路径。

无干扰性：策略收紧范围限于高危操作（`override`、`exec` 等）及 AppProject 写入，项目内合规同步操作未受影响。准入控制规则先以审计模式运行，确认无违规清单误拦截后切换至强制模式。

可部署性：RBAC 策略与 Argo CD 配置均以 YAML 形式纳入 Git 基线，通过 `kubectl apply` 一步部署，实验集群中无冲突落地。Gatekeeper 规则可模板化复用于多集群场景。

就 CVE-2022-24768 而言，KPEDefense 筛选的最优策略组合在三维验证中通过。Kubernetes RBAC 收敛与准入控制协同后，低权用户借助 `sync+override` 诱导控制平面创建集群级高权限绑定的路径被关闭。该方案可作为版本升级前的补充缓解措施，最终修复仍需升级至官方补丁版本（2.1.14、2.2.8 或 2.3.2）。

6.3 案例二：CVE-2025-2241 敏感信息泄露

6.3.1 漏洞复现

漏洞背景

Hive 是 Red Hat Multicluster Engine（MCE）与 Advanced Cluster Management（ACM）中的集群生命周期管理组件，常用于在多集群场景下自动化创建与托管 OpenShift/Kubernetes 集群。在 vSphere 场景中，Hive 需要使用 vCenter 凭据完成集群的基础设施供应与后续管理。

CVE 描述：CVE-2025-2241 指出 Hive 在完成 vSphere 集群供应（`provisioning`）后，会将 vCenter 凭据暴露在 `ClusterProvision` 对象中。由于 `ClusterProvision` 属于自定义资源（CR），拥有其读取权限的用户可直接从该对象中提取敏感凭据，即使该用户并不具备对 `Kubernetes Secret` 的访问权限。该问题会导致未授权 vCenter 访问、基础设施与集群管理风险，并可能

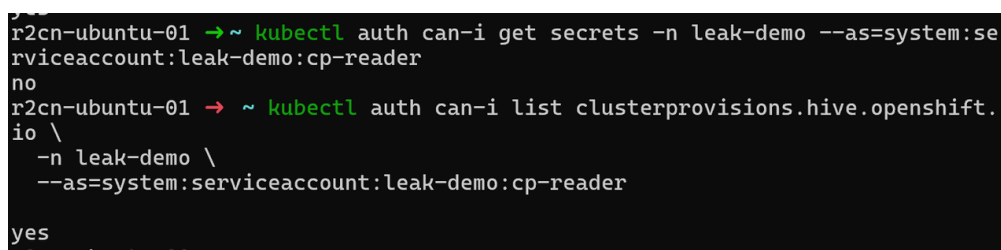
进一步演化为权限提升。

CVSS 信息: CNA (Red Hat) 给出的向量为 CVSS:3.1/AV:N/AC:H/PR:L, 基准分 8.2 (High)。其含义可理解为: 攻击者需要一定前置权限 (PR:L) 和较高攻击复杂度 (AC:H), 但一旦满足条件, 影响范围跨越安全边界 (S:C), 并造成高机密性与完整性影响 (C:H/I:H)。

主要成因分类: 敏感信息泄露 / 配置与对象生命周期缺陷, 即敏感数据出现在可读的 CR 状态字段中, 形成对 Secret 隔离边界的旁路。

攻击链条描述: 攻击链可概括为 Secret 隔离被 CR 可读性绕过:

1. **权限前置阶段:** 攻击者拥有对 ClusterProvision 的 get/list/watch 权限 (通常来自聚合角色 view/edit 或错误的绑定);
2. **数据旁路阶段:** Hive 将 vCenter 凭据写入 ClusterProvision, 位于 status 字段;
3. **凭据提取阶段:** 攻击者通过读取 CR YAML/JSON 直接获得凭据;
4. **后续风险阶段:** 利用 vCenter 凭据对虚拟化平台执行未授权操作, 进而影响集群节点与工作负载, 造成横向移动或权限提升。



```
r2cn-ubuntu-01 → ~ kubectl auth can-i get secrets -n leak-demo --as=system:serviceaccount:leak-demo:cp-reader
no
r2cn-ubuntu-01 → ~ kubectl auth can-i list clusterprovisions.hive.openshift.io \
-n leak-demo \
--as=system:serviceaccount:leak-demo:cp-reader
yes
```

图 6-3 CVE-2025-2241: vCenter 凭据通过 ClusterProvision 暴露的风险示意

Fig. 6-3 CVE-2025-2241: illustration of vCenter credential exposure through ClusterProvision

漏洞复现

实验在部署 ACM/MCE 与 Hive 的 OpenShift 测试集群中进行, 以具备 ClusterProvision 读取权限、但无 Secret 访问权限的低权身份接入。vSphere 测试专用 vCenter 账号采用最小权限配置, 详细 YAML 与命令序列见附录 D。

触发 ClusterDeployment 供应流程后, 以该低权身份读取 ClusterProvision 对象 YAML, 可在 status 字段中直接定位明文 vCenter 用户名与密码。由于同一身份无法读取任何 Secret 对象, 凭据暴露构成了对 Secret 隔离边界的旁路。

6.3.2 安全策略生成

将漏洞场景描述输入 KPEDefense 系统,调用 GPT5、GLM-4.5 与 DeepSeek-V3.2 三个大模型生成候选安全策略,依据有效性 (E)、干扰性 (I)、可部署性 (D) 三个维度静态评估后筛选最优策略集合。最终选定 GPT5 生成的策略,以 ClusterProvision 读取权限收敛与对象生命周期治理为核心,覆盖 CR 访问控制、敏感字段审计与供应完成后的对象清理,详细配置见附录D。

6.3.3 策略验证

将最优策略集合部署至实验集群后,以低权身份重执行漏洞复现步骤,对有效性、无干扰性与可部署性三个维度逐项验证。

有效性: RBAC 收敛后,低权身份对 ClusterProvision 的 get/list 请求被拒绝,无法再导出 YAML。供应完成后再次复查对象, status 字段中不再保留明文 vCenter 凭据,凭据旁路路径被阻断。

无干扰性: RBAC 收紧后,高权限访问改为按需、限时、可审计授权,正常供应流程未受影响。自动清理任务在预生产环境验证后确认不影响 Hive 对账流程,再推广至生产。审计与告警不影响业务主链路。

可部署性: RBAC 为平台原生能力,可模板化发布至多集群。自动清理任务可通过 GitOps 流水线或定时任务实现,无需人工介入。

就 CVE-2025-2241 而言, KPEDefense 筛选的最优策略组合在三维验证中通过。RBAC 收敛与供应完成后的对象清理协同后,敏感凭据可读窗口进一步缩短。该方案可作为平台防御基线的一部分持续管理,最终修复仍依赖厂商补丁,根因在于 Hive 控制器将凭据写入 CR 状态字段的设计缺陷。

6.4 案例三: CVE-2023-30512 过度权限配置

6.4.1 漏洞复现

漏洞背景

CubeFS 是面向云原生场景的分布式存储系统,常通过 CSI (Container Storage Interface) 插件与 Kubernetes 集成。在 Kubernetes 中, CSI 节点插件通常以 DaemonSet 形式在每个节点上运行 (例如 cfs-csi-node), 控制器组件以 Deployment 形式运行 (例如 cfs-csi-controller)。这类组件需要与

Kubernetes API 交互，但其权限应遵循最小权限原则。

CVE 描述: CVE-2023-30512 的核心问题是 CubeFS CSI 插件的服务账号（如 `cfs-csi-service-account`）被绑定到了包含**过度权限**的集群级角色（`ClusterRole`），从而具备对集群范围 `Secret` 的读取能力（如 `get/list`）。一旦攻击者在某节点获得对 CSI 节点插件 Pod 的控制，例如通过节点被入侵、容器逃逸或对系统命名空间特权工作负载的执行能力，便可能滥用该服务账号的权限边界，读取全局敏感信息并进一步演化为集群级权限提升。

主要成因分类：配置缺陷 / 过度授权（Over-privileged RBAC）。

攻击链条描述: 攻击链可概括为系统组件身份被过度授权, 且在被攻陷后可枚举集群秘密:

1. **身份前置阶段**: CSI 组件使用默认或被误配置的 `ServiceAccount` 运行;
2. **权限放大阶段**: `ServiceAccount` 被绑定到含 `secrets` 读取能力的 `ClusterRole`;
3. **控制获取阶段**: 攻击者控制某节点上的 CSI Pod (例如 `cfs-csi-node`) 并获得其运行时上下文;
4. **数据获取阶段**: 攻击者以该 `ServiceAccount` 身份读取集群 `Secret`, 从而获取高价值凭据 (管理员 `Token`、云凭据等), 实现从节点或 Pod 控制向集群控制的升级。

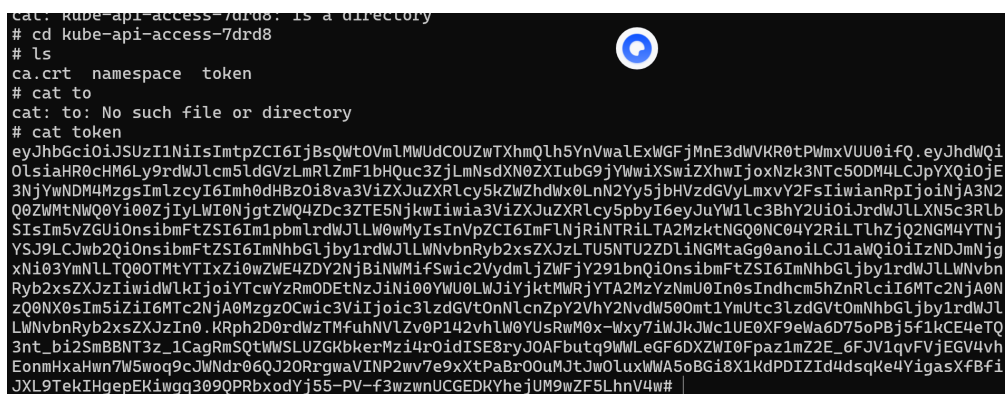


图 6-4 CVE-2023-30512: 攻击成功证据 (基于 CSI ServiceAccount 的越权访问结果)

Fig. 6-4 CVE-2023-30512: evidence of successful attack via unauthorized access through a CSI ServiceAccount

漏洞复现（验证性复现）

实验在 Kubernetes 测试集群中部署受影响版本的 CubeFS CSI，明确节点插件与控制器组件使用的 `cfs-csi-service-account`，并导出其对应的 `ClusterRole` 与 `ClusterRoleBinding`。权限证据显示，该服务账号关联的 `ClusterRole`

含有 secrets 资源的 get/list/watch 动词，完整命令与策略清单见附录E。

以 cfs-csi-service-account 身份执行 `kubectl auth can-i list secrets --all-namespaces`，结果返回 yes，表明该账号具备跨命名空间读取 Secret 的能力。攻击者一旦控制 CSI Pod 运行时上下文，即可沿该权限路径继续获取高价值凭据。

```
r2cn-ubuntu-01 → ~ curl -k -H "Authorization: Bearer $TOKEN" $API_SERVER/api/v1/
{
  "kind": "APIResourceList",
  "groupVersion": "v1",
  "resources": [
    {
      "name": "bindings",
      "singularName": "binding",
      "namespaced": true,
      "kind": "Binding",
      "verbs": [
        "create"
      ]
    },
    {
      "name": "componentstatuses",
      "singularName": "componentstatus",
      "namespaced": false,
      "kind": "ComponentStatus",
      "verbs": [
        "get",
        "list"
      ],
      "shortNames": [
        "cs"
      ]
    },
    {
      "name": "configmaps",
      "singularName": "configmap",
      "namespaced": true,
      "kind": "ConfigMap",
      "verbs": [
        "create",
        "delete",
        "deletecollection",
        "get",
        "list",
        "patch",
        "update",

```

图 6-5 CVE-2023-30512：攻击成功证据（Secrets 读取能力验证/权限提升结果截图）
Fig. 6-5 CVE-2023-30512: evidence of successful attack showing secrets-read capability and privilege-escalation results

攻击结果（摘要）：若服务账号具备集群范围 Secret 读取权限，则攻击者一旦控制 CSI Pod/节点，存在获取高价值凭据并进一步获得集群控制的风险。

6.4.2 安全策略生成

将漏洞场景描述输入 KPEDefense 系统，调用 GPT5、GLM-4.5 与 DeepSeek-V3.2 三个大模型生成候选安全策略，依据有效性（E）、干扰性（I）、可部署性（D）三个维度静态评估后筛选最优策略集合。最终选定 GPT5 生成的策略，以 RBAC 最小化与准入防回归为核心，覆盖 ServiceAccount 权限收敛、

ClusterRole 规则约束与过度授权变更阻断，详细配置见附录E。

6.4.3 策略验证

将最优策略集合部署至实验集群后，重新以 `cfs-csi-service-account` 身份执行漏洞复现中的权限验证，对有效性、无干扰性与可部署性三个维度逐项验证。

有效性: RBAC 收敛后, `kubectl auth can-i list secrets --all-namespaces` 的结果由 `yes` 变为 `no`，说明 CSI ServiceAccount 对 Secret 的跨命名空间读取能力已移除。准入规则生效后，包含 `secrets` 读取动词的 ClusterRole 变更无法再次进入集群，过度授权回归路径被阻断。

无干扰性: 策略收紧对象限于 CSI 组件的非必要权限，PVC/PV 生命周期、挂载与卸载流程保持正常。准入规则先以审计模式运行，确认无平台必需权限被误拦截后再切换至强制模式。

可部署性: RBAC 规则与准入策略均采用 YAML 形式管理，可直接纳入 Git 基线并通过 `kubectl apply` 部署。该方案依赖 Kubernetes 原生权限机制与常用准入组件，适合在多集群场景中复用。

就 CVE-2023-30512 而言，KPEDefense 筛选的最优策略组合在三维验证中通过。CSI ServiceAccount 的 Secret 读取能力被移除后，节点或 Pod 被攻陷后继续触达集群敏感凭据的路径被阻断。该方案可作为版本修复前的补充缓解措施，最终仍需结合官方修复与安装清单更新完成根因处理。

6.5 案例四：CVE-2020-4062 缺乏网络隔离策略

6.5.1 漏洞复现

漏洞背景

Conjur OSS 是云原生场景下常用的机密管理系统，属于 Secrets Management 系统。在典型部署中，Conjur Server 负责鉴权与策略校验，后端数据库 PostgreSQL 用于持久化存储策略、审计与机密相关数据。Kubernetes 默认网络模型在多数容器网络接口 CNI 配置下呈扁平互通状态，Pod 间通信默认允许。因此，对于数据库等敏感组件，需要显式配置 NetworkPolicy 等隔离策略，并建立默认拒绝策略 Default Deny，以控制最小暴露面。

CVE 描述: CVE-2020-4062 影响 Conjur OSS Helm Chart 2.0.0 之前版本。

旧版 Chart 默认未提供对 **Postgres** 的访问控制策略，例如缺失 **NetworkPolicy**，导致数据库服务在集群内对任意 Pod 可达。即使数据库未通过 **NodePort** 或 **LoadBalancer** 暴露到公网，攻击者一旦获得集群内任意工作负载的执行上下文，仍可能横向移动并直连数据库，绕过应用层安全控制，从而造成机密数据泄露、策略被篡改与审计记录被破坏等后果。

严重等级：Critical。

主要成因分类：配置缺陷 / 网络访问控制缺失 **Missing NetworkPolicy**。

攻击链条描述：该漏洞可抽象为集群内横向移动与直连数据库绕过应用层安全控制的攻击链：

1. **初始立足点：**攻击者控制集群内任意 Pod，例如通过业务漏洞或错误授权获得容器执行能力；
2. **侦察发现：**在目标命名空间中发现 **Postgres Service** 与 5432 端口；
3. **网络直连：**由于缺失 **NetworkPolicy**，攻击者可从任意 Pod 直连 **Postgres**；
4. **影响扩展：**若获取数据库凭据或数据密钥（来源可能包括过宽 **RBAC**、配置泄露或备份泄露），风险可进一步扩展为机密窃取、权限篡改与审计删除。

漏洞复现

实验在支持 **NetworkPolicy** 的 **Kubernetes** 集群中部署受影响版本的 **Conjur OSS Helm Chart**，并确认 **Postgres Service** 已在目标命名空间暴露 5432 端口。检查该命名空间中的网络策略配置后，未发现针对 **Postgres** 入站访问的限制规则。完整环境准备与策略清单见附录F。

在非 **Conjur** 组件工作负载中对 **Postgres** 5432 端口发起受控 **TCP** 连通性探测，探测结果为可达，表明数据库在集群内对普通 Pod 暴露。部署默认拒绝且仅允许 **Conjur Server** 访问 **Postgres** 的 **NetworkPolicy** 后，对同一路径重新探测，结果变为不可达，业务 Pod 直连数据库的路径被阻断。

攻击结果（摘要）：若未隔离 **Postgres**，攻击者一旦获得集群内任意 Pod 的执行上下文，即存在横向移动并直连数据库的机会，构成从业务容器到机密管理后端的跨域突破路径。

6.5.2 安全策略生成

将漏洞场景描述输入 **KPEDefense** 系统，调用 **GPT5**、**GLM-4.5** 与 **DeepSeek-V3.2** 三个大模型生成候选安全策略，依据有效性（E）、干扰性（I）、可部署

性（D）三个维度静态评估后筛选最优策略集合。最终选定 GPT5 生成的策略，以 NetworkPolicy 为核心，通过默认拒绝收敛 Postgres 入站访问面，仅保留 Conjur Server 到 5432 端口的必要通信路径，并以基线治理防止隔离策略回退。详细配置见附录F。

6.5.3 策略验证

将最优策略集合部署至实验集群后，重复漏洞复现中的连通性探测，对有效性、无干扰性与可部署性三个维度逐项验证。

有效性： NetworkPolicy 部署前，非 Conjur 工作负载可连通 Postgres 5432 端口；部署后，同一路径探测结果变为不可达，说明业务 Pod 已无法直连数据库。基线检查同时覆盖策略缺失与规则弱化场景，可降低配置回退带来的风险。

无干扰性： Conjur Server 到 Postgres 的合法通信保持正常，Conjur 组件启动与机密访问流程未受影响。策略上线前完成标签匹配核对与预生产回归验证，未出现因选择器偏差导致的误拦截。

可部署性： NetworkPolicy 为 Kubernetes 原生资源，可直接纳入 Git 基线并通过 Helm 或 kubectl apply 发布。数据库服务与端口固定，规则结构简单，适合在同类部署环境中模板化复用。

就 CVE-2020-4062 而言，KPEDefense 筛选的最优策略组合在三维验证中通过。默认拒绝与精确放通生效后，业务 Pod 直连 Postgres 的路径被阻断，数据库暴露面明显下降。该方案依赖集群 CNI 对 NetworkPolicy 的支持，不支持时仍需借助命名空间隔离等替代措施降低风险。

6.6 案例五：CVE-2022-24877 代码注入（路径穿越）

6.6.1 漏洞复现

漏洞背景

Flux 是云原生 GitOps 交付体系中的常用组件，典型部署包含 kustomize-controller 等控制器，用于在集群中持续对齐 Git 仓库的期望状态。在该工作流中，控制器会拉取仓库内容并调用 Kustomize 解析 kustomization.yaml，进而生成并应用资源清单。由于 kustomization.yaml 以文件路径为核心配置对象，控制器必须将其中指定的资源、补丁与组件路径解析为实际文件系

统访问。若路径规范化与边界检查不足，则恶意构造的 `kustomization.yaml` 可能触发路径穿越，使控制器读取超出预期工作目录的文件，从而造成敏感信息暴露，并在多租户部署中形成隔离边界破坏的风险。

CVE 描述: CVE-2022-24877 是 Flux `kustomize-controller` 的路径穿越漏洞。攻击者可通过恶意 `kustomization.yaml` 触发控制器读取其 Pod 文件系统中的非预期文件，从而暴露敏感数据；在多租户部署下，该能力可能进一步放大为权限提升风险。该漏洞已在 `kustomize-controller` v0.24.0 中修复，并随 `flux2` v0.29.0 集成发布。工程侧常见缓解手段包括在 CI/CD 流水线中自动校验 `kustomization.yaml` 是否符合安全策略。

主要成因分类: 输入校验缺陷 / 路径穿越 (Path Traversal, 路径规范化与目录边界检查不足)。

攻击链条描述: 攻击链可概括为仓库内容被信任、路径解析越界以及敏感数据外泄。攻击者将恶意 `kustomization.yaml` 提交至可被控制器同步的仓库；控制器在构建阶段解析并访问其中指定的路径；若路径可越界访问控制器容器文件系统，则非预期文件内容可能被注入到构建产物、事件输出或后续流程中，造成敏感信息暴露，并在多租户环境中破坏隔离边界。

漏洞复现

实验在部署受影响版本 `kustomize-controller` 的 Flux 测试环境中进行，将包含异常路径引用的 `kustomization.yaml` 提交至受控仓库并触发同步。正文仅保留控制器构建阶段的可观测证据，不展开可直接复用的敏感文件读取细节。完整环境准备、验证步骤与证据判据见附录G。

触发同步后，控制器在构建日志中出现与越界路径解析相关的异常信息，构建产物同时表现出非预期文件访问迹象，说明恶意 `kustomization.yaml` 已进入控制器构建流程，路径穿越风险得到验证。

6.6.2 安全策略生成

将漏洞场景描述输入 KPEDefense 系统，调用 GPT5、GLM-4.5 与 DeepSeek-V3.2 三个大模型生成候选安全策略，依据有效性 (E)、干扰性 (I)、可部署性 (D) 三个维度静态评估后筛选最优策略集合。最终选定 DeepSeek-V3.2 生成的策略，以前置校验与 GitOps 变更治理为核心，在提交阶段校验 `kustomization.yaml` 中的异常路径引用，并收紧仓库写入、同步变更与控制器运行权限。详细配置见附录G。

6.6.3 策略验证

将最优策略集合部署至实验环境后，重新执行漏洞复现中的仓库提交与同步流程，对有效性、无干扰性与可部署性三个维度逐项验证。

有效性：CI 校验生效后，包含异常路径引用的 `kustomization.yaml` 在提交阶段即被拦截，相关变更无法进入同步链路。即使在受控环境中绕过提交流程直接触发同步，控制器运行权限收紧后，异常路径访问范围仍被限制。

无干扰性：策略收紧范围限于异常路径引用与高风险变更权限，合规 `kustomization.yaml` 的构建与同步流程保持正常。校验规则先以告警模式运行，用于识别历史仓库中的不规范写法，再切换至强制模式。

可部署性：CI 校验规则与权限配置均采用声明式文件管理，可直接纳入现有流水线与 GitOps 基线。该方案主要依赖仓库准入检查和 Kubernetes 原生权限配置，适合在多租户环境中复用。

就 CVE-2022-24877 而言，KPEDefense 筛选的最优策略组合在三维验证中通过。异常路径引用被前移拦截后，恶意 `kustomization.yaml` 进入控制器构建流程的机会明显下降。该方案可作为补丁发布前的补充缓解措施，最终修复仍依赖升级至 `kustomize-controller v0.24.0` 及以上版本。

6.7 案例六：CVE-2022-29171 远程代码执行

6.7.1 漏洞复现

漏洞背景

Sourcegraph 是面向大规模代码仓库的搜索与导航平台，其典型部署包含负责仓库拉取与 Git 操作的 `gitserver` 服务。在企业集成场景中，Sourcegraph 可接入多种代码托管与协作系统。为获得额外元数据，部分集成允许管理员在系统中配置外部命令，由后端服务在特定路径下执行并返回结果。此类机制天然具有较高风险：若命令可被高权限用户任意改写，且执行环境与网络边界缺少隔离，则可能形成配置驱动的命令执行入口。

CVE 描述：CVE-2022-29171 指出 Sourcegraph `gitserver` 在 Gitolite 与 Phabricator 联动集成中存在远程代码执行风险。在受影响版本 ($< 3.38.0$) 中，站点管理员可配置用于获取 Phabricator 元数据的 `callsignCommand`；若攻击者同时具备对 Sourcegraph 实例的站点管理员权限，以及对其内置 Grafana 监控实例的管理员权限，则可进一步获取内部服务地址信息，并将上述命令改

写为任意系统命令，从而在 `gitserver` 上触发远程执行。该问题已在 3.38.0 版本修复，官方仍建议即使采取临时缓解也应尽快升级。

主要成因分类：安全逻辑缺陷 / 命令注入（配置驱动的命令执行缺乏约束）与权限边界缺陷（高敏感配置权限与观测面权限向执行面耦合）。

攻击链条描述：攻击链可概括为高权限配置、执行面触达以及基础设施控制。攻击者首先获得可编辑或新增 Gitolite code host 的管理权限；随后借助 Grafana 管理权限获取 `gitserver` 的内部寻址信息；最终改写 `callsignCommand` 并触发执行。风险的核心在于管理面配置、观测面信息与执行面能力发生了耦合。

漏洞复现

实验在隔离环境中部署受影响版本的 Sourcegraph，并保留 Gitolite 与 Phabricator 联动集成。正文仅保留配置变更后 `gitserver` 的可观测执行证据，不展开可直接复用的命令载荷与内部寻址细节。完整环境准备、验证判据与回归检查步骤见附录H。

在修改 `callsignCommand` 相关配置后，`gitserver` 的任务执行记录与审计证据中出现外部命令被触发的迹象，说明管理面配置已可影响执行面行为。禁用相关集成或切换至修复版本后，同类迹象不再出现，命令执行风险得到验证。

6.7.2 安全策略生成

将漏洞场景描述输入 KPEDefense 系统，调用 GPT5、GLM-4.5 与 DeepSeek-V3.2 三个大模型生成候选安全策略，依据有效性（E）、干扰性（I）、可部署性（D）三个维度静态评估后筛选最优策略集合。最终选定 GPT5 生成的策略，以切断命令执行入口与隔离执行面为核心，包括禁用或移除 Gitolite 集成、对 `gitserver` 实施网络隔离，以及收紧外部服务配置与 Grafana 管理权限。详细配置见附录H。

6.7.3 策略验证

将最优策略集合部署至实验环境后，重新执行漏洞复现中的配置变更与观测流程，对有效性、无干扰性与可部署性三个维度逐项验证。

有效性：禁用 Gitolite 集成后，`callsignCommand` 对应的输入面被移除，相关外部命令触发迹象不再出现。网络隔离与权限分离同时生效后，即使管理面

存在误授权限，配置变化也难以继续进入 `gitserver` 执行面。

无干扰性：策略收紧范围集中在高风险集成与管理权限，未依赖 Gitolite 的仓库搜索与浏览流程保持正常。网络隔离规则上线前经预生产环境核对入站与出站依赖，未出现合法通信误拦截。

可部署性：集成功能开关、NetworkPolicy 与权限配置均可声明式管理，并纳入 GitOps 基线持续维护。该方案依赖 Kubernetes 常用网络隔离能力与平台权限治理机制，适合在同类部署环境中复用。

就 CVE-2022-29171 而言，KPEDefense 筛选的最优策略组合在三维验证中通过。命令执行入口被切断后，管理面配置继续影响 `gitserver` 执行面的路径被阻断。该方案可作为版本升级前的补充缓解措施，最终仍建议升级至修复版本。

6.8 案例七：CVE-2023-22482 内部权限失控

6.8.1 漏洞复现

漏洞背景

Argo CD 是 Kubernetes 场景中常用的声明式 GitOps 持续交付工具，提供 Web 与 API 接口供用户进行应用同步、回滚与差异审计等操作。在企业环境中，Argo CD 常通过 OIDC 与统一身份提供方对接，由身份提供方向 Argo CD 签发包含用户身份与组信息的令牌。在 OIDC 令牌中，`aud`（audience）声明用于标识该令牌的预期接收方。通常情况下，服务端除验证签名与发行者外，还需要校验 `aud` 是否包含本服务的受众标识，以避免将为其他服务签发的令牌误用到本服务上。

CVE 描述：CVE-2023-22482 指出 Argo CD 在部分版本中存在不恰当授权缺陷。其能够验证令牌由已配置的 OIDC 提供方签名，但未对令牌的 `aud` 声明进行校验，从而可能接受并非面向 Argo CD 的有效令牌。若同一身份提供方还与其他系统签发令牌，攻击者一旦获得其他系统的有效令牌，就可能将其用于访问 Argo CD。Argo CD 将基于令牌中的 `groups` 声明授予权限，即使这些组信息并非为 Argo CD 的授权场景设计。该问题在 2.6.0-rc3、2.5.6、2.4.19 与 2.3.13 及以上版本修复；受影响范围包含 $\geq 1.8.2$ 且低于上述修复版本的版本分支。

主要成因分类：安全逻辑缺陷 / 身份与授权校验缺失（令牌受众 `aud` 未

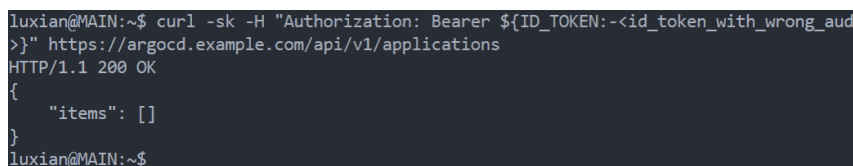
校验导致跨受众令牌复用)。

攻击链条描述：攻击链可概括为多受众身份提供、跨受众令牌复用以及基于组声明的越权访问。攻击者获得同一 OIDC 提供方为其他受众签发的有效令牌；将其提交给 Argo CD 的接口；若 Argo CD 未校验 aud，则会接受该令牌并根据 groups 映射授予权限，进而造成对应用资源与交付流程的非预期访问。该缺陷同时放大了令牌泄露事件的影响范围。

漏洞复现

实验在接入 OIDC 身份提供方的 Argo CD 测试环境中进行，重点验证 aud 不匹配令牌是否仍会被受影响版本接受。正文仅保留配置证据、令牌声明与接口返回结果，不展开可直接复用的令牌获取与请求构造细节。完整环境准备、验证判据与回归检查步骤见附录I。

在受影响版本中，将签名有效但 aud 不匹配的令牌提交至 Argo CD 接口后，系统仍返回受 RBAC 控制的资源内容，说明该令牌已被错误接受。切换至修复版本或在前置入口增加受众校验后，同类请求在校验阶段被拒绝，跨受众令牌复用风险得到验证。



```
luxian@MAIN:~$ curl -sk -H "Authorization: Bearer ${ID_TOKEN:-<id_token_with_wrong_aud>}" https://argocd.example.com/api/v1/applications
HTTP/1.1 200 OK
{
  "items": []
}
luxian@MAIN:~$
```

图 6-6 验证性复现证据示意（审计/接口响应截图的经处理摘要）

Fig. 6-6 Validation-oriented reproduction evidence (processed summary of audit and API response screenshots)

6.8.2 安全策略生成

将漏洞场景描述输入 KPEDefense 系统，调用 GPT5、GLM-4.5 与 DeepSeek-V3.2 三个大模型生成候选安全策略，依据有效性（E）、干扰性（I）、可部署性（D）三个维度静态评估后筛选最优策略集合。最终选定 GPT5 生成的策略，以身份提供方客户端隔离、前置入口 aud 校验与 Argo CD RBAC 收敛为核心，既限制跨系统令牌复用，也压缩误用令牌后的权限上限。详细配置见附录I。

6.8.3 策略验证

将最优策略集合部署至测试环境后，重新执行漏洞复现中的令牌访问流程，对有效性、无干扰性与可部署性三个维度逐项验证。

有效性：前置入口启用 `aud` 强制校验后，签名有效但受众不匹配的令牌在进入 Argo CD 前即被拒绝，受影响版本中出现的越权访问结果不再复现。RBAC 与项目边界收敛同时生效后，即使异常令牌进入授权链路，可触达资源范围也被明显压缩。

无干扰性：策略收紧范围集中在受众校验与高权限组映射收敛，合法令牌的 CLI、Web 与自动化访问流程保持正常。身份提供方客户端隔离上线前完成组映射核对与回滚预案准备，未出现既有授权关系异常中断。

可部署性：身份提供方配置、前置网关校验规则与 RBAC 策略均可声明式管理，并纳入 GitOps 基线持续维护。该方案依赖常见身份网关与 Kubernetes 权限治理能力，适合在多集群环境中复用。

就 CVE-2023-22482 而言，KPEDefense 筛选的最优策略组合在三维验证中通过。跨受众令牌在应用入口前被拦截后，令牌误用扩展至 Argo CD 授权面的路径被阻断。该方案可作为补丁窗口期的补充缓解措施，最终修复仍需升级至官方修复版本。

表 6-3 从有效性、无干扰性与可部署性三个维度汇总了 7 个案例的验证结论。

6.9 小结

7 个覆盖不同成因类型的典型 CVE 均在隔离集群中完成了漏洞复现与策略部署验证。如表 6-3 所示，生成策略在有效性维度上均能阻断或显著缓解攻击路径；在无干扰性维度上，多数策略对业务影响可控，但仍需在预生产环境验证；在可部署性维度上，策略主要依赖 Kubernetes 原生能力与成熟准入控制组件，具备模板化推广的条件。

表 6-3 案例验证三维质量汇总
Tab. 6-3 Summary of three-dimensional quality verification across cases

序号	CVE 编号	有效性 <i>E</i>	无干扰性 <i>I</i>	可部署性 <i>D</i>
1	CVE-2022-24768	多环节阻断攻击链，复测确认提权失败	保留正常同步能力，建议先审计后强制	RBAC/准入控制均为原生能力
2	CVE-2025-2241	RBAC 收敛 + 审计报告 + 对象清理降低凭据暴露	需以按需授权替代全局可读	原生能力，准入建议 dryrun 起步
3	CVE-2023-30512	直接移除 secrets 读取能力，复测 can-i 为 no	需验证 CSI 的 PVC/PV 生命周期	RBAC/准入可模板化推广
4	CVE-2020-4062	默认拒绝策略阻断横向移动，复测端口不可达	需验证组件间通信与标签一致性	NetworkPolicy 原生资源，维护成本低
5	CVE-2022-24877	CI 校验阻断异常路径引用进入仓库	历史不规范写法可能触发误报	CI 准入检查可复用，GitOps 交付
6	CVE-2022-29171	禁用 Gitolite 消除命令执行入口	影响依赖该集成的仓库同步功能	标准化配置管理，可模板化推广
7	CVE-2023-22482	前置网关 aud 校验阻断跨受众令牌复用	需评估对 CLI/Web/流水线的影	身份侧配置可跨集群复用

7 结论与展望

7.1 总结

Kubernetes 已成为容器化集群系统的重要技术设施被生产企业广泛应用。Kubernetes 集群存在高危的第三方开源应用存在权限提升漏洞 CVE，攻击者可以借此进行权限提升攻击来获取危险权限，甚至控制整个集群，而相关漏洞从披露到生产环境中修复补丁就绪之间存在被攻击窗口期。现有 Kubernetes 安全技术针对权限提升高危漏洞的防控方面存在缺陷：一方面关注静态配置风险检测问题，缺乏针对 CVE 的有效性；另一方面动态安全监控和防护工具依赖领域特定语言，缺乏普适性。为补充现有的方法缺陷，本文提出了一种基于大模型的智能化漏洞防控方法 KPEDefense。该技术基于大模型的多智能体工作流，面向权限提升相关 CVE 自动生成可执行的监控策略与防护策略。同时开发了支持工具，在 55 个真实漏洞样本进行实验研究验证了其可行性。本文主要成果如下。

(1) **提了一种基于安全策略生成的智能化漏洞防控方法 KPEDefense** 该方法首先构建面向权限提升漏洞分类框架和相关开源项目安全知识库，基于大模型的多智能体模型进行安全策略的代码生成，并通过安全上下文提示漏洞语义、通过思维链推理和反馈优化进行安全策略代码质量增强。为筛选最优安全策略并评估智能体方法论，提出基于多源模型排序的三维质量量化评估方法。

(2) **实现了 KPEDefense 支持工具**。该工具集成了 CVE 知识库管理、策略生成工作流、评审反馈交互、专家排序与策略部署等功能模块，能够为安全工程师与专家评审员提供从漏洞分析到策略部署的可操作工具支持。

(3) **通过实验研究和案例分析验证了 KPEDefense 在真实漏洞场景中的有效性**。本文在 55 个真实云原生权限提升漏洞开展了实验研究。对比实验结果表明，KPEDefense 方法相较现有静态风险扫描方法能针对具体漏洞给出更有效的防控手段；消融实验结果表明多智能体工作流中安全上下文构建 (RAG)、思维链推理 (COT) 与反馈优化 (Feedback) 等步骤均对策略质量有正向贡献。此外，基于多源模型排序的多维质量量化评估方法能够有效评估策略质量，解决安全策略质量评估中的测试语言问题。进一步地，本文选取访问控制缺陷、冗余权限、命令注入、身份验证绕过等多种成因类型下的

7 个典型 CVE，在隔离环境中完成漏洞复现与策略部署，从三个评价维度验证了策略在真实攻击场景下的实际效果。

7.2 展望

尽管本文已完成方法设计、系统实现与实验验证，但仍有进一步完善和扩展的空间。后续工作可从以下三个方向继续推进：

（1）**高危漏洞的自动化监控与知识库实时更新**。当前知识库采用离线批量构建方式，无法及时响应新披露漏洞。未来可对 GitHub Security Advisory、NVD 等公开数据源建立持续监控与订阅机制，实现增量知识库更新与策略重评估的自动化触发，缩短从漏洞披露到策略交付的响应时间。

（2）**漏洞利用的自动化复现与验证**。当前策略验证主要依赖人工模拟或静态推理，缺乏真实利用环境下的自动化验证能力。未来可基于漏洞披露信息自动搭建包含漏洞的 Kubernetes 测试集群，结合 PoC 代码执行攻击重放，量化评估策略的阻断成功率与误报率，为策略质量评价提供更客观的证据支撑。

（3）**研究范围向其他漏洞类型扩展**。本文聚焦于权限提升漏洞，未来可将方法体系扩展至拒绝服务漏洞、供应链与镜像安全漏洞等云原生环境中的其他高危类型，通过扩展分类框架与优化评价维度，构建更全面的智能化安全防控能力。

若上述方向能够持续推进，云原生安全防控有望进一步由被动应对走向主动防御，并由单点防护走向更稳定的体系化防御。

参考文献

- [1] BOUGUETTAYA A, SINGH M, HUHNS M, et al. A service computing manifesto: the next 10 years[J/OL]. Commun. ACM, 2017, 60(4): 64 – 72. <https://doi.org/10.1145/2983528>.
- [2] ZHONG C, LI S, HUANG H, et al. Domain-driven design for microservices: An evidence-based investigation[J/OL]. IEEE Transactions on Software Engineering, 2024, 50(6): 1425-1449. DOI: 10.1109/TSE.2024.3385835.
- [3] Cloud Native Computing Foundation. Cncf cloud native definition v1.1 [EB/OL]. 2024. <https://github.com/cncf/toc/blob/main/DEFINITION.md>.
- [4] Cloud Native Computing Foundation. Who we are[EB/OL]. 2026. <https://www.cncf.io/about/who-we-are/>.
- [5] 吴逸文, 张洋, 王涛, 等. 从 Docker 容器看容器技术的发展: 一种系统文献综述的视角[J]. 软件学报, 2023, 34(12): 5527-5551.
- [6] VERMA A, PEDROSA L, KORUPOLU M, et al. Large-scale cluster management at google with borg[C/OL]//EuroSys ' 15: Proceedings of the Tenth European Conference on Computer Systems. ACM, 2015: 1 – 17. <http://dx.doi.org/10.1145/2741948.2741964>.
- [7] LAWSON A, SICA J. CNCF annual cloud native survey: The infrastructure of AI's future[R/OL]. The Linux Foundation, 2026. <https://www.cncf.io/reports/the-cncf-annual-cloud-native-survey/>.
- [8] FERRAILOLO D, CUGINI J, KUHN D R, et al. Role-based access control (rbac): Features and motivations[C]//Proceedings of the 11th Annual Computer Security Applications Conference. 1995: 241-48.
- [9] GU Y, TAN X, ZHANG Y, et al. EPScan: Automated detection of excessive RBAC permissions in Kubernetes applications[C/OL]//Proceedings of the 2025 IEEE Symposium on Security and Privacy (SP). 2025: 3199-3217. DOI: 10.1109/SP61157.2025.00011.
- [10] SUN C A, HAN X, GONG Y. KubeGuard: A systematic permission-oriented risk detection approach for Kubernetes applications[C/OL]//Proceedings of the 2025 IEEE International Conference on Web Services (ICWS). 2025: 855-

865. DOI: 10.1109/ICWS67624.2025.00111.
- [11] YANG N, SHEN W, LI J, et al. Take over the whole cluster: Attacking Kubernetes via excessive permissions of third-party applications[C/OL]// Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. New York, NY, USA: Association for Computing Machinery, 2023: 3048–3062. <https://doi.org/10.1145/3576915.3623121>.
- [12] SHAMIM S I, HU H, RAHMAN A. On prescription or off prescription? an empirical study of community-prescribed security configurations for Kubernetes[C/OL]//Proceedings of the 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE). 2025: 2432-2444. DOI: 10.1109/ICSE55347.2025.00170.
- [13] RAHMAN A, SHAMIM S I, BOSE D B, et al. Security misconfigurations in open source Kubernetes manifests: An empirical study[J/OL]. ACM Trans. Softw. Eng. Methodol., 2023, 32(4). <https://doi.org/10.1145/3579639>.
- [14] Falco (CNCF). Falco: Cloud native runtime security[EB/OL]. 2025. <https://falco.org/>.
- [15] Open Policy Agent (CNCF). Open Policy Agent: Policy-based control for cloud native environments[EB/OL]. 2025. <https://www.openpolicyagent.org/>.
- [16] Kyverno (CNCF). Kyverno: Kubernetes native policy management[EB/OL]. 2025. <https://kyverno.io/>.
- [17] MINNA F, BLAISE A, REBECCHI F, et al. Understanding the security implications of Kubernetes networking[J/OL]. IEEE Security & Privacy, 2021, 19(5): 46-56. DOI: 10.1109/MSEC.2021.3094726.
- [18] BUDIGIRI G, BAUMANN C, MÜHLBERG J T, et al. Network policies in Kubernetes: Performance evaluation and security analysis[C/OL]//Proceedings of the 2021 Joint European Conference on Networks and Communications & 6G Summit (EuCNC/6G Summit). 2021: 407-412. DOI: 10.1109/EuCNC/6GSummit51104.2021.9482526.
- [19] WRIGHT C P, DAVE J, GUPTA P, et al. Versatility and unix semantics in namespace unification[J/OL]. ACM Trans. Storage, 2006, 2(1): 74 – 105. <https://doi.org/10.1145/1138041.1138045>.
- [20] Kubernetes. Kubernetes documentation[EB/OL]. 2024. <https://kubernetes.io/>

docs/home/.

- [21] MEAD N R. Computer security: Art and science[J]. IEEE Security & Privacy, 2003, 1(3): 14-14.
- [22] Aqua Security. Trivy: A simple and comprehensive vulnerability scanner for containers[EB/OL]. 2025. <https://github.com/aquasecurity/trivy>.
- [23] Anchore. Gype: A vulnerability scanner for container images and filesystems [EB/OL]. 2025. <https://github.com/anchore/gype>.
- [24] Anchore. Syft: A cli tool and library for generating sboms from container images[EB/OL]. 2025. <https://github.com/anchore/syft>.
- [25] Kubescape. Kubescape: Kubernetes security scanning and hardening tool [EB/OL]. 2025. <https://github.com/kubescape/kubescape>.
- [26] kube-linter. kube-linter: Static analysis for kubernetes yaml and helm charts [EB/OL]. 2025. <https://github.com/stackrox/kube-linter>.
- [27] kube-score. kube-score: Kubernetes object analysis tool for best practices [EB/OL]. 2025. <https://github.com/zegl/kube-score>.
- [28] KubeSec. Kubesec: Security risk analysis for kubernetes resources[EB/OL]. 2025. <https://kubesec.io/>.
- [29] Fairwinds. Polaris: Kubernetes best-practices and configuration checks [EB/OL]. 2025. <https://github.com/FairwindsOps/polaris>.
- [30] Bridgecrew (Checkov). Checkov: Infrastructure as code (iac) static analysis tool[EB/OL]. 2025. <https://github.com/bridgecrewio/checkov>.
- [31] CYBERARK. KubiScan: Kubernetes risk assessment tool[EB/OL]. 2024. <https://github.com/cyberark/KubiScan>.
- [32] APPVIA. Krane: Kubernetes rbac static analysis & visualisation tool[EB/OL]. 2024. <https://github.com/appvia/krane>.
- [33] NETWORKS P A. Kubernetes privilege escalation: Excessive permissions in popular platforms[EB/OL]. 2022[2024-03-20]. <https://www.paloaltonetworks.com/resources/whitepapers/kubernetes-privilege-escalation-excessive-permissions-in-popular-platforms>.
- [34] HØILAND-JØRGENSEN T, BROUER J D, BORKMANN D, et al. The express data path: Fast programmable packet processing in the operating system kernel[C]//Proceedings of the 14th international conference on emerging net-

- working experiments and technologies. 2018: 54-66.
- [35] SALTZER J, SCHROEDER M. The protection of information in computer systems[J/OL]. Proceedings of the IEEE, 1975, 63(9): 1278-1308. DOI: 10.1109/PROC.1975.9939.
- [36] SANDHU R, BHAMIDIPATI V, MUNAWER Q. The arbac97 model for role-based administration of roles[C/OL]//Proceedings of the 14th Annual Computer Security Applications Conference. ACM, 1998: 332-339. DOI: 10.1145/300830.300839.
- [37] FERRAILOLO D F, BARKLEY J, KUHN D R. A role-based access control model and reference implementation within a corporate intranet[C/OL]//Proceedings of the 2nd ACM Workshop on Role-Based Access Control. ACM, 1997: 2-9. DOI: 10.1145/300830.300834.
- [38] HU V C, FERRAILOLO D, KUHN R, et al. Nist special publication 800-162: [M/OL]. National Institute of Standards and Technology, 2014. DOI: 10.6028/NIST.SP.800-162.
- [39] PROVOS N, FRIEDL M, HONEYMAN P. Preventing privilege escalation [C]//Proceedings of the 12th USENIX Security Symposium. 2003.
- [40] WATSON R N M, WOODRUFF J, NEUMANN P G, et al. Cheri: A hybrid capability-system architecture for scalable software compartmentalization[J/OL]. IEEE Symposium on Security and Privacy, 2015: 20-37. DOI: 10.1109/SP.2015.9.
- [41] O'SHEA G. Redundant access rights[J]. Computers & Security, 1995, 14(4): 323-348.
- [42] MOLLOY I, PARK Y, CHARI S. Generative models for access control policies: Applications to role mining over logs with attribution[C/OL]//Proceedings of the 17th ACM Symposium on Access Control Models and Technologies. New York, NY, USA: Association for Computing Machinery, 2012: 45-56. <https://doi.org/10.1145/2295136.2295145>.
- [43] SANDERS M W, YUE C. Minimizing privilege assignment errors in cloud services[C/OL]//Proceedings of the Eighth ACM Conference on Data and Application Security and Privacy. New York, NY, USA: Association for Computing Machinery, 2018: 2-12. <https://doi.org/10.1145/3176258.3176307>.

- [44] AU K W Y, ZHOU Y F, HUANG Z, et al. Pscout: analyzing the android permission specification[C/OL]//Proceedings of the 2012 ACM Conference on Computer and Communications Security. New York, NY, USA: Association for Computing Machinery, 2012: 217-228. <https://doi.org/10.1145/2382196.2382222>.
- [45] BAGHERI H, SADEGHI A, GARCIA J, et al. DelDroid: An automated approach for determination and enforcement of least-privilege architecture in Android[J]. Journal of Systems and Software, 2019.
- [46] WU D, GAO D, LI Y, et al. SecComp: Towards practically defending against component hijacking in Android applications[C]//Proceedings of the 32nd Annual Conference on Computer Security Applications (ACSAC). 2016.
- [47] BUGIEL S, DAVI L, DMITRIENKO A, et al. Towards taming privilege-escalation attacks on android[C/OL]//Proceedings of the 2012 Network and Distributed System Security Symposium. 2012. <https://api.semanticscholar.org/CorpusID:14960107>.
- [48] SHARMEEN S, HUDA S, ABAWAJY J H, et al. An adaptive framework against Android privilege escalation threats using deep learning and semi-supervised approaches[J]. Applied Soft Computing, 2020.
- [49] TRIPP O, PISTOIA M, FINK S J, et al. Taj: effective taint analysis of web applications[C/OL]//Proceedings of the 30th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2009). New York, NY, USA, 2009: 87-97. DOI: 10.1145/1542476.1542486.
- [50] ENCK W, GILBERT P, HAN S, et al. TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones[J/OL]. ACM Transactions on Computer Systems, 2014, 32(2): 1-29. DOI: 10.1145/2619091.
- [51] YOU W, LIANG B, SHI W, et al. TaintMan: An ART-compatible dynamic taint analysis framework on unmodified and non-rooted Android devices[J/OL]. IEEE Transactions on Dependable and Secure Computing, 2020, 17(1): 209-222. DOI: 10.1109/TDSC.2017.2740169.
- [52] ISLAM SHAMIM M S, AHAMED BHUIYAN F, RAHMAN A. XI commandments of Kubernetes security: A systematization of knowledge related

- to Kubernetes security practices[C/OL]//Proceedings of the 2020 IEEE Secure Development (SecDev). 2020: 58-64. DOI: 10.1109/SecDev45635.2020.21525.
- [53] SHAMIM S I. Mitigating security attacks in Kubernetes manifests for security best practices violation[C/OL]//Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. New York, NY, USA: Association for Computing Machinery, 2021: 1689–1690. <https://doi.org/10.1145/3468264.3473495>.
- [54] CURTIS J A, EISTY N U. The Kubernetes security landscape: AI-driven insights from developer discussions[C/OL]//Proceedings of the 2025 IEEE/ACIS 23rd International Conference on Software Engineering Research, Management and Applications (SERA). 2025: 179-186. DOI: 10.1109/SERA65747.2025.11154503.
- [55] SHAMIM S I, HU H, RAHMAN A. Dynamic application security testing for Kubernetes deployment: An experience report from industry[C/OL]//Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering. New York, NY, USA: Association for Computing Machinery, 2025: 514–519. <https://doi.org/10.1145/3696630.3728573>.
- [56] ZEROUALI A, OPDEBEECK R, DE ROOVER C. Helm charts for Kubernetes applications: Evolution, outdatedness and security risks[C/OL]//Proceedings of the 2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR). 2023: 523-533. DOI: 10.1109/MSR59073.2023.00078.
- [57] BLAISE A, REBECCHI F. Stay at the Helm: Secure Kubernetes deployments via graph generation and attack reconstruction[C/OL]//Proceedings of the 2022 IEEE 15th International Conference on Cloud Computing (CLOUD). 2022: 59-69. DOI: 10.1109/CLOUD55607.2022.00022.
- [58] UL HAQUE M, KHOLOOSI M M, BABAR M A. KGSecConfig: A knowledge graph based approach for secured container orchestrator configuration [C/OL]//Proceedings of the 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER). 2022: 420-431. DOI:

- 10.1109/SANER53432.2022.00057.
- [59] DELL'IMMAGINE G, SOLDANI J, BROGI A. Kubehound: Detecting microservices' security smells in kubernetes deployments[J/OL]. Future Internet, 2023, 15(7). <https://www.mdpi.com/1999-5903/15/7/228>. DOI: 10.3390/fi15070228.
 - [60] PECKA N, BEN OTHMANE L, VALANI A. Privilege escalation attack scenarios on the DevOps pipeline within a Kubernetes environment[C/OL]// Proceedings of the International Conference on Software and System Processes and International Conference on Global Software Engineering. New York, NY, USA: Association for Computing Machinery, 2022: 45 - 49. <https://doi.org/10.1145/3529320.3529325>.
 - [61] KIM B, KIM J, LEE S. Exploring security enhancements in Kubernetes CNI: A deep dive into network policies[J/OL]. IEEE Access, 2025, 13: 35322-35338. DOI: 10.1109/ACCESS.2025.3543841.
 - [62] ZHU H, GEHRMANN C. Kub-sec, an automatic Kubernetes cluster AppArmor profile generation engine[C/OL]//Proceedings of the 2022 14th International Conference on COMMunication Systems & NETWORKS (COMSNETS). 2022: 129-137. DOI: 10.1109/COMSNETS53615.2022.9668504.
 - [63] LE M V, AHMED S, WILLIAMS D, et al. Securing container-based clouds with syscall-aware scheduling[C/OL]//Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security. New York, NY, USA: Association for Computing Machinery, 2023: 812 - 826. <https://doi.org/10.1145/3579856.3582835>.
 - [64] CHANG Y, WANG X, WANG J, et al. A survey on evaluation of large language models[J/OL]. ACM Transactions on Intelligent Systems and Technology, 2024, 15(3): 39:1-39:45. DOI: 10.1145/3641289.
 - [65] YIN X, NI C, WANG S. Multitask-based evaluation of open-source llm on software vulnerability[J/OL]. IEEE Transactions on Software Engineering, 2024, 50(11): 3071-3087. DOI: 10.1109/TSE.2024.3470333.
 - [66] FU Y, YUAN X, WANG D. Ras-eval: A comprehensive benchmark for security evaluation of llm agents in real-world environments[A/OL]. 2025. arXiv: 2506.15253. <https://arxiv.org/abs/2506.15253>.

- [67] ZHANG C, COTE M A, ALBADA M, et al. Defenderbench: A toolkit for evaluating language agents in cybersecurity environments[A/OL]. 2025. arXiv: 2506.00739. <https://arxiv.org/abs/2506.00739>.
- [68] GHOSH R, VON STOCKHAUSEN H M, SCHMITT M, et al. CVE-LLM: Ontology-assisted automatic vulnerability evaluation using large language models[C]//Proceedings of the AAAI Conference on Artificial Intelligence: Vol. 39. 2025: 28757-28765.
- [69] MALUL E, MEIDAN Y, MIMRAN D, et al. GenKubeSec: LLM-based Kubernetes misconfiguration detection, localization, reasoning, and remediation [A/OL]. 2024. arXiv: 2405.19954. <https://arxiv.org/abs/2405.19954>.
- [70] MINNA F, MASSACCI F, TUMA K. Analyzing and mitigating (with LLMs) the security misconfigurations of Helm charts from Artifact Hub[J]. Empirical Software Engineering, 2025, 30(5): 132.
- [71] KONSTANTARAS I, CHATZOGLU E, KAMPOURAKIS K E, et al. Evaluating LLMs for the automated generation of operational detection rules in enterprise EDR environments[Z]. 2026.
- [72] AL-SABBAGH A, ABDO E, SAMROUTH K, et al. AutoSecAgent: A semi-automated AI-driven penetration testing framework through recursive memory and real-time RAG[J/OL]. The Journal of Supercomputing, 2026, 82(5). DOI: 10.1007/s11227-026-08439-z.
- [73] YANG R, CHENG M, DENG G, et al. AutoEG: Exploiting known third-party vulnerabilities in black-box web applications[A/OL]. 2026. arXiv: 2604.00704. <https://arxiv.org/abs/2604.00704>.
- [74] BARR E T, HARMAN M, MCMINN P, et al. The oracle problem in software testing: A survey[J/OL]. IEEE Trans. Softw. Eng., 2015, 41(5): 507 – 525. <https://doi.org/10.1109/TSE.2014.2372785>.
- [75] REIMERS N, GUREVYCH I. Sentence-BERT: Sentence embeddings using siamese BERT-networks[C]//Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). 2019: 3982-3992.
- [76] SONG K, TAN X, QIN T, et al. MPNet: Masked and permuted pre-training

- for language understanding[J]. *Advances in Neural Information Processing Systems*, 2020, 33: 16857-16867.
- [77] DOUZE M, et al. The faiss library[A/OL]. 2024. arXiv: 2401.08281. <https://arxiv.org/abs/2401.08281>.
- [78] WU X, SARAF P P, LEE G, et al. Unveiling scoring processes: Dissecting the differences between LLMs and human graders in automatic scoring[J]. *Technology, Knowledge and Learning*, 2025: 1-16.
- [79] WANG P, LI L, et al. Large language models are not fair evaluators[A/OL]. 2023. arXiv: 2305.17926. <https://arxiv.org/abs/2305.17926>.
- [80] HWANG Y, LEE D, KANG T, et al. Can you trick the grader? adversarial persuasion of LLM judges[A/OL]. 2025. arXiv: 2508.07805. <https://arxiv.org/abs/2508.07805>.
- [81] VELEZ J, et al. Evaluating large language models as raters in large-scale writing assessments[J]. *Learning and Instruction*, 2025.
- [82] LIU Y, et al. G-Eval: NLG evaluation using GPT-4 with better human alignment[A/OL]. 2023. arXiv: 2303.16634. <https://arxiv.org/abs/2303.16634>.
- [83] WANG Y, SONG Y, ZHU T, et al. TRUSTJUDGE: Inconsistencies of LLM-as-a-judge and how to alleviate them[A/OL]. 2025. arXiv: 2509.21117. <https://arxiv.org/abs/2509.21117>.
- [84] KENDALL M G, SMITH B B. The problem of m rankings[J]. *The annals of mathematical statistics*, 1939, 10(3): 275-287.
- [85] SPEARMAN C. The proof and measurement of association between two things.[M]. *Appleton-Century-Crofts*, 1961.

附录 A 云原生权限提升漏洞数据集统计

表 A-1 云原生权限提升漏洞数据集

Tab. A-1 Dataset of cloud-native privilege escalation vulnerabilities

CVE 编号	所属平台/组件	主要成因
CVE-2018-0268	Cisco DNA Center	网络隔离
CVE-2018-1000400	Kubernetes / CRI-O	内部权限失控
CVE-2018-5256	CoreOS Tectonic	内部权限失控
CVE-2019-10165	OpenShift	敏感信息泄露
CVE-2019-11247	Kubernetes	过度权限配置
CVE-2019-3869	Ansible Tower	内部权限失控
CVE-2020-15257	containerd	容器运行时漏洞
CVE-2020-1742	nmstate/kubernetes-nmstate-handler	代码注入
CVE-2020-4062	Conjur OSS	网络隔离
CVE-2020-8559	Kubernetes	敏感信息泄露
CVE-2021-41254	kustomize-controller (Flux2)	代码注入
CVE-2021-43858	MinIO	内部权限失控
CVE-2022-1902	Red Hat Advanced Cluster Security	敏感信息泄露
CVE-2022-21701	Istio	内部权限失控
CVE-2022-23652	capsule-proxy	内部权限失控
CVE-2022-24730	Argo CD	敏感信息泄露
CVE-2022-24768	Argo CD	内部权限失控
CVE-2022-24817	Flux2 / helm-controller	代码注入
CVE-2022-24877	Flux / kustomize-controller	代码注入
CVE-2022-29165	Argo CD	内部权限失控
CVE-2022-29171	Sourcegraph	代码注入
CVE-2022-29179	Cilium	过度权限配置
CVE-2022-31102	Argo CD	代码注入
CVE-2022-37968	Azure Arc-enabled Kubernetes	内部权限失控
CVE-2022-46167	Capsule	过度权限配置
CVE-2023-22482	Argo CD	内部权限失控
CVE-2023-22645	SUSE kubewarden	过度权限配置
CVE-2023-22651	SUSE Rancher	内部权限失控
CVE-2023-23947	Argo CD	代码注入

(续表)

CVE 编号	所属平台/组件	主要成因
CVE-2023-2250	Open Cluster Management	过度权限配置
CVE-2023-26484	KubeVirt	过度权限配置
CVE-2023-29018	OpenFeature Operator	过度权限配置
CVE-2023-30512	CubeFS	过度权限配置
CVE-2023-30617	Kruise	过度权限配置
CVE-2023-30622	Clusternet	内部权限失控
CVE-2023-30840	Fluid	过度权限配置
CVE-2023-3676	Kubernetes (Windows)	内部权限失控
CVE-2023-3893	Kubernetes / kubernetes-csi-proxy	内部权限失控
CVE-2023-3955	Kubernetes (Windows)	内部权限失控
CVE-2023-46254	capsule-proxy	内部权限失控
CVE-2023-48312	capsule-proxy	内部权限失控
CVE-2023-50726	Argo CD	内部权限失控
CVE-2023-5408	OpenShift	内部权限失控
CVE-2023-5528	Kubernetes (Windows)	内部权限失控
CVE-2024-31989	Argo CD	网络隔离
CVE-2024-33522	Calico	代码注入
CVE-2024-40628	JumpServer	内部权限失控
CVE-2024-40629	JumpServer	内部权限失控
CVE-2024-43403	Kanister	过度权限配置
CVE-2024-43803	Bare Metal Operator	过度权限配置
CVE-2024-48921	Kyverno	内部权限失控
CVE-2024-56513	Karmada	过度权限配置
CVE-2025-2241	Hive (ACM/MCE)	敏感信息泄露
CVE-2025-24784	kubewarden-controller	内部权限失控
CVE-2025-32445	Argo Events	内部权限失控

附录 B 云原生权限提升漏洞分类与 CWE 对应关系

表 B-1 云原生权限提升漏洞分类框架与 CWE 对应关系
Tab. B-1 Mapping between the classification framework of cloud-native privilege escalation vulnerabilities and CWE

一级分类	二级分类	主要 CWE	CWE 描述
配置缺陷	冗余权限	CWE-269	不当访问控制（通用型）
		CWE-266	权限分配错误
		CWE-732	关键资源权限分配错误
	网络隔离不足	CWE-653	隔离措施不足
		CWE-668	资源暴露至错误安全域
		CWE-669	资源在不同安全域间转移错误
安全逻辑缺陷	访问控制缺陷	CWE-863	授权验证错误
		CWE-284	不当访问控制
		CWE-862	缺失授权机制
	身份验证绕过	CWE-287	不当身份验证
		CWE-290	硬编码密码进行身份验证
		CWE-327	使用存在缺陷或高风险的加密算法
	输入验证不足	CWE-20	不当输入验证
		CWE-74	输出内容中特殊元素的中和处理不当
		CWE-79	Web 页面生成过程中输入的中和处理不当
	授权验证不完善	CWE-285	授权验证不当
CWE-250		以非必要权限执行操作	
CWE-268		特权操作执行不当	
敏感信息泄露	API 端口暴露	CWE-200	敏感信息泄露给未授权主体
		CWE-358	动态识别资源的操作限制不当
		CWE-276	默认权限配置错误
	日志泄露	CWE-532	敏感信息写入日志文件
		CWE-214	存储/传输前敏感信息清除不当
		CWE-497	敏感系统信息泄露至未授权控制域
	配置文件泄露	CWE-922	敏感信息存储不安全
		CWE-200	敏感信息泄露给未授权主体
		CWE-270	权限上下文切换错误
代码注入	命令注入	CWE-78	操作系统命令中特殊元素的中和处理不当
		CWE-94	代码生成控制不当
		CWE-601	URL 重定向至不可信站点
	路径遍历	CWE-22	路径名限制不当，未限定在指定目录
		CWE-36	绝对路径遍历
		CWE-74	输出内容中特殊元素的中和处理不当

KubeGuard 权限提升检测规则汇总

本节补充给出前期工作 KubeGuard 在四类权限提升场景下采用的代表性检测规则汇总。

表 B-2 多种权限提升类型及其检测规则
Tab. B-2 Types of privilege escalation and their detection rules

类型	机制	检测规则
凭证窃取	密钥访问	检测请求的源 IP 地址或服务账户与资源所属 Pod 不一致的 API 请求
	Pod 执行	
	容器逃逸	
账户冒用	服务账户模拟	检测服务账户、操作和资源的组合是否属于该账户的正常使用行为集合
RBAC 操纵	角色修改	检测对角色或角色绑定资源的创建、修补或更新操作
	角色绑定修改	
间接执行	工作负载修改	检测对工作负载资源的创建、修补或更新操作，以及对 Pod 的 exec 操作
	Pod 执行	

附录 C CVE202224768 复现步骤与策略清单

C.1 漏洞复现详细步骤

步骤 1: 部署漏洞版本 Argo CD (v2.3.1)

```
kubectl create namespace argocd
kubectl apply -n argocd -f
↪ https://raw.githubusercontent.com/argoproj/argo-cd/v2.3.1/manifests/install.yaml
kubectl rollout status deployment/argocd-server -n argocd
```

步骤 2: 获取初始管理员密码并登录 UI/CLI

```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" |
↪ base64 -d && echo
kubectl port-forward svc/argocd-server -n argocd 8080:443
```

步骤 3: 创建宽松的 AppProject (用于演示风险)

```
cat <<EOF | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: demo-project
  namespace: argocd
spec:
  destinations:
  - namespace: '*'
    server: '*'
  sourceRepos:
  - '*'
  clusterResourceWhitelist:
  - group: '*'
    kind: '*'
EOF
```

步骤 4: 创建低权用户 bob, 并授予其对目标 Application 的 sync 与 override 权限

```
policy.default: role:readonly
policy.csv: |
  p, role:syncer, applications, sync, demo-project/*, allow
  p, role:syncer, applications, override, demo-project/*, allow
  g, bob, role:syncer
```

```
accounts.bob: apiKey, login
```

```
kubectl rollout restart deployment/argocd-server -n argocd
```

步骤 5: 创建 Application 指向恶意仓库

```
cat <<EOF | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: bob-app
  namespace: argocd
spec:
  project: demo-project
  source:
```

```
repoURL: https://github.com/Ivanbeethoven/bad_repo
path: .
targetRevision: HEAD
destination:
  server: https://kubernetes.default.svc
  namespace: default
syncPolicy:
  automated:
    prune: true
    selfHeal: true
EOF
```

步骤 6: 模拟 bob 触发同步

```
argocd login localhost:8080 --username bob --password <token-or-password>
argocd app sync bob-app --force
```

步骤 7: 验证权限提升

```
kubectl auth can-i '*' '*' --as bob
```

C.2 策略代码清单

Kubernetes RBAC: 收敛 AppProject 的写权限

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: argocd-appproject-edit
rules:
- apiGroups: ["argoproj.io"]
  resources: ["appprojects"]
  verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: argocd-appproject-readonly
rules:
- apiGroups: ["argoproj.io"]
  resources: ["appprojects"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: argocd-appproject-edit-platform-admins
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: argocd-appproject-edit
subjects:
- kind: Group
  name: argo:platform-admin
  apiGroup: rbac.authorization.k8s.io
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: argocd-appproject-readonly-tenants
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: argocd-appproject-readonly
subjects:
- kind: Group
  name: argo:tenant-readonly
  apiGroup: rbac.authorization.k8s.io
```

Argo CD RBAC: 默认只读 + 高风险能力隔离

```
apiVersion: v1
kind: ConfigMap
```

```

metadata:
  name: argocd-rbac-cm
  namespace: argocd
data:
  policy.csv: |
    g, argo:platform-admin, role:admin
    g, argo:tenant-readonly, role:readonly

    p, role:admin, projects, *, *, allow
    p, role:admin, repositories, *, *, allow
    p, role:admin, clusters, *, *, allow
    p, role:admin, applications, *, *, allow
    p, role:admin, exec, *, *, allow
    p, role:admin, logs, *, *, allow

    p, role:readonly, applications, get, *, allow
    p, role:readonly, repositories, get, *, allow
    p, role:readonly, projects, get, *, allow
    p, role:readonly, clusters, get, *, allow
    p, role:readonly, logs, get, *, allow
    p, role:readonly, exec, *, *, deny
policy.default: role:readonly

```

准入控制：Gatekeeper（推荐）

```

apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8sargocdappprojectpolicies
spec:
  crd:
    spec:
      names:
        kind: K8sArgoCDAppProjectPolicies
      validation:
        openAPIV3Schema:
          properties:
            enforceProjPrefix:
              type: boolean
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package k8sargocdappprojectpolicies

        trim_space(s) = out {
          out := regex.replace("^\\s+|\\s+$", "", s)
        }

        # 1) 禁止出现 "..., *, *, allow" 的全局通配符放权
        deny[msg] {
          input.review.kind.group == "argoproj.io"
          input.review.kind.kind == "AppProject"
          roles := input.review.object.spec.roles
          some i
          policies := roles[i].policies
          some j
          policy := policies[j]
          regex.match(".*,\\s*\\*,\\s*\\*,\\s*allow\\s*$", policy)
          msg := sprintf("AppProject policy contains global wildcard allow: %s", [policy])
        }

        # 2) 要求 subject 采用 proj:<project>:<role> 前缀（可选开关）
        deny[msg] {
          input.review.kind.group == "argoproj.io"
          input.review.kind.kind == "AppProject"
          input.parameters.enforceProjPrefix == true
          ap := input.review.object.metadata.name

          roles := input.review.object.spec.roles
          some i
          policies := roles[i].policies
          some j
          policy := policies[j]

          startswith(policy, "p,")
          parts := split(policy, ",")
          count(parts) >= 2
          subject := trim_space(parts[1])
          not startswith(subject, sprintf("proj:%s:", [ap]))
          msg := sprintf("AppProject policy subject must start with 'proj:%s:' %s", [ap,
            ↵ policy])
        }

        # 3) 禁止授予 projects/repositories/clusters/exec 的写权限
        deny[msg] {
          input.review.kind.group == "argoproj.io"
          input.review.kind.kind == "AppProject"

```

```

roles := input.review.object.spec.roles
some i
policies := roles[i].policies
some j
policy := policies[j]
regex.match("^p,.*(projects|repositories|clusters|exec),\\s*(create|update|patch|de|
↪ lete|\\s*),.*(allow)\\s*$", policy)
msg := sprintf("AppProject policy grants write on global resource: %s", [policy])
}

```

(2) Constraint

```

apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sArgoCDAppProjectPolicies
metadata:
  name: restrict-argocd-appproject-policies
spec:
  match:
    kinds:
      - apiGroups: ["argoproj.io"]
        kinds: ["AppProject"]
  parameters:
    enforceProjPrefix: true

```

准入控制：Kyverno（替代方案）

```

apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: restrict-argocd-appproject-policies
spec:
  validationFailureAction: Audit
  background: true
  rules:
    - name: forbid-global-wildcard-allow
      match:
        any:
          - resources:
              kinds:
                - argoproj.io/v1alpha1/AppProject
      validate:
        message: "forbid global wildcard allow in AppProject role policies"
        foreach:
          - list: "request.object.spec.roles[].policies[]"
            deny:
              conditions:
                any:
                  - key: "{{ regex_match('.*\\s*\\s*\\s*\\s*\\s*allow\\s*$', element) }}"
                    operator: Equals
                    value: true
    - name: enforce-proj-prefix
      match:
        any:
          - resources:
              kinds:
                - argoproj.io/v1alpha1/AppProject
      validate:
        message: "policy subject must use proj:<project>:<role> prefix"
        foreach:
          - list: "request.object.spec.roles[].policies[]"
            deny:
              conditions:
                all:
                  - key: "{{ regex_match('^p,\\s*proj:[^:]+:[^,]+$', element) }}"
                    operator: Equals
                    value: false
    - name: forbid-global-resource-write
      match:
        any:
          - resources:
              kinds:
                - argoproj.io/v1alpha1/AppProject
      validate:
        message: "forbid write on projects/repositories/clusters/exec in AppProject policies"
        foreach:
          - list: "request.object.spec.roles[].policies[]"
            deny:
              conditions:
                any:
                  - key: "{{ regex_match('^p,.*(projects|repositories|clusters|exec),\\s*(create|
↪ update|patch|delete|\\s*),.*(allow)\\s*$', element) }}"

```

```
operator: Equals  
value: true
```


附录 D CVE-2025-2241 复现步骤与策略清单

D.1 漏洞复现详细步骤

前提条件

- 已部署 ACM/MCE 与 Hive，并启用 vSphere 集群供应能力（具体版本与安装方式依赖环境，以厂商公告与实验环境为准）；
- 至少存在一个“只读身份”。该身份不具备 secrets 读取权限，但具备 clusterprovisions.hive.openshift.io 的 get/list/watch 权限（该条件用于验证旁路泄露）。

步骤 1：触发供应后，用低权身份读取 ClusterProvision 对象

```
# 低权用户自检 (Kubernetes 通用)
kubectl auth can-i get clusterprovision --as=<user> --all-namespaces
kubectl auth can-i get secrets --as=<user> --all-namespaces

# 导出对象（名称可能带随机后缀，按实际替换）
kubectl get clusterprovision -n your-namespace
kubectl get clusterprovision <name> -n your-namespace -o yaml
```

D.2 策略代码清单

1) 立刻收敛 ClusterProvision 的读取权限 (RBAC)

```
oc adm policy who-can get clusterprovision
oc adm policy who-can list clusterprovision
oc adm policy who-can watch clusterprovision
```

2) 对读取行为启用审计并联动告警

```
apiVersion: audit.k8s.io/v1
kind: Policy
rules:
- level: Metadata
  verbs: ["get", "list", "watch"]
  resources:
  - group: "hive.openshift.io"
    resources: ["clusterprovisions"]
  omitStages: ["RequestReceived"]
```

3) 缩短暴露窗口：供应完成后清理 ClusterProvision

```
oc get clusterprovision -n <ns>
oc delete clusterprovision -n <ns> <name>
```

4) ”敏感字段守护”检测 (Gatekeeper/Kyverno, 建议审计模式)

```
sensitive_key(k) {
  lk := lower(k)
  contains(lk, "password")
}
sensitive_key(k) {
```

```

    lk := lower(k)
    contains(lk, "token")
  }
  sensitive_key(k) {
    lk := lower(k)
    contains(lk, "secret")
  }
  sensitive_key(k) {
    lk := lower(k)
    contains(lk, "credential")
  }

  # 深度遍历 object, 收集所有键
  walk(obj, path, val) {
    is_object(obj)
    some k
    v := obj[k]
    walk(v, array.concat(path, [k]), val)
  }
  walk(obj, path, val) {
    is_array(obj)
    some i
    v := obj[i]
    walk(v, array.concat(path, [sprintf("%v", [i])]), val)
  }
  walk(obj, path, obj) {
    not is_object(obj)
    not is_array(obj)
  }

  deny[msg] {
    input.review.kind.group == "hive.openshift.io"
    input.review.kind.kind == "ClusterProvision"
    walk(input.review.object, [], _)
    some k
    _ = input.review.object[k]
    sensitive_key(k)
    msg := sprintf("ClusterProvision contains sensitive key at top-level: %s", [k])
  }

  deny[msg] {
    input.review.kind.group == "hive.openshift.io"
    input.review.kind.kind == "ClusterProvision"
    walk(input.review.object, path, _)
    some i
    k := path[i]
    sensitive_key(k)
    msg := sprintf("ClusterProvision contains sensitive key on path %v", [path])
  }
}
---
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sClustProvKeySensitive
metadata:
  name: audit-clusterprovision-sensitive-keys
spec:
  enforcementAction: dryrun
  match:
    kinds:
      - apiGroups: ["hive.openshift.io"]
        kinds: ["ClusterProvision"]

```


附录 E CVE-2023-30512 复现步骤与策略清单

E.1 漏洞复现详细步骤

变量约定（按实际替换）

```

NAMESPACE=<cubefs-namespace>
SA=cfs-csi-service-account
CLUSTERROLE=cfs-csi-cluster-role

# 如 SA 名称不同, 请先定位:
kubectl get sa -A | findstr /i "csi cfs cubefs"
kubectl get clusterrole,clusterrolebinding -A | findstr /i "csi cfs cubefs"

```

步骤 1：导出 RBAC 证据（确认 secrets 权限存在）

```

kubectl get clusterrole $CLUSTERROLE -o yaml
kubectl get clusterrolebinding -o yaml | findstr /i "cfs-csi cubefs"

# 重点检查 ClusterRole 规则中是否出现:
# resources: ["secrets"]
# verbs: ["get","list","watch"]

```

步骤 2：受控权限验证（不执行实际数据提取）

```

# 直接验证该 SA 是否具备跨命名空间读取 secrets 的能力
kubectl auth can-i get secrets --as=system:serviceaccount:$NAMESPACE:$SA --all-namespaces
kubectl auth can-i list secrets --as=system:serviceaccount:$NAMESPACE:$SA --all-namespaces
kubectl auth can-i watch secrets --as=system:serviceaccount:$NAMESPACE:$SA --all-namespaces

```

权限提升路径

```

CubeFS CSI 安装环境
|
+--> DaemonSet: cfs-csi-node (节点插件)
|
+--> Deployment: cfs-csi-controller (控制器)
|
v
ServiceAccount: cfs-csi-service-account
|
v
ClusterRole: cfs-csi-cluster-role
(误包含 secrets 的 get/list/watch)
|
v
一旦攻击者控制节点或 CSI Pod => 可滥用该 SA 的身份边界
(风险: 读取高价值凭据 -> 演化为集群级权限提升)

```

E.2 策略代码清单

1) RBAC 最小化：移除 ClusterRole 中的 secrets 权限

```

cat <<'EOF' | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: cfs-csi-cluster-role
  labels:
    app.kubernetes.io/name: cfs-csi
rules:
- apiGroups: [""]
  resources: ["nodes","persistentvolumes","pods"]
EOF

```

```

    verbs: ["get", "list", "watch"]
-   apiGroups: ["storage.k8s.io"]
    resources: ["volumeattachments", "csinodes"]
    verbs: ["get", "list", "watch"]
# 注意: 不应包含 resources:["secrets"]
EOF

# 复测 (期望输出 no)
kubectl auth can-i list secrets --as=system:serviceaccount:$NAMESPACE:$SA --all-namespaces

```

2) 准入防回归 (OPA Gatekeeper): 禁止创建包含 secrets 读取动词的 ClusterRole

```

has_secrets_rule(rule) {
    rule.resources[_] == "secrets"
    rule.verbs[_] == "get"
}
has_secrets_rule(rule) {
    rule.resources[_] == "secrets"
    rule.verbs[_] == "list"
}
has_secrets_rule(rule) {
    rule.resources[_] == "secrets"
    rule.verbs[_] == "watch"
}

violation[{"msg": msg}] {
    input.review.kind.kind == "ClusterRole"
    some i
    rule := input.review.object.rules[i]
    has_secrets_rule(rule)
    msg := "ClusterRole must not grant get/list/watch on secrets"
}
EOF

cat <<'EOF' | kubectl apply -f -
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sDenySecretsRBAC
metadata:
  name: deny-clusterrole-secrets-read
spec:
  enforcementAction: dryrun
  match:
    kinds:
    - apiGroups: ["rbac.authorization.k8s.io"]
      kinds: ["ClusterRole"]
EOF

```

3) 审计与检测: 对 secrets 枚举行为进行可观测化

```

apiVersion: audit.k8s.io/v1
kind: Policy
rules:
- level: Metadata
  verbs: ["get", "list", "watch"]
  resources:
  - group: ""
    resources: ["secrets"]
  omitStages: ["RequestReceived"]

```

附录 F CVE-2020-4062 复现步骤与策略清单

F.1 漏洞复现详细步骤

前提条件

- 集群 CNI 支持 NetworkPolicy，例如 Calico。
- Helm v3 可用。
- 受影响的 Conjur OSS Helm Chart 版本为 < 2.0.0。

步骤 1：在隔离命名空间部署旧版 Chart，并进行必要的兼容性适配

```
helm repo add cyberark https://cyberark.github.io/helm-charts
helm repo update
helm search repo cyberark/conjur-oss --versions

helm install conjur-oss cyberark/conjur-oss \
  --namespace $NAMESPACE \
  --create-namespace \
  --version <2.0.0>

kubectl -n $NAMESPACE get pods
```

步骤 2：确认 Postgres Service 与 5432 端口暴露

```
NAMESPACE=conjur-vuln

kubectl get ns $NAMESPACE
kubectl -n $NAMESPACE get svc

# 重点确认 Postgres Service（名称以实际为准）与 5432 端口存在
kubectl -n $NAMESPACE get svc | findstr /i "postgres 5432 conjur"
```

步骤 3：确认未配置 NetworkPolicy

```
kubectl -n $NAMESPACE get networkpolicy
```

步骤 4：连通性验证

```
kubectl -n $NAMESPACE run netcheck \
  --image=busybox:1.36 \
  --restart=Never \
  --rm -i -- sh -c "nc -vz $POSTGRES_SVC 5432"
```

F.2 策略代码清单

1) 缓解措施：NetworkPolicy，默认拒绝并仅放通 Conjur Server 访问 Postgres 5432

```
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: conjur-postgres-default-deny
  namespace: conjur-vuln
```

```
spec:
  podSelector:
    matchLabels:
      app: conjur-oss-postgres
  policyTypes:
  - Ingress
  ingress: []
EOF

cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: conjur-postgres-allow-conjur
  namespace: conjur-vuln
spec:
  podSelector:
    matchLabels:
      app: conjur-oss-postgres
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: conjur-oss
    ports:
    - protocol: TCP
      port: 5432
EOF
```

附录 G CVE-2022-24877 复现步骤与策略清单

G.1 漏洞复现详细步骤

参考命令（用于采集受控证据）

```
FLUX_NS=flux-system
APP_NS=<kustomization-namespace>
KS_NAME=<kustomization-name>

kubectl -n $FLUX_NS get deploy kustomize-controller -o wide
kubectl -n $APP_NS get kustomization $KS_NAME -o yaml

# 在隔离仓库中准备异常路径测试样例后，触发一次受控重调和
flux reconcile kustomization $KS_NAME -n $APP_NS --with-source

# 采集控制器日志与事件作为证据
kubectl -n $FLUX_NS logs deploy/kustomize-controller --tail=200
kubectl -n $APP_NS get events --sort-by=.lastTimestamp | findstr /i "Kustomization
↪ Reconciliation"
```

G.2 策略清单

1) 前置校验：CI 流水线拦截异常路径引用

```
# CI 阶段对 kustomization.yaml 进行路径穿越检查
# 以下脚本可嵌入 GitHub Actions / GitLab CI 等流水线
#!/usr/bin/env bash
set -euo pipefail

REPO_ROOT=$(git rev-parse --show-toplevel)
VIOLATIONS=0

# 扫描所有 kustomization.yaml 中的路径字段
for f in $(find "$REPO_ROOT" -name "kustomization.yaml" \
  -o -name "kustomization.yml"); do
  # 检测 resources / patches / components 中的 .. 穿越
  if grep -Pn '\s*(- |path:\s*).*\.\.' "$f"; then
    echo "[BLOCK] 路径穿越检测: $f"
    VIOLATIONS=$((VIOLATIONS + 1))
  fi
done

if [ "$VIOLATIONS" -gt 0 ]; then
  echo "共发现 $VIOLATIONS 处异常路径引用，提交被拦截"
  exit 1
fi
echo "路径校验通过"
```

2) 权限收敛：限制 Flux 源仓库写入与 Kustomization 同步范围

```
# 收紧 Flux GitRepository 与 Kustomization 的 ServiceAccount 权限
cat <<'EOF' | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: flux-kustomize-restricted
  namespace: flux-system
rules:
- apiGroups: ["kustomize.toolkit.fluxcd.io"]
  resources: ["kustomizations"]
  verbs: ["get", "list", "watch"]
- apiGroups: ["source.toolkit.fluxcd.io"]
  resources: ["gitrepositories"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: flux-kustomize-restricted-binding
```

```

namespace: flux-system
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: flux-kustomize-restricted
subjects:
- kind: ServiceAccount
  name: flux-restricted-sa
  namespace: flux-system
EOF

kubectl -n flux-system get role,rolebinding | \
  findstr /i "flux-kustomize-restricted"

```

3) 运行时加固：收紧 kustomize-controller 容器安全上下文

```

# 为 kustomize-controller 添加只读根文件系统与非特权约束
kubectl -n flux-system patch deploy kustomize-controller \
  --type=strategic -p '
spec:
  template:
    spec:
      containers:
      - name: manager
        securityContext:
          readOnlyRootFilesystem: true
          runAsNonRoot: true
          allowPrivilegeEscalation: false
          capabilities:
            drop: ["ALL"]
        volumeMounts:
        - name: tmp
          mountPath: /tmp
      volumes:
      - name: tmp
        emptyDir:
          sizeLimit: 64Mi
'

# 验证安全上下文已生效
kubectl -n flux-system get deploy kustomize-controller \
  -o jsonpath='{.spec.template.spec.containers[0].securityContext}' | \
  python -m json.tool

```

4) 网络隔离：限制 kustomize-controller 出站访问

```

cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: kustomize-controller-egress
  namespace: flux-system
spec:
  podSelector:
    matchLabels:
      app: kustomize-controller
  policyTypes: ["Egress"]
  egress:
  # 仅允许 DNS 解析
  - to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: kube-system
      podSelector:
        matchLabels:
          k8s-app: kube-dns
    ports:
    - protocol: UDP
      port: 53
    - protocol: TCP
      port: 53
  # 仅允许访问 source-controller 拉取制品
  - to:
    - podSelector:
        matchLabels:
          app: source-controller
    ports:
    - protocol: TCP
      port: 9090
  # 允许访问 Kubernetes API Server
  - to:
    - ipBlock:
        cidr: <API-SERVER-CIDR>/32

```

```
ports:
- protocol: TCP
  port: 443
EOF

kubectl -n flux-system get networkpolicy | \
  findstr /i "kustomize-controller"
```


附录 H CVE-2022-29171 复现步骤与策略清单

H.1 漏洞复现详细步骤

参考命令（用于日志与工作负载证据采集）

```
NAMESPACE=sourcegraph

kubectl -n $NAMESPACE get deploy,pod,svc | findstr /i "gitserver"
kubectl -n $NAMESPACE logs deploy/gitserver --tail=200
kubectl -n $NAMESPACE get events --sort-by=.lastTimestamp | findstr /i "gitserver"

# 在管理面触发一次元数据获取/同步后，重复采集日志并与禁用集成后的结果对比
kubectl -n $NAMESPACE logs deploy/gitserver --since=10m | findstr /i "gitolite phabricator
↪  callsign"
```

H.2 策略清单

1) 网络隔离：限制 gitserver 的可达面（默认拒绝并按需放通）

```
NAMESPACE=sourcegraph

# 1) 进站收敛：仅允许 Sourcegraph 内部组件访问 gitserver
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: gitserver-ingress-allow-internal
  namespace: sourcegraph
spec:
  podSelector:
    matchLabels:
      app: gitserver
  policyTypes: ["Ingress"]
  ingress:
    - from:
      - podSelector:
          matchLabels:
            app: sourcegraph-frontend
      - podSelector:
          matchLabels:
            app: repo-updater
    ports:
      - protocol: TCP
        port: 3178
EOF

# 2) 出站收敛：仅放通 DNS；代码托管出口按需另行放通
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: gitserver-egress-dns-only
  namespace: sourcegraph
spec:
  podSelector:
    matchLabels:
      app: gitserver
  policyTypes: ["Egress"]
  egress:
    - to:
      - namespaceSelector:
          matchLabels:
            kubernetes.io/metadata.name: kube-system
        podSelector:
          matchLabels:
            k8s-app: kube-dns
    ports:
      - protocol: UDP
        port: 53
      - protocol: TCP
        port: 53
EOF
```

```
# 验证：策略已创建，且 gitserver 的出站/入站符合预期
kubect1 -n $NAMESPACE get networkpolicy | findstr /i "gitserver"
```

附录 I CVE-2023-22482 复现步骤与策略清单

I.1 漏洞复现详细步骤

参考命令（用于配置、令牌与响应证据采集）

```
ARGO_NS=argocd
ARGOCD_URL=https://argocd.example.com
TOKEN=<oidc-token-with-wrong-aud>

kubectl -n $ARGO_NS get cm argocd-cm -o yaml
kubectl -n $ARGO_NS get deploy argocd-server -o wide

# 保留令牌 payload 作为 aud/iss 证据（必要时补齐 base64 padding）
printf '%s' "$TOKEN" | cut -d. -f2 | tr '_' '/' | base64 -d

# 选择只读接口作为验证点
curl -k -H "Authorization: Bearer $TOKEN" "$ARGOCD_URL/api/v1/applications"
kubectl -n $ARGO_NS logs deploy/argocd-server --since=10m | findstr /i "oidc token aud"
```

I.2 策略清单

权限收敛：RBAC 最小化与项目边界约束

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: argocd-rbac-cm
  namespace: argocd
data:
  policy.default: role:readonly
  policy.csv: |
    p, role:admin, applications, *, *, allow
    p, role:admin, projects, *, *, allow
    p, role:admin, repositories, *, *, allow
    p, role:admin, clusters, *, *, allow

    p, role:readonly, applications, get, *, *, allow
    p, role:readonly, projects, get, *, *, allow
    p, role:readonly, repositories, get, *, *, allow

    p, role:deployer, applications, get, my-project/*, allow
    p, role:deployer, applications, sync, my-project/*, allow
    p, role:deployer, applications, action/*, my-project/*, deny
```


致 谢

作者简历及在学研究成果

一、主要教育经历/工作经历（从大学起，到硕士入学止）

起止年月	学习或工作单位	备注
2019 年 9 月至 2023 年 6 月	在北京科技大学计算机科学与技术专业攻读学士学位	

二、在学期间从事的科研工作

三、在学期间所获的科研奖励

四、在学期间发表的论文

[1] 第二作者（导师一作）.ICWS.CCF-B.2025.

独创性声明

本人声明所呈交的论文是我个人在导师指导下进行的研究工作及取得的
研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包
含其他人已经发表或撰写过的研究成果，也不包含为获得北京科技大学或其
它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所
做的任何贡献均已在论文中作了明确的说明并表示了谢意。

作者签名：_____ 日期：_____ 年_____ 月_____ 日

备注：提交时须有作者亲笔签名！

关于论文使用授权的说明

本人完全了解北京科技大学有关保留、使用学位论文的规定，即：学校有权保留送交论文的复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存论文。

（保密的论文在解密后应遵循此规定）

签名：_____ 导师签名：_____ 日期：_____

学位论文数据集

关键词 *	密级 *	中图分类号 *	UDC	论文资助
Kubernetes 权限提升 提升安全防护	公开			
学位授予单位名称 *		学位授予单位代码 *	学位类别 *	学位级别 *
北京科技大学		10008		
论文题名 *		并列题名		论文语种 *
面向云原生应用权限提升漏洞的智能化防控技术研究				中文
作者姓名 *			学号 *	
培养单位名称 *	培养单位代码 *	培养单位地址	邮编	
北京科技大学	10008	北京市海淀区学院路 30 号	100083	
学科专业 *	研究方向 *	学制 *	学位授予年 *	
论文提交日期 *				
导师姓名 *			职称 *	
评阅人	答辩委员会主席 *	答辩委员会成员		
电子版论文提交格式文本 () 图像 () 视频 () 音频 () 多媒体 () 其他 ()				
推荐格式: application/msword; application/pdf				
电子论文出版 (发布) 者		电子论文出版 (发布) 地		权限声明
论文总页数 *				
共 33 项, 其中带 * 为必填数据, 为 22 项。				